

CARMINDER

Designing a Backend-Centric Architecture for Vehicle Maintenance Systems

Mohammed Ayman Abdulkader Allaw
Ghazi Fida Abu Shaeera
Evan Hanif Belal

SUPERVISOR

Halvard Øysæd

University of Agder, 2026
Faculty of Engineering and Science
Department of Engineering and Sciences

Obligatorisk gruppeerklæring

Den enkelte student er selv ansvarlig for å sette seg inn i hva som er lovlige hjelpemidler, retningslinjer for bruk av disse og regler om kildebruk. Erklæringen skal bevisstgjøre studentene på deres ansvar og hvilke konsekvenser fusk kan medføre. Manglende erklæring fritar ikke studentene fra sitt ansvar.

1.	Vi erklærer herved at vår besvarelse er vårt eget arbeid, og at vi ikke har brukt andre kilder eller har mottatt annen hjelp enn det som er nevnt i besvarelsen.	Ja
2.	Vi erklærer videre at denne besvarelsen: • Ikke har vært brukt til annen eksamen ved annen avdeling/universitet/høgskole innenlands eller utenlands. • Ikke refererer til andres arbeid uten at det er oppgitt. • Ikke refererer til eget tidligere arbeid uten at det er oppgitt. • Har alle referansene oppgitt i litteraturlisten. • Ikke er en kopi, duplikat eller avskrift av andres arbeid eller besvarelse.	Ja
3.	Vi er kjent med at brudd på ovennevnte er å betrakte som fusk og kan medføre annullering av eksamen og utestengelse fra universiteter og høyskoler i Norge, jf. Universitets- og høyskoleloven §§4-7 og 4-8 og Forskrift om eksamen §§ 31.	Ja
4.	Vi er kjent med at alle innleverte oppgaver kan bli plagiatkontrollert.	Ja
5.	Vi er kjent med at Universitetet i Agder vil behandle alle saker hvor det forligger mistanke om fusk etter høyskolens retningslinjer for behandling av saker om fusk.	Ja
6.	Vi har satt oss inn i regler og retningslinjer i bruk av kilder og referanser på biblioteket sine nettsider.	Ja
7.	Vi har i flertall blitt enige om at innsatsen innad i gruppen er merkbart forskjellig og ønsker dermed å vurderes individuelt. Ordinært vurderes alle deltakere i prosjektet samlet.	Ja

Publiseringsavtale

Fullmakt til elektronisk publisering av oppgaven Forfatter(ne) har opphavsrett til oppgaven. Det betyr blant annet enerett til å gjøre verket tilgjengelig for allmennheten (Åndsverkloven. §2).

Oppgaver som er unntatt offentlighet eller taushetsbelagt/konfidensiell vil ikke bli publisert.

Vi gir herved Universitetet i Agder en vederlagsfri rett til å gjøre oppgaven tilgjengelig for elektronisk publisering:	Ja
Er oppgaven båndlagt (konfidensiell)?	Nei
Er oppgaven unntatt offentlighet?	Nei

Acknowledgements

We would like to express our sincere gratitude to our supervisor, Halvard Øysæd, for his guidance, support, and valuable feedback throughout this project. His expertise and constructive comments have been essential to the development and improvement of our work.

We would also like to thank the University of Agder for providing a strong academic foundation and a high-quality learning environment over the past three years. The knowledge and skills we have gained during our studies have been crucial in completing this bachelor's thesis.

Furthermore, we would like to thank our fellow students and everyone who supported and encouraged us during the project period. Your input, discussions, and motivation have been greatly appreciated.

Finally, we would like to thank our families and friends for their patience, understanding, and continuous support throughout our studies.

Abstract

Vehicle maintenance data is often fragmented across heterogeneous sources, leading to inconsistent interpretation and limited cross-platform usability. This thesis investigates how domain logic can be centralized in a modular backend architecture while maintaining thin, independent client applications.

A prototype system, CarMinder, was implemented using a layered FastAPI backend and two independently developed clients. The architecture was validated through automated conformance testing, behavioural unit testing of the domain logic, and external schema-based contract testing of the deployed API.

The results demonstrate that enforcing strict module boundaries combined with automated conformance tooling provides verifiable guarantees of architectural consistency. Centralizing domain logic in the backend ensures deterministic behaviour and eliminates duplication across clients. These findings suggest that backend-centric architectures are a viable approach for systems requiring consistent interpretation across multiple platforms.

Contents

Acknowledgements	ii
Abstract	iii
List of Figures	viii
List of Tables	x
1 Introduction	1
1.1 Background	1
1.2 Motivation	1
1.3 Problem Context and Limitations	2
1.4 Problem Statement	2
1.5 Use of AI assistance	2
2 Theoretical Background	3
2.1 Vehicle Maintenance as Lifecycle Data	3
2.2 Maintenance Strategies as Event Types	4
2.3 Heterogeneous Inputs and the Need for Normalization	4
2.4 Event-Based Histories, Data Integrity, and Traceability	5
2.5 Deterministic Interpretation and Safe Request Handling	5
2.6 Modular Backend Architecture and Layered Responsibility	5
2.7 Client–Server and Thin Client Principles	6
2.8 RESTful API Principles as System Boundary	6
2.9 Related Work	6
2.9.1 MyAutoLog	7
2.9.2 Simply Auto	7
2.9.3 CARFAX Car Care	7
2.9.4 FIXD	7
3 Method	9
3.1 Research Strategy	9
3.2 Deriving Architectural Requirements	9
3.3 Design Method	10
3.3.1 Adopting a Layered Modular Backend	10
3.3.2 Backend-First Development Sequencing	10
3.3.3 Enforcing Thin Clients Through Independent Development	11
3.3.4 Isolating the Scoring Engine as a Pure Domain Module	11
3.4 Validation Strategy	11
3.4.1 Architectural Conformance Testing	11
3.4.2 Behavioural Testing of Domain Logic	11
3.4.3 Thin Client Validation	12

3.4.4	External API Contract Testing	12
3.4.5	Mutation Testing of Scoring Logic	13
3.5	Development Methodology	13
3.6	Requirements Elicitation	13
3.7	Method Limitations	14
4	Results	15
4.1	The CarMinder Artefact	15
4.1.1	System Architecture	15
4.1.2	Backend Internal Structure	16
4.1.3	Client Applications	17
4.1.4	External Integrations	17
4.2	Backend Implementation	18
4.2.1	Database Schema	18
4.2.2	OBD-II Reading Pipeline	19
4.2.3	Service Interval and Urgency Engine	20
4.2.4	Vehicle Lookup Integration	20
4.2.5	Notifications	21
4.2.6	PDF Export	21
4.3	Client Applications	21
4.3.1	Authentication	22
4.3.2	Car Management	22
4.3.3	Car Care Score	22
4.3.4	Maintenance Events	23
4.3.5	Incident Reports	23
4.3.6	Calendar	23
4.3.7	Public History Lookup	23
4.3.8	Transfer Ownership	24
4.3.9	Vehicle Retirement	24
4.3.10	Platform-Specific Features	25
4.4	Validation Results	26
4.4.1	R1: Centralized Domain Logic	26
4.4.2	R2: Stable API Contract	26
4.4.3	R3: Modular Backend Structure	28
4.4.4	External API Contract Testing	30
4.4.5	Mutation Testing	31
4.4.6	Requirement Traceability Summary	32
5	Discussion	34
5.1	Answering the Research Question	34
5.2	Validity of the Validation Approach	35
5.3	Architectural Trade-offs and Limitations	36
5.4	Security Trade-offs	38
5.5	Generalizability of the Approach	40
6	Social Perspective	41
6.1	Societal and Environmental Perspective	41
6.1.1	Environmental implications of maintenance transparency	41
6.1.2	Transparency, accident reporting, and trust in the second-hand market	41
7	Conclusions	43
7.1	Summary of Work	43
7.2	Conclusion	43

7.3	Limitations	44
7.4	Future Work	44
A	Selected Test Units	46
A.0.1	Thin-client conformance tests	46
A.0.2	Scoring engine tests	48
A.0.3	API contract tests	49
A.0.4	Import-linter contract	49
A.0.5	AST-based architecture helpers	51
A.1	Mutation Testing Artifacts	54
A.1.1	Reproduction Command	54
A.1.2	Killed Mutant Example	54
A.1.3	Survived Mutant Example	55
A.1.4	Mutation Testing Wrapper Script	55
A.1.5	Per-File Mutation Breakdown	55
B	Client Application Screenshots	56
B.1	Vehicle Retirement Screenshots	57
B.2	Notifications	58
B.3	PDF Export	59
B.4	Authentication	60
B.5	Car Management	63
B.6	Car Care Score	65
B.7	Maintenance Events	66
B.8	Incident Reports	67
B.9	Calendar	68
B.10	Public History Lookup	69
B.11	Web Fleet Dashboard	70
B.12	Vehicle Ownership Transfer Interface	71
B.13	OBD-II Real-Device Scanning	72
B.14	OCR Receipt Scanner	75
C	User Stories	76
D	Generated API Documentation	78
D.1	Generated API Documentation	78
E	Self Assessments	80
E.1	Evan Self assessment and Timesheet	80
E.2	Ghazi Self assessment and Timesheet	90
E.3	Ayman Self assessment and Timesheet	94
E.3.1	My Contributions	94
E.3.2	Strengths	94
E.3.3	Challenges	95
E.3.4	Learning Outcomes	95
F	Git-log	99
F.1	Git-log for the project	99
F.2	Exported Git-log	99
F.3	Schemathesis Commands	102
	Bibliography	104

List of Figures

2.1	Lifecycle-oriented view of vehicle maintenance.	4
4.1	CarMinder system architecture showing the three-tier structure of thin clients, a centralized backend API, and supporting external services.	15
4.2	Sequence diagrams for protected API request flows: success, not found, and authentication failure.	17
4.3	CarMinder database schema and entity relationships.	18
4.4	OBD-II reading pipeline from mobile capture to backend upload and client retrieval.	20
4.5	Distribution of category weights in the car care scoring engine.	22
4.6	Vehicle ownership transfer flow (code generation and claim process).	24
4.7	Thin-client conformance test results showing all four tests passed, confirming that no domain logic is duplicated in either client.	27
4.8	API integration test results showing all 13 tests passed using FastAPI's <code>TestClient</code> , exercising the full request path without a running server.	27
4.9	Sequence diagrams demonstrating thin-client behaviour across event type retrieval and score consumption. Left: both clients fetch valid event types from the backend at runtime rather than maintaining a local enumeration. Right: both clients consume the complete scoring result from the backend without implementing any part of the scoring, confidence, or recommendation logic. In both flows the client is responsible only for presentation; all domain rules remain in the backend.	28
4.10	Import-linter results showing that all seven architectural contracts were kept.	29
4.11	AST-based architecture test results showing that all eight tests passed.	29
4.12	Pytest results showing that all 20 tests passed successfully.	29
4.13	Schemathesis final validation run against the deployed Railway API after corrective changes. All 36 operations passed across the Coverage, Fuzzing, and Stateful phases, with 4,759 generated test cases and zero contract violations. The three remaining warnings concern test inputs rather than contract correctness.	31
5.1	Local Storage inspection of the CarMinder web client, showing Firebase ID token stored under the key <code>auth_token</code>	39
B.1	Destructive confirmation step preceding the call to <code>POST /cars/{car_id}/retire</code> on the mobile (left) and web (right) clients. Both clients display the same warning text before invoking the backend retirement endpoint.	57
B.2	Test push notification displayed on the mobile client, confirming successful delivery via the backend notification system. Referenced from the Notifications subsection of Chapter 4.	58
B.3	PDF export output on both platforms. The document is generated entirely by the backend; the client only triggers the request and receives the file.	59
B.4	Login screens on the mobile (left) and web (right) clients.	60

B.5	Apple login	61
B.6	Registration screens on the mobile (left) and web (right) clients.	62
B.7	Add car flow on the mobile (left) and web (right) clients, both consuming the same backend vehicle lookup endpoint.	63
B.8	Vehicle list on the mobile profile tab (left) and web profile page (right). . . .	64
B.9	Car Care Score on the mobile (left) and web (right) clients. Both render the same backend response without client-side computation.	65
B.10	Add maintenance event form on the mobile (left) and web (right) clients, both submitting to the same backend endpoint.	66
B.11	Report incident form on the mobile (left) and web (right) clients.	67
B.12	Calendar view on the mobile (left) and web (right) clients.	68
B.13	Public history lookup on the mobile (left) and web (right) clients, both consuming the same unauthenticated backend endpoint.	69
B.14	Web home dashboard showing fleet-level service overview, summary cards, and per-vehicle care score and urgency status.	70
B.15	Vehicle ownership transfer interface on mobile and web. Left: mobile transfer code generation. Centre: web transfer code generation. Right: web claim car dialog for the recipient. The underlying flow is illustrated in Figure 4.6 in Chapter 4.	71
B.16	Real-device OBD-II scanning on two vehicles. Left: Toyota Yaris scan showing RPM, coolant, speed, engine load, battery voltage, and two unrecognised DTCs. Right: metric trend charts from the Mazda 3 across the test period. .	72
B.17	Fuel-related OBD-II metrics on the mobile diagnostics screen. Left: live gauges including fuel consumption and fuel rate. Right: historical charts for fuel consumption, fuel rate, and mass air flow.	73
B.18	OBD-II diagnostics view on the web client, showing live metrics including fuel consumption, fuel rate, and mass air flow, with corresponding trend charts for the Mazda 3 across 80 stored readings.	74
B.19	OCR receipt scanning on the mobile client. The captured image is uploaded to the backend, which performs Tesseract-based extraction and returns candidate fields for the user to review before submission.	75
D.1	Generated OpenAPI/Swagger documentation for the CarMinder backend API, showing grouped endpoints and authentication-protected routes.	79

List of Tables

2.1	Summary of reviewed applications across observable dimensions relevant to the CarMinder research question. CarMinder row reflects design goals, not commercially verified features.	7
4.1	Car care score category weights.	23
4.2	Schemathesis baseline run against the deployed API.	30
4.3	Mutation testing results for the backend scoring domain.	32
4.4	Requirement traceability summary mapping each architectural requirement to its validation method and result.	33
A.1	Mutation results broken down by source file.	55

Chapter 1

Introduction

1.1 Background

Modern vehicle ownership generates a substantial amount of maintenance-related data over time, including service receipts, diagnostic fault codes, inspection reports, and user-recorded events. This information is critical for assessing vehicle reliability, planning preventive maintenance, estimating total cost of ownership, and establishing trust during resale.

In practice, however, vehicle maintenance history is highly fragmented. Records are commonly distributed across physical receipts, email attachments, proprietary workshop systems, and ad-hoc user notes. These data sources vary widely in structure, format, and longevity, making long-term aggregation and interpretation difficult. As a result, vehicle owners often lack a coherent and reliable overview of their vehicle’s maintenance state.

1.2 Motivation

The lack of a unified and verifiable maintenance history introduces several challenges. Vehicle owners struggle to understand the mechanical relevance and urgency of maintenance recommendations and diagnostic warnings. In private vehicle transactions, sellers are often unable to credibly demonstrate proper maintenance adherence, leading to reduced resale value and increased information asymmetry between buyers and sellers.

Existing automotive applications typically focus on isolated functionality such as expense tracking, diagnostic code lookup, or manufacturer-specific service reminders. These systems often present functionality through platform-specific or vendor-controlled clients, making it difficult to assess, reuse, or independently extend their domain logic outside the provided application ecosystem. As vehicle software ecosystems continue to diversify across mobile, web, and emerging platforms, these limitations become increasingly problematic.

Beyond the technical challenge of unifying fragmented maintenance data, the problem also has a broader societal dimension. McLuhan [1] argues that the significance of a technology lies not in its content, but in how it changes the scale and pattern of human activity and association. Seen in that light, a system such as CarMinder is not merely a tool for storing vehicle information; it may also reshape how vehicle owners document maintenance, how buyers evaluate used cars, and how trust is established in private transactions.

1.3 Problem Context and Limitations

From a software engineering perspective, the core challenge lies in managing unclear and partially unstructured maintenance data while ensuring consistent interpretation across multiple client platforms. Maintenance evidence may originate from noisy inputs such as scanned receipts, asynchronous diagnostic telemetry, or manually entered records.

Many existing solutions expose functionality primarily through specific client applications or vendor-controlled ecosystems, making it difficult to introduce independent clients or new platforms without relying on vendor-provided interfaces. Furthermore, inconsistent data processing pipelines undermine determinism and traceability, reducing trust in derived maintenance assessments.

These limitations highlight the need for a centralized, platform-agnostic system architecture in which domain logic is consolidated and authoritative, while client applications remain lightweight and replaceable.

1.4 Problem Statement

Modern vehicle maintenance data is fragmented across unclear and partially unstructured sources. This fragmentation complicates consistent interpretation, long-term verification, and cross-platform system integration.

This thesis investigates the following research question:

How can an automotive maintenance system be architected with a modular backend that centralizes ingestion, normalization, and domain reasoning logic, while allowing multiple client applications to operate as thin presentation layers?

1.5 Use of AI assistance

This thesis made use of AI-assisted tools at specific points in the writing and development process. The following describes how these tools were used.

Writing assistance. ChatGPT [2] was used to check spelling, grammar, and unclear phrasing in selected sections of the thesis.

Diagram generation. ChatGPT [2] was used to convert a hand-drawn sequence diagram into a TikZ figure such as Figure 4.6.

Development assistance. Claude [3] was used during development to suggest test-suite structure, assist with debugging, identify logically inconsistent code, and generate code suggestions for selected implementation tasks. These suggestions were not used uncritically. All AI-generated code and test proposals were reviewed, understood, modified where necessary, and validated by the authors before being integrated into the project. AI tools did not independently design the system architecture or define the domain logic; final responsibility for all architectural decisions, implementation choices, and validation remained with the authors.

Code documentation and library support. AI tools were used as a reference when understanding library APIs, framework documentation, and error messages, as an alternative to traditional web searches. In all cases, the authors reviewed, understood, and integrated any suggested fixes.

Chapter 2

Theoretical Background

This chapter defines the concepts required to design and justify a backend-centric, platform-independent vehicle maintenance system. The focus is on (i) vehicle maintenance data as a long-lived lifecycle history, (ii) heterogeneous data ingestion and normalization, (iii) event-based representations supporting data integrity, traceability, and deterministic interpretation, and (iv) modular backend architecture enabling thin client applications.

2.1 Vehicle Maintenance as Lifecycle Data

Vehicle maintenance can be modeled as a lifecycle process spanning the entire operational lifetime of a vehicle. Across acquisition, ownership, and resale or disposal, a vehicle accumulates maintenance-relevant evidence such as inspections, scheduled services, repairs, component replacements, diagnostic observations, and owner-reported issues. Taken together, these events represent the evolving technical condition of the vehicle over time [4].

A lifecycle perspective emphasizes that individual maintenance actions gain meaning primarily in relation to historical context. Preventive actions influence future reliability and cost, while deferred maintenance increases the likelihood of degradation and unplanned repairs. Consequently, maintenance history is best treated as a cumulative record rather than isolated transactions [4].

Traditional practices often store maintenance evidence as fragmented entries (e.g., paper receipts, workshop-specific records, and proprietary digital service books). Fragmentation reduces the ability to aggregate and interpret long-term trends, compare observations over time, and reuse evidence when ownership changes. A digital system aligned with lifecycle principles therefore requires a structured representation of maintenance history that can persist, be extended, and be interpreted consistently over long time horizons [4].

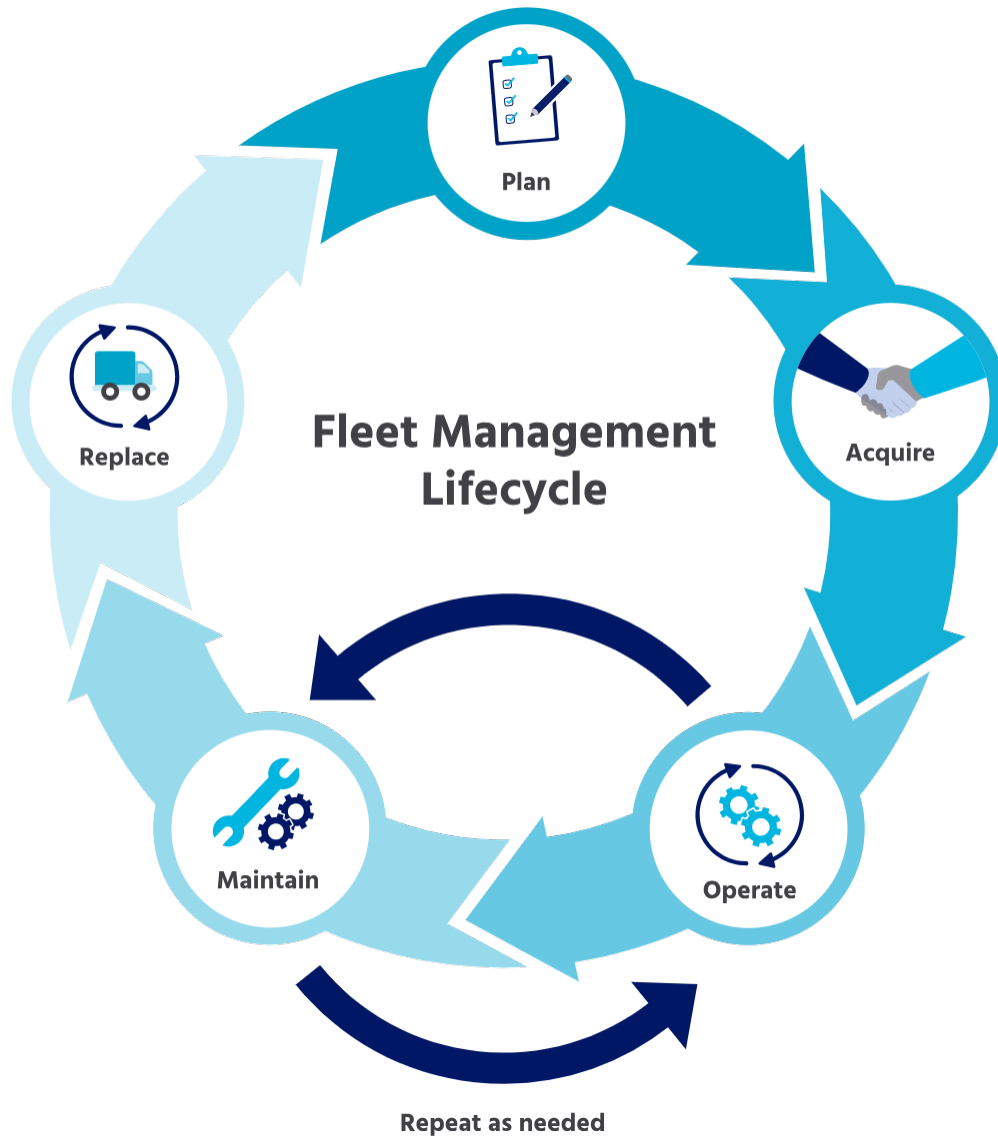


Figure 2.1: Lifecycle-oriented view of vehicle maintenance.

2.2 Maintenance Strategies as Event Types

Maintenance engineering commonly distinguishes between preventive and corrective maintenance. **Preventive maintenance** consists of planned interventions intended to reduce the probability of failure, typically scheduled by time or mileage (e.g., oil changes, inspections, timing belt replacement). **Corrective maintenance** refers to unplanned repairs performed after faults or failures occur (e.g., alternator replacement, sensor failure repair) [5].

For information systems, this distinction is useful as a classification dimension for events and as a basis for analysis and interpretation rules. A complete history commonly contains both types; the objective is not to eliminate corrective events, but to ensure all events are recorded as verifiable evidence and interpreted consistently in context [5].

2.3 Heterogeneous Inputs and the Need for Normalization

Vehicle maintenance data is heterogeneous in both source and structure. Examples include semi-structured workshop invoices, unstructured text notes, images of receipts processed by OCR, and machine-generated diagnostic codes. Heterogeneity introduces variability in

terminology, granularity, and completeness, and increases the risk of inconsistent downstream interpretation if raw inputs are used directly [6].

Normalization is the process of converting different types of input data into a common internal format. In a vehicle maintenance system, this means that data from sources such as receipts, diagnostic codes, and user-entered notes is stored using the same fields, units, and meanings [6]. This common schema makes it easier to validate records, compare events over time, and apply the same domain rules consistently [6]. For systems that store data over many years, the schema must also be able to evolve through versioning, so that older records can still be understood even if the system changes [6].

2.4 Event-Based Histories, Data Integrity, and Traceability

A central requirement for maintenance history systems is that stored evidence remains accurate, consistent, and resistant to untracked modification. **Data integrity** refers to preserving accuracy and consistency of data throughout its lifecycle [7]. If records can be altered or removed without trace, trust in any derived assessment is undermined [7].

To support integrity and auditability, maintenance history can be represented as an **event-based** model. Instead of storing only a mutable current state, the system records each observation or maintenance action as a discrete, timestamped event appended to the history. The event sequence becomes the primary source of truth, enabling reconstruction of state and analysis at any point in time [8].

Traceability denotes the ability to connect derived outputs (e.g., a maintenance interpretation) back to the specific events and rules that produced them. Traceability enables auditing, debugging, and verification of system behaviour [8], and is especially important when multiple data sources contribute to a single conclusion [8].

2.5 Deterministic Interpretation and Safe Request Handling

In backend-centric systems, the backend is responsible for turning stored vehicle events into useful information, such as event categories, consistency checks, service status, and maintenance summaries. **Determinism** means that the same input data, evaluated using the same rules, should always produce the same output. This is important because the web and mobile clients should receive the same result for the same vehicle history [9].

Safe request handling is also important when new data is submitted to the system. If a network problem or retry causes the same maintenance event to be submitted more than once, the backend should avoid creating duplicate records or inconsistent histories. This property is known as **idempotency**. Idempotent API operations help preserve a reliable and consistent vehicle history under real-world network conditions [10].

2.6 Modular Backend Architecture and Layered Responsibility

Modularity refers to decomposing a system into cohesive units with clear responsibilities and explicit interfaces [11]. Two core design principles are **high cohesion** within modules and **low coupling** between modules. High cohesion means a module focuses on a single concern; low coupling minimizes cross-module dependencies, reducing change propagation and enabling independent development and testing [11].

A common structural approach is a **layered architecture**, separating concerns such as: (i) presentation (UI and interaction), (ii) application (use-case orchestration), (iii) domain (rules and invariants), and (iv) infrastructure (persistence and external integrations) [12]. Layering

isolates domain logic from UI and technology choices, which supports maintainability and controlled evolution in long-lived systems [12].

For this project’s context, modularity is assessed by whether ingestion, normalization, and domain reasoning are implemented as backend modules with stable contracts, and whether these modules can be tested in isolation without client-specific dependencies [13].

2.7 Client–Server and Thin Client Principles

In the client server model, clients implement presentation and interaction, while the server implements centralized computation, validation, and storage [14]. A **thin client** approach further reduces client-side domain logic by delegating validation, normalization, and reasoning to a single backend. This reduces logic duplication and prevents divergent interpretations across platforms as clients evolve independently [15].

Centralizing interpretation in the backend aligns with the determinism and traceability requirements: a single rule implementation and a single authoritative event history provide consistent outputs regardless of client type or version [15].

2.8 RESTful API Principles as System Boundary

The system boundary between clients and the backend is commonly implemented as an HTTP API. REST is an architectural style emphasizing resource-oriented endpoints, stateless requests, and uniform operations using standard HTTP methods. Statelessness improves scalability and simplifies client behaviour because each request contains the information required for processing [14].

Resource-oriented design is well suited to maintenance histories where entities such as vehicles and maintenance events can be addressed as resources [14]. Combined with idempotent write strategies, RESTful boundaries support robust ingestion and consistent backend-centric behaviour across heterogeneous clients [14, 10].

2.9 Related Work

This section reviews existing vehicle maintenance applications across observable dimensions relevant to the research question in Section 1.4: cross-platform availability, the conditions under which cross-device access is free to individual users, and the presence of a documented API contract that more than one client can be built against. All observations are based on publicly available descriptions, app store listings, and official product websites. Internal architecture is not publicly documented, and no claims about it are made.

Defining “documented API contract”

For the purposes of this review, an application exposes a documented API contract when its backend is reachable through a publicly documented programming interface (REST, GraphQL, or SDK) accompanied by a machine-readable schema such as OpenAPI. This dimension captures whether a stable contract exists that more than one client can be built against. It does not require that external developers are an intended audience. The verification procedure for each application consisted of inspecting the official product website for an API or developer section, searching for documentation under `docs.` or `developer.` subdomains, and checking standard developer directories for listed API references or SDKs. The evaluation reports only what is observable through this procedure. Absence of public evidence is reported as such, not interpreted as proof that no internal API exists.

2.9.1 MyAutoLog

MyAutoLog is a vehicle maintenance tracker available on iOS, with an Android version listed by the developer as in development [16]. It supports service history logging, mileage-based and date-based reminders, fuel tracking, document attachments, and PDF export, with iCloud sync between Apple devices [17]. No web client exists. No public API documentation or machine-readable schema was identified.

2.9.2 Simply Auto

Simply Auto is a vehicle maintenance and mileage tracking application available on iOS and Android, with a companion web interface at `simplyauto.app` [18]. Full cross-device synchronization and web access are paid Pro features [19, 18], so the free tier does not provide the same cloud-backed multi-device experience. No public API documentation or machine-readable schema was identified. The web and mobile clients consume the backend through a vendor-internal path.

2.9.3 CARFAX Car Care

CARFAX Car Care is a free mobile-only application on iOS and Android that integrates with the CARFAX vehicle history database to surface dealer-reported services automatically when a vehicle is identified by VIN or license plate, and accepts manual entry for services performed outside the CARFAX network [20, 21]. No standalone web client exists for the Car Care app. No public API documentation or machine-readable schema for the Car Care backend was identified.

2.9.4 FIXD

FIXD pairs a Bluetooth OBD-II sensor with a mobile-only application that translates diagnostic trouble codes (DTCs) into plain-language descriptions and tracks maintenance intervals [22, 23]. The application is of limited use without the physical sensor. No web client exists, and persistent diagnostic history independent of an active sensor session is not documented. No public API documentation or machine-readable schema was identified.

Summary of Observations and Design Motivation

Table 2.1 summarizes the four reviewed applications across the observable dimensions. The CarMinder row reflects design goals, not verified features of a commercially deployed product.

Table 2.1: Summary of reviewed applications across observable dimensions relevant to the CarMinder research question. CarMinder row reflects design goals, not commercially verified features.

Application	Web client	Mobile client	Free cross-device access	Documented API contract identified
MyAutoLog	No	iOS only	Apple ecosystem only	No
Simply Auto	Yes (pro)	Yes	No, pro tier only	No
CARFAX Car Care	No	Yes	No web access identified	No
FIXD	No	Yes	No web access identified	No
CarMinder (goal)	Yes	Yes	Yes	Yes, OpenAPI/REST

A consistent pattern emerges. The reviewed applications either operate as single-platform consumer products, restrict cross-device access to paid tiers, or operate within proprietary ecosystems. Across all four, no public API documentation or machine-readable schemas were identified. Backend services exist, but no public contract is exposed that more than one client could be built against. None of the reviewed applications presents multi-client consumption of a shared documented contract as a public characteristic of the system.

This motivates CarMinder’s thin-client architecture, in which a normalized event schema is exposed through shared backend endpoints documented by an auto-generated OpenAPI specification and consumed by both clients. The contract is the integration boundary, and each client is built against it independently.

In the delivered prototype, manual entry is the primary input source for maintenance history. OBD-II readings and receipt scanning were implemented as exploratory extensions to demonstrate that additional input sources can be ingested through the same backend-centric architecture, but full multi-source unified-history integration is outside the validated scope of this thesis.

This observation informs the architectural decisions described in Chapter 4 and forms the basis against which the system is evaluated in Chapter 5.

Chapter 3

Method

This chapter describes the methods used to design the CarMinder architecture and to validate whether the resulting system satisfies the research question stated in Section 1.4. It covers the overall research strategy, the derivation of architectural requirements, the design method, the validation strategy, the development methodology, requirements elicitation, and the limitations of the method.

3.1 Research Strategy

This thesis was conducted as a design project in the tradition of design science research [7]. The primary contribution is a constructed artefact, the CarMinder system, together with a reasoned account of the design decisions that shaped it and an evaluation of whether those decisions achieved the stated architectural goals. The research question is therefore not answered by observation or experiment alone, but by showing that a system can be designed and implemented according to specific architectural principles, and that the resulting structure can be validated against those principles.

This research strategy is appropriate for the problem addressed in Chapter 1. Fragmented maintenance data, inconsistent interpretation, and platform-coupled domain logic constitute a design problem rather than a problem requiring statistical inference over an existing population. The task was therefore to construct and evaluate an architectural solution capable of centralizing ingestion, normalization, and domain reasoning while allowing multiple client applications to remain thin presentation layers.

The method consisted of three stages. First, architectural requirements were derived from the problem context and related work. Second, design decisions were made and justified against those requirements. Third, the implemented system was validated through structural and behavioural testing to determine whether the architectural goals had been achieved.

3.2 Deriving Architectural Requirements

Before implementation began, the team established a set of architectural requirements from two sources: the problem analysis in Chapter 1 and the review of existing applications in Section 2.9.

The problem analysis identified three structural constraints. First, maintenance data originates from different sources with inconsistent formats, which requires a centralized normalization step before interpretation can occur. Second, domain logic embedded in platform-specific clients can lead to duplicated rules and divergent interpretation across platforms.

Third, the system must support multiple independent client applications without requiring each client to reimplement core functionality.

The related work review was used to identify observable limitations in existing applications, particularly limited cross-platform access, hardware dependence, and lack of a clearly documented backend contract for independent client development. These observations were used as design input rather than as a complete market survey.

From this analysis, three architectural requirements were established:

1. **Centralized domain logic.** All normalization, validation, and reasoning must reside in the backend. Client applications must contain no domain logic of their own.
2. **Stable API contract.** Clients must interact with the backend exclusively through a documented REST API. The contract must be stable enough for the web and mobile clients to be developed and maintained separately without duplicating backend logic or introducing platform-specific domain rules.
3. **Modular backend structure.** The backend must be organized into cohesive layers with clearly defined responsibilities and minimal cross-layer coupling, so that individual modules can be tested and modified independently.

3.3 Design Method

The architectural design was developed through a sequence of decisions intended to satisfy the three requirements defined above. Each decision was selected because it made one or more of the requirements testable in the implemented system.

3.3.1 Adopting a Layered Modular Backend

The backend was organized into distinct layers with defined responsibility boundaries: routers, services, repositories, domain, schemas, and configuration. This directly addresses Requirements 1 and 3. The domain layer contains the core reasoning logic, while routers handle HTTP concerns, services coordinate application behaviour, repositories isolate persistence, schemas define data contracts, and configuration handles infrastructure setup.

This structure was chosen because the research question concerns the separation of ingestion, normalization, and domain reasoning. These concerns must be separable in the implementation in order to be independently tested and architecturally verified. A flatter organization would make this separation depend mainly on convention rather than on enforceable boundaries.

This decision is grounded in the modularity principles established in Section 2.6. The layered structure reflects the high cohesion and low coupling principles described by Stevens, Myers, and Constantine [11], where each layer encapsulates a defined concern and exposes only the interface required by the layer above it.

3.3.2 Backend-First Development Sequencing

The backend was implemented before the client applications. This sequencing followed directly from the thin client requirement. If clients are to contain no domain logic and depend on the backend API for interpretation, the backend contract must exist before client functionality can be implemented meaningfully.

Backend-first development also reduced the risk of implementing domain behaviour separately in the clients. Client development was constrained to consuming backend responses rather than defining local rules for scoring, urgency, event types, or validation.

3.3.3 Enforcing Thin Clients Through Independent Development

The web and mobile clients were developed independently, with no shared domain code between them. The REST API contract was the only shared boundary. This made independent client development a practical test of the thin client principle: if two clients could provide equivalent functionality without sharing business logic, then the backend was sufficiently self-contained for the implemented scope.

No shared business-logic library was created between the clients. Functionality requiring domain knowledge was expected to be obtained from the backend rather than reproduced locally.

3.3.4 Isolating the Scoring Engine as a Pure Domain Module

The car care scoring engine was implemented as a pure domain module with no dependencies on FastAPI, the database, configuration, or external services. The engine receives all required data through function arguments and produces deterministic output for identical inputs.

This design supports Requirement 1 because scoring remains backend-side domain logic. It also supports Requirement 3 because the scoring module can be tested independently from infrastructure. A pure module with no side effects makes the central reasoning logic easier to validate and reduces the risk that infrastructure concerns influence the scoring result.

3.4 Validation Strategy

The three architectural requirements are not satisfied merely because the design describes them. To provide verifiable evidence that the implementation conforms to the intended architecture, five validation methods were applied. Each method is mapped to the requirement or requirements it addresses.

3.4.1 Architectural Conformance Testing

Architectural conformance testing was used to verify that layer boundaries were enforced in the implemented codebase. The `import-linter` tool was configured with seven contracts covering layer ordering, domain isolation, router restrictions, repository upward-dependency restrictions, service-to-router restrictions, and schema isolation. These contracts were used to detect illegal imports between architectural layers.

A parallel set of AST-based architecture tests was also used to inspect the source files directly for forbidden dependencies. This provided a second static-analysis mechanism independent of `import-linter`'s own contract system.

This validation method addresses Requirement 3. If the conformance contracts and AST-based tests pass, they provide automated evidence that the implemented backend follows the intended modular structure rather than merely documenting it. The outcomes are reported in Section 4.4.

3.4.2 Behavioural Testing of Domain Logic

Behavioural testing was used to verify that the scoring engine behaved correctly as isolated domain logic. The tests were written directly against the domain module and did not require the database, FastAPI, or external services.

The tests covered determinism, runtime isolation, score-direction properties, confidence dampening, grade-boundary mapping, and the handling of repaired incidents. Metamor-

phic testing [9, 24] was used to check that controlled changes in input produced expected directional changes in output.

API-level behaviour was tested using FastAPI’s `TestClient`. This allowed the full request flow to be exercised, including authentication, router validation, service-layer calls, and response generation, without starting a live server.

This validation method supports Requirement 1 because the score is produced consistently by the backend regardless of client. It also supports Requirement 3 because the scoring logic can be tested independently from infrastructure. The outcomes are reported in Section 4.4.

3.4.3 Thin Client Validation

Thin-client validation combined three forms of evidence.

The first form was structural. The web and mobile clients were developed independently against the same REST API contract, with no shared domain logic. This tested whether client functionality could be implemented without duplicating backend rules.

The second form was runtime observation. Domain-relevant values such as event types and scoring output were retrieved from the backend at runtime and consumed by both clients as API responses. This showed that the clients depended on the backend for interpretation rather than maintaining their own equivalent rules.

The third form was automated static analysis of client source files. Pattern-based tests scanned the client codebases for known violation idioms, including hardcoded grade thresholds, urgency classifications, and locally synthesized scoring structures. The purpose was not to prove the absence of all possible business logic, but to check for the specific forms of duplicated domain logic that the backend was designed to expose through documented contracts.

This validation method addresses Requirement 2 through the demonstrated ability of two independently developed clients to consume the same backend contract. It also supports Requirement 1 by checking that domain-relevant values are obtained from the backend rather than implemented locally.

3.4.4 External API Contract Testing

External API contract testing was used to complement the project-authored validation methods. The deployed API was exercised against its generated OpenAPI document using Schemathesis [25]. Schemathesis treats the OpenAPI specification as the contract under test, generates request payloads from the schema, sends them to the deployed endpoint, and checks responses against documented status codes, response schemas, and HTTP semantics.

This method tests the API from the perspective of an external client. The test cases are generated mechanically from the schema rather than authored manually by the project team, which reduces dependence on the team’s own assumptions about expected behaviour.

This validation method addresses Requirement 2 by checking whether the documented API contract holds against externally generated workloads. It also partially mitigates the self-assessed validation limitation discussed in Section 3.7. The outcomes are reported in Section 4.4.4.

Continuous execution of the validation suite. The validation suite was configured to run through a GitHub Actions workflow on pushes and pull requests to the main development branches. The workflow included backend tests, architectural conformance checks, API

contract tests, and a production build of the web client. This made the validation process repeatable and helped detect architectural drift during development.

3.4.5 Mutation Testing of Scoring Logic

Mutation testing was applied as a fifth validation method to evaluate the sensitivity of the behavioural test suite for the scoring engine. Mutation testing, introduced by DeMillo, Lipton, and Sayward [26] and surveyed by Jia and Harman [27], evaluates a test suite by introducing small artificial changes to the source code and rerunning the tests. Each modified version of the code is called a mutant. If at least one test fails on the mutated code, the mutant is killed, indicating that the test suite detected the behavioural change. If all tests still pass, the mutant survives, indicating a behavioural change that the tests fail to detect. The ratio of killed mutants to the total number of non-equivalent mutants is the mutation score. Mutation testing literature reports that mature test suites typically achieve mutation scores in the 60 to 80 percent range [27].

The technique was scoped to the scoring domain on methodological grounds. The scoring domain is deterministic, contains no I/O or external dependencies, and consists of pure business logic, which makes it the most reliable target for mutation analysis. It is also the canonical instance of architectural requirement R1, centralized domain logic. Mutation testing therefore complements the other validation methods rather than duplicating them: import-linter verifies architectural conformance, the pytest suite verifies functional behaviour, and mutation testing probes the strength of the functional tests themselves.

The tool used was `mutmut`. The outcomes are reported in Section 4.4.5.

3.5 Development Methodology

The project was managed using an agile approach based on the Kanban method [28]. Kanban was chosen over Scrum because the workload was non-uniform across the semester. Early work focused on problem analysis, literature review, and architectural design, while later work focused on implementation, testing, and validation. A continuous flow model was therefore more suitable than fixed-length sprints.

Task management was handled in Linear. Work items moved through five columns: *New*, *Ready*, *In Progress*, *Ready for Test*, and *Done*. This provided a shared overview of project progress and allowed tasks to be reprioritized as architectural understanding developed.

The agile method was used as a project-execution method, not as the analytical core of the thesis. The academic method of the thesis was the design and validation of a software artefact.

3.6 Requirements Elicitation

User stories were used to translate the problem context into functional scope. They were written in the form *As a [user], I want [functionality], so that [benefit]*. User stories were chosen instead of use case diagrams because the system has one primary actor, the vehicle owner, and because the thesis focuses on architecture rather than detailed interaction modelling.

The stories were used to distinguish core functionality from exploratory extensions. Core stories guided the implemented and validated prototype scope, while exploratory extension stories, such as OBD-II integration and receipt scanning, were treated as implemented prototype extensions rather than fully validated architectural evidence. The complete list of user stories is provided in Appendix C.

3.7 Method Limitations

Any research method carries limitations that affect the validity and generalizability of its conclusions. The limitations of the method used in this thesis are acknowledged here.

Single-artefact generalizability. Design science research produces knowledge through the construction and evaluation of a specific artefact. The findings of this thesis are therefore grounded in CarMinder as one implemented system. The thesis does not claim that the architecture is universally applicable to all maintenance systems, but that the demonstrated pattern is viable for the implemented case and may provide a basis for similar systems.

Self-assessed validation. The architectural requirements, design decisions, and project-authored validation instruments were produced by the same team. No independent expert review of the architecture was conducted. This creates a risk that the validation methods reflect the team’s own assumptions. External API contract testing with Schemathesis partially mitigates this for Requirement 2, but an independent architectural review would have provided stronger external validity.

Related work selection. The applications reviewed in Section 2.9 were selected to represent distinct categories of the problem space rather than through a systematic literature search protocol. The review is therefore used as structured motivation for the design goals, not as an exhaustive survey of all vehicle maintenance applications.

Tool and language specificity. The architectural conformance approach relies on `import-linter` and Python’s `ast` module. The general principle of enforcing architectural boundaries through automated tooling can transfer to other ecosystems, but the specific implementation is tied to Python and would require different tools in other technology stacks.

Identity and ownership verification. The prototype does not include strong identity verification linked to official vehicle ownership records. In a production deployment, this would require integration with national identity providers and authoritative vehicle registries. This limitation affects the trustworthiness of ownership-dependent features such as transfer, public history, and accountability, but it does not affect the architectural validation of centralized domain logic, modular backend structure, or thin-client behaviour.

Chapter Summary

This chapter described the method used to design and validate the CarMinder architecture. The thesis was framed as a design science project in which a software artefact was constructed and evaluated against explicit architectural requirements. Three requirements were derived from the problem context and related work: centralized domain logic, a stable API contract, and a modular backend structure. The system was then validated through architectural conformance testing, behavioural domain testing, thin-client validation, and external API contract testing. The chapter also described the Kanban-based development workflow, the use of user stories for requirements elicitation, and the main limitations of the chosen method.

Chapter 4

Results

This chapter presents the implemented CarMinder system. It describes the overall system architecture, the internal structure of the backend, and the two client applications. The chapter concludes with a summary of the implemented feature set and the scope of the delivered prototype. UI screenshots illustrating each client feature are reproduced in Appendix B; the figures retained inline in this chapter are those that constitute architectural or validation evidence.

4.1 The CarMinder Artefact

4.1.1 System Architecture

CarMinder is structured as a three-tier system consisting of a centralized backend API, a web client, and a mobile client. The architecture is illustrated in Figure 4.1.

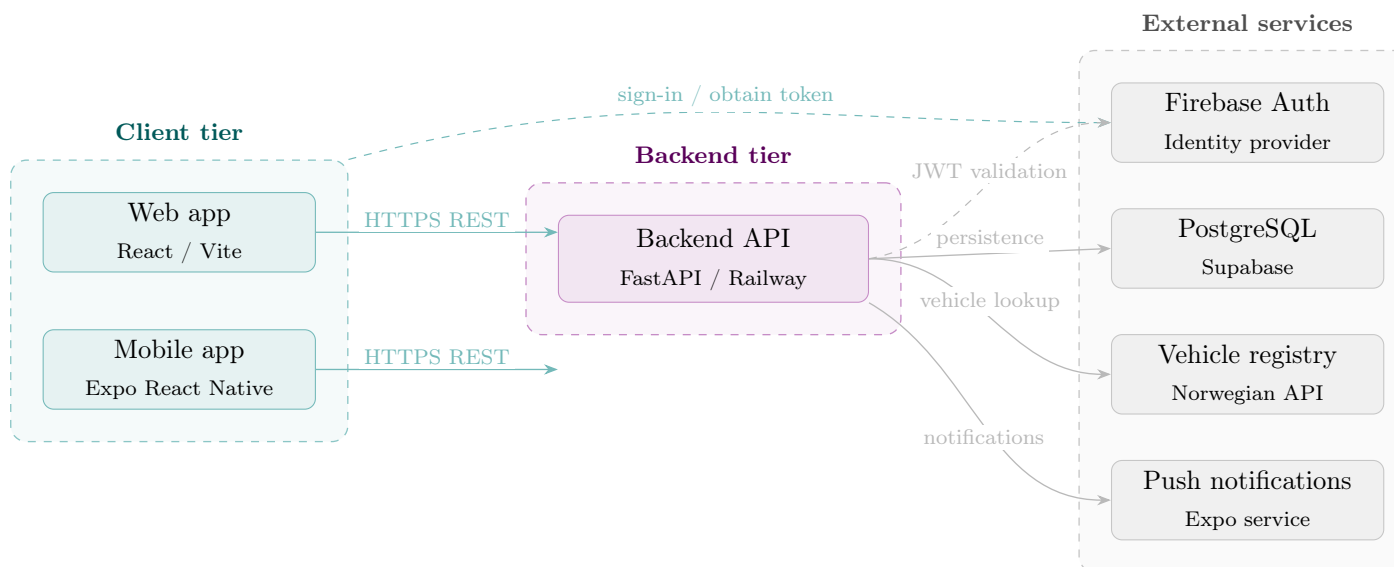


Figure 4.1: CarMinder system architecture showing the three-tier structure of thin clients, a centralized backend API, and supporting external services.

The backend is implemented in Python using the FastAPI framework and is deployed to Railway, a cloud hosting platform. It serves as the single authoritative layer for all domain logic, data validation, and persistence. Neither client contains domain reasoning; both consume the backend exclusively through a documented REST API over HTTPS.

Persistent data is stored in a PostgreSQL database hosted on Supabase. The backend communicates with Supabase through a dedicated repository layer, keeping query logic isolated from business logic. Authentication is handled through a combination of Firebase Authentication, which manages identity and credential verification, and a custom JWT layer implemented in the backend, which issues and validates access tokens for all protected API endpoints.

4.1.2 Backend Internal Structure

The backend follows a layered modular architecture organized into six distinct layers, each with a clearly defined responsibility:

- **Routers** (`routers/`): The HTTP boundary layer. Routers parse incoming requests using schema-defined models, delegate to the service layer, and map domain exceptions to appropriate HTTP status codes. Routers contain no business logic.
- **Services** (`services/`): The application logic layer. Services enforce ownership rules, coordinate validation, and orchestrate reads and writes across repositories. All business rules reside in this layer or in the domain layer below it.
- **Repositories** (`repositories/`): The data access layer. Repositories encapsulate all Supabase queries and return plain data structures to the service layer. They contain no business logic and are not accessed directly by routers.
- **Schemas** (`schemas/`): Pydantic models that define the API contract. Schemas enforce type validation at the boundary of the system and serve as the stable interface consumed by both client applications.
- **Domain** (`domain/`): Pure domain logic with no dependencies on FastAPI, repositories, or configuration. Currently used exclusively by the scoring engine, which is implemented as a side-effect-free, deterministic computation that can be tested in isolation.
- **Config** (`config/`): Environment configuration, database client initialization, and the authentication dependency that validates Bearer JWTs on all protected routes.

A typical protected request follows this path through the layers: the router receives the request and invokes the authentication dependency, which validates the Bearer JWT and injects the verified user identifier. The router then calls the relevant service function, which performs ownership checks before delegating data access to the repository layer. The service maps the result to a schema response model, which the router returns as the HTTP response. Domain exceptions raised during this process are caught at the router layer and converted to appropriate HTTP status codes. Figure 4.2 illustrates the complete request lifecycle for a protected endpoint across all three outcome paths.

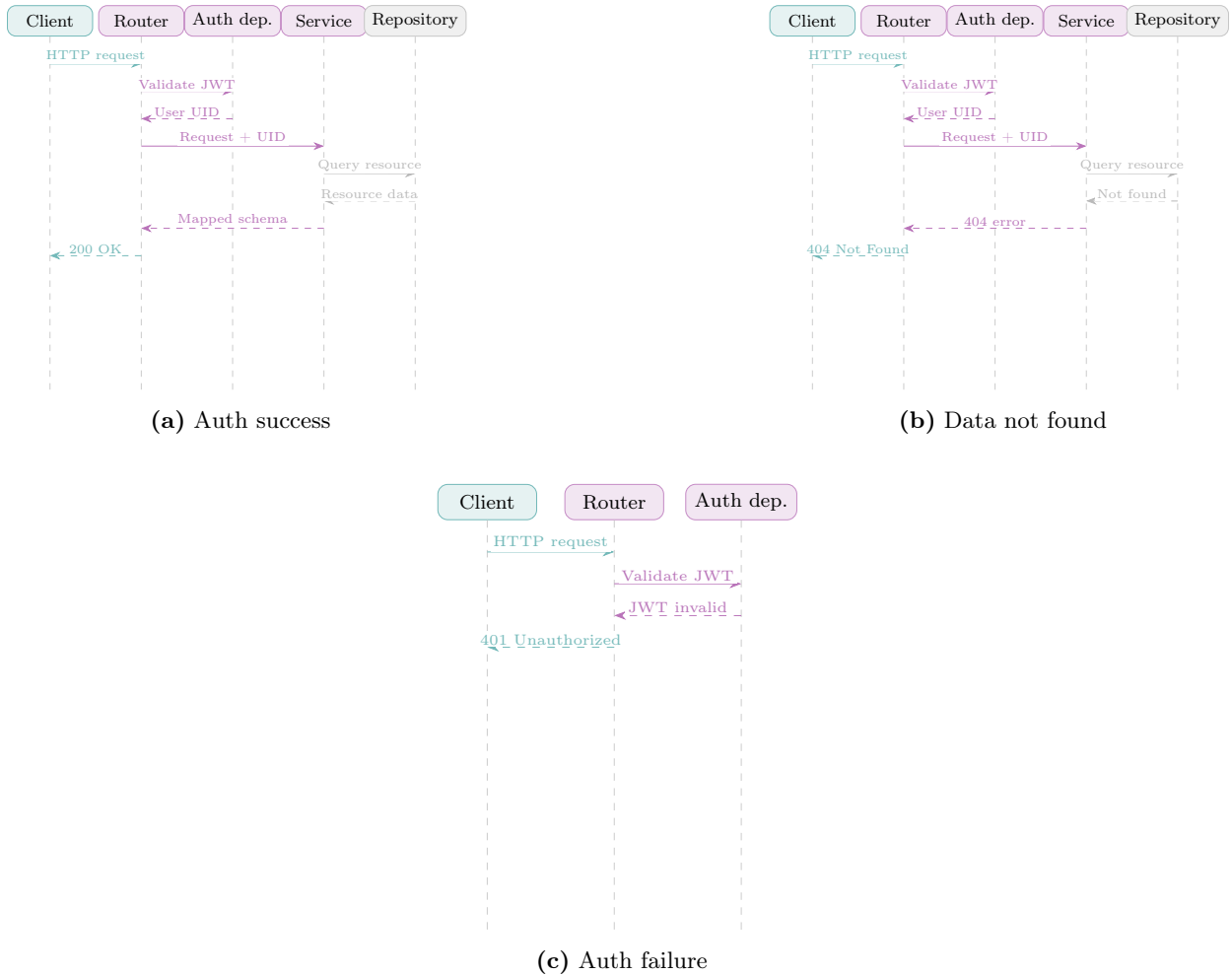


Figure 4.2: Sequence diagrams for protected API request flows: success, not found, and authentication failure.

4.1.3 Client Applications

Two independent client applications consume the backend API. The web application is built with React and Vite and targets desktop and browser-based access. The mobile application is built with Expo React Native and targets iOS and Android. Both clients were developed independently against the same REST API contract, with no shared domain code between them. This independence is a practical consequence of the thin client architecture: because all domain logic resides in the backend, neither client required access to the other's codebase or any shared logic library in order to implement equivalent functionality.

4.1.4 External Integrations

The system integrates with four external services. Firebase Authentication handles identity management and credential verification, with the backend communicating with the Firebase Identity Toolkit via its REST API. Supabase provides the hosted PostgreSQL database and is accessed exclusively through the repository layer. An external vehicle registry is integrated through the `/vehicle_lookup_service`, enabling vehicle data to be retrieved by registration number during the car onboarding flow. Push notifications are delivered via Expo's push notification infrastructure, with device tokens stored in the database and notification dispatch orchestrated by the backend.

4.2 Backend Implementation

This section describes the backend implementation in detail, covering the database schema, the scoring engine, the service interval and urgency engine, and the OBD reading pipeline.

4.2.1 Database Schema

The database is hosted on Supabase and consists of six tables. Figure 4.3 shows the complete schema with relationships.

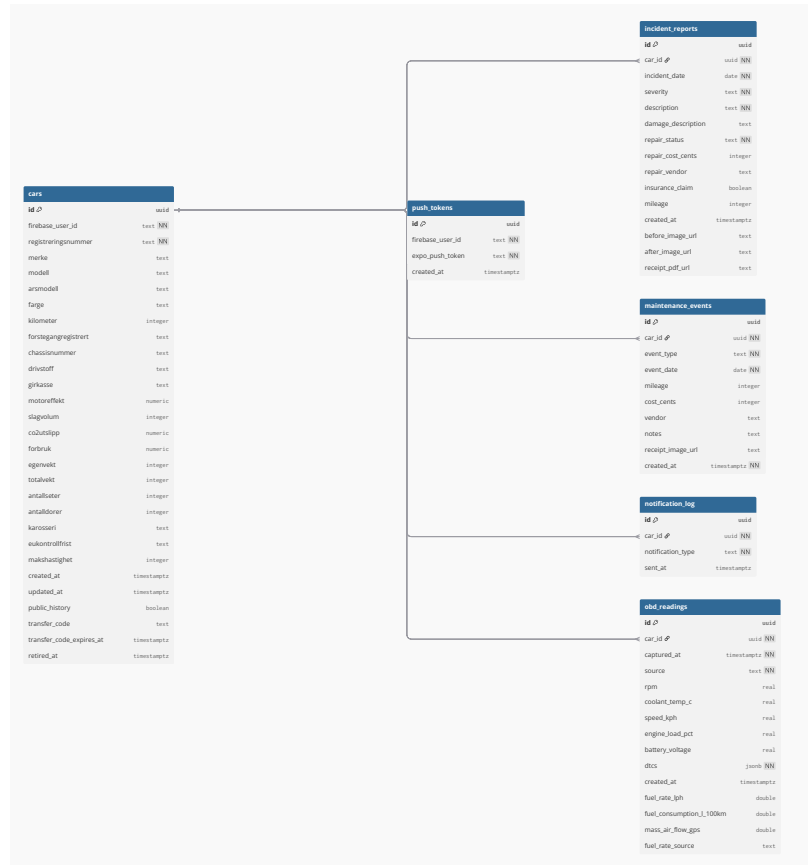


Figure 4.3: CarMinder database schema and entity relationships.

Cars:

is the central table. Each row represents a vehicle owned by a user and contains fields sourced from the external vehicle registry, including registration number, brand, model, year, fuel type, chassis number, EU inspection deadline (*eukontrollfrist*), and current mileage (*kilometer*). The *firebase_user_id* column associates each car with an authenticated user. A *public_history* boolean flag controls whether the car's maintenance history is accessible through the unauthenticated public endpoint.

maintenance_events:

records individual maintenance actions performed on a car. Each event captures the event type, date, optional mileage, cost in cents, vendor, free-text notes, and an optional receipt image URL. The *car_id* foreign key links each event to its car. The *event_type* column is constrained at the database level via a *CHECK* constraint enforcing the same controlled vocabulary defined in the backend, providing two independent validation boundaries: one at the API layer through backend validators, and one at the storage layer through the database constraint.

incident_reports:

records damage incidents associated with a car. Fields include incident date, severity, a short description, damage description, repair status, repair cost in cents, repair vendor, insurance claim status, and mileage at the time of the incident. The `severity` and `repair_status` columns are each constrained by `CHECK` constraints at the database level, mirroring the controlled vocabularies enforced by the backend validators. This ensures that invalid values are rejected independently of the application layer.

obd_readings:

stores OBD-II diagnostic snapshots uploaded from the mobile application. Each row captures a timestamp, the data source (real BLE scan or simulated), and key vehicle metrics: RPM, coolant temperature, speed, engine load, battery voltage, fuel rate (L/h), fuel consumption (L/100km), mass air flow (g/s), and a JSON array of decoded diagnostic trouble codes (DTCs). The fuel rate source column records whether fuel rate was read directly from PID 0x5E or derived from mass air flow when the direct PID is unsupported by the vehicle.

push_tokens:

stores Expo push notification tokens for authenticated users. Each row associates a `firebase_user_id` with an `expo_push_token`, enabling the backend to deliver push notifications to specific devices. Tokens are registered on login and removed on sign-out, ensuring that notifications are only delivered to active sessions. A unique constraint on (`firebase_user_id`, `expo_push_token`) prevents duplicate token registrations for the same device.

notification_log:

records which notifications have been sent per car and notification type, preventing duplicate deadline notifications from being dispatched by the cron-triggered endpoint.

All primary keys are UUIDs. Foreign key relationships link `maintenance_events`, `incident_reports`, `obd_readings`, and `notification_log` to the `cars` table via `car_id`. Controlled vocabulary columns across `maintenance_events` and `incident_reports` are enforced at both the application and database layers, ensuring data integrity independently of the API boundary.

Input Normalization

Before the engine is invoked, raw Supabase rows are converted into canonical typed data-classes by the normalization layer (`domain/scoring/normalize.py`). This step maps unknown or invalid enum values to safe defaults, parses ISO date strings, and converts all monetary values to integer cents. The resulting `CarData`, `MaintenanceEventData`, and `IncidentData` structures are the only inputs the engine accepts, ensuring that unnecessary data from the database cannot reach the scoring logic.

4.2.2 OBD-II Reading Pipeline

The OBD-II reading pipeline enables diagnostic data from a vehicle's on-board diagnostics port to be captured on the mobile application and persisted to the backend. The pipeline consists of three stages, illustrated in Figure 4.4.

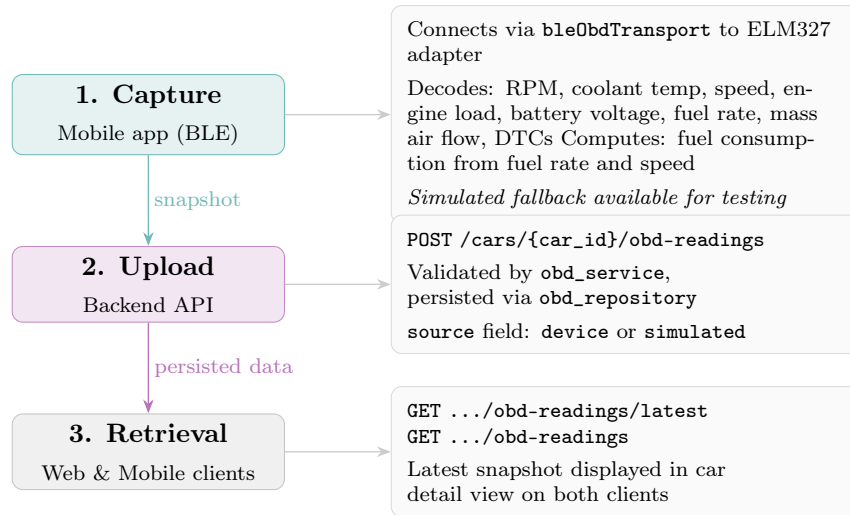


Figure 4.4: OBD-II reading pipeline from mobile capture to backend upload and client retrieval.

4.2.3 Service Interval and Urgency Engine

The service interval and urgency engine is implemented as a dedicated backend module in `service_interval_service.py`, separate from the scoring engine. It is responsible for computing the due status of each tracked service type per vehicle on request, and its output drives both the per-vehicle service status cards and the fleet-level summary on the web dashboard.

The engine tracks four service types: oil change, brake service, tyre change, and EU inspection. For the three regular service types, the engine retrieves the most recent matching maintenance event for the vehicle and applies interval rules defined in `DEFAULT_INTERVALS`. From these inputs it computes four values: `due_date`, `days_until_due`, `due_mileage`, and `km_until_due`. The urgency state is then derived from these values according to fixed thresholds. A service is classified as **overdue** if `days_until_due` or `km_until_due` is negative, **soon** if either value falls within 30 days or 1 000 km respectively, **unknown** if no prior event exists or the basis for calculation is insufficient, and **ok** otherwise.

EU inspection is handled separately. Rather than deriving a due date from service history, the engine reads the authoritative `eukontrollfrist` field directly from the car record, which is sourced from the Norwegian vehicle registry at onboarding time. This ensures that inspection compliance is always based on the legally registered deadline rather than on a user-entered event.

4.2.4 Vehicle Lookup Integration

Vehicle onboarding in both clients depends on the backend retrieving authoritative vehicle data from the Norwegian Vegvesen vehicle registry API. This integration is implemented in `vehicle_lookup_service.py` and is invoked when a user submits a registration number through the add car flow on either client.

The backend does not pass the raw registry response through to the client. Instead, `vehicle_lookup_service.py` extracts selected fields from the nested Vegvesen JSON structure and maps them into the backend's own `CarBase` schema, defined in `schemas/car.py`. This normalization step ensures that the external API's response format and field naming conventions are isolated behind the backend boundary. Both clients receive a consistent, schema-validated response regardless of changes to the upstream registry format, and neither client is coupled to the structure of the Vegvesen API directly.

The normalized fields returned to clients include registration number, brand, model, year, body type, fuel type, transmission, engine output, engine displacement, CO₂ emissions, fuel consumption, weight, seating and door count, chassis number, and the authoritative EU inspection deadline (`eukontrollfrist`). This last field is particularly significant: because it is sourced directly from the registry at onboarding time and stored on the car record, it serves as the input to the EU inspection urgency calculation described in Section 4.2.3 rather than relying on any user-entered data.

4.2.5 Notifications

The notification subsystem delivers service deadline alerts to users based on the backend service interval and urgency logic described in Section 4.2.3. Deadline states are derived centrally in the backend from interval-based service calculations and the EU inspection deadline, and these states are used as the basis for notification decisions rather than any client-side evaluation.

Notification dispatch is executed through a cron-triggered backend endpoint that periodically evaluates all vehicles and identifies alerts that should be sent. For each eligible alert, the backend resolves recipient device tokens from the `push_tokens` table, sends the notification through Expo’s push notification service, and records the dispatch in the `notification_log` table. The log ensures that duplicate notifications are not sent for the same car and notification type. A test push notification confirming successful delivery on the mobile client is shown in Figure B.2 (Appendix B).

4.2.6 PDF Export

The backend includes a PDF export feature for vehicle history reporting. This feature is exposed through a dedicated API endpoint that is invoked on demand when a user requests an export for a specific vehicle. The endpoint follows the same authenticated request flow as other protected car-specific operations.

When the endpoint is called, the backend retrieves and aggregates data from the `cars`, `maintenance_events`, and `incident_reports` tables for the selected vehicle. The collected records are combined into a single structured dataset representing the vehicle’s lifecycle history, including core vehicle metadata, chronological maintenance entries, and registered incident records. This aggregated dataset is then rendered into one document instance for the request.

Document generation is handled entirely within the backend boundary. The client does not assemble, transform, or format export content. Both client applications only trigger the export request and receive the generated PDF file in the HTTP response for download or sharing. The rendered output on both platforms is shown in Figure B.3 (Appendix B).

4.3 Client Applications

Two independent client applications consume the backend API: a web application built with React and Vite targeting desktop and browser-based access, and a mobile application built with Expo React Native targeting iOS and Android. Both clients were developed independently against the same REST API contract, with no shared domain code between them. This section presents the implemented features as delivered across both platforms. UI screenshots for each feature are reproduced in Appendix B; only architectural and validation figures are kept inline. Features specific to a single platform are described separately in Section 4.3.10.

4.3.1 Authentication

Login on both clients offers email and password sign-in and Google Sign-In; mobile additionally offers Apple Sign-In on iOS. Successful authentication issues a JWT access token persisted on the device and used to authorize all subsequent API requests. Registration and forgot-password flows are accessible directly from the login screen. Login and registration screens for both platforms are shown in Figures B.4 and B.6 (Appendix B).

4.3.2 Car Management

Both clients implement the same onboarding flow: the user enters a registration number, the client queries the backend, which retrieves vehicle data from the Norwegian vehicle registry, and the result is displayed as a structured preview before the vehicle is saved. Retrieved data includes brand, model, first registration date, body type, fuel type, engine details, and the authoritative EU inspection deadline. The profile screen lists all of the user's registered vehicles with view, manage, and remove actions. Screens are shown in Figures B.7 and B.8 (Appendix B).

4.3.3 Car Care Score

Both clients display the Car Care Score computed by the backend scoring engine. The score view presents the overall integer score, letter grade, confidence label, and a per-category breakdown with colour-coded progress bars. Below the breakdown, the backend returns a set of deterministic recommendations derived from the scoring result, for example, flagging overdue services, unrepaired incidents, or missing mileage data. No scoring logic is performed in either client; both screens are a direct presentation of the response from the `/cars/{car_id}/score` endpoint.

The score is computed across five weighted categories, illustrated in Figure 4.5 and summarised in Table 4.1.

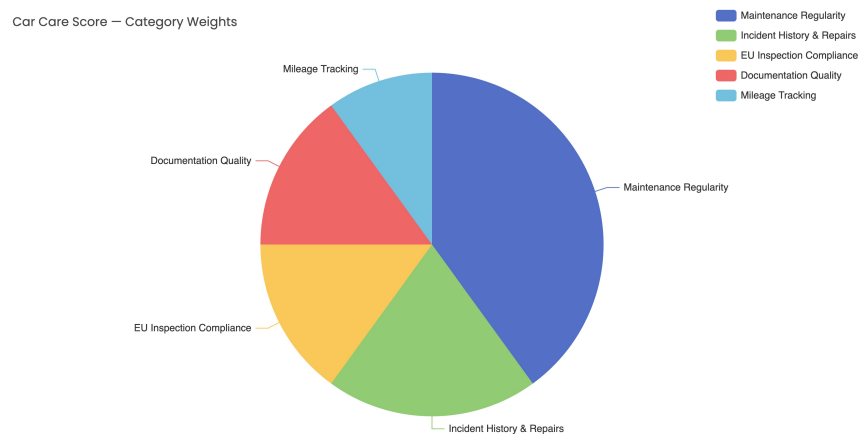


Figure 4.5: Distribution of category weights in the car care scoring engine.

Category	Description	Weight
Maintenance Regularity	Adherence to service intervals over a 5-year window	40%
EU Inspection Compliance	Current and historical inspection compliance	15%
Incident History & Repairs	Incident severity and repair diligence	20%
Mileage Tracking	Consistency and reasonableness of mileage records	10%
Documentation Quality	Completeness of event and incident records	15%

Table 4.1: Car care score category weights.

Because scores derived from sparse data are less reliable, the engine applies a confidence factor between 0.0 and 1.0 based on car age, total event count, presence of mileage data, and presence of an EU inspection deadline. When confidence is below 0.5, the final score is dampened towards a neutral value of 50. The confidence level is returned alongside the score as a labelled value (`very_low`, `low`, `moderate`, or `high`) and is displayed in both client interfaces. The final integer score is converted to a letter grade: A (≥ 90), B (≥ 75), C (≥ 60), D (≥ 40), and F (below 40). The score screen on both platforms is shown in Figure B.9 (Appendix B).

4.3.4 Maintenance Events

Both clients implement a maintenance event form capturing event type, date, and optional fields including mileage, cost, vendor, and notes. Event types are fetched from the backend at runtime rather than hardcoded in the client, ensuring the available types remain consistent with the backend schema. Submitted payloads are validated and persisted entirely by the backend. Both clients also display a chronological maintenance history per vehicle. The form on both platforms is shown in Figure B.10 (Appendix B).

4.3.5 Incident Reports

Both clients implement an incident report form capturing severity, date, description, damage details, repair status, insurance claim status, and optional mileage and cost. As with maintenance events, payloads are validated and persisted by the backend; no incident logic is handled in the client. The form on both platforms is shown in Figure B.11 (Appendix B).

4.3.6 Calendar

Both clients implement a calendar view presenting a monthly grid with dot indicators on days containing maintenance events, and a scrollable chronological event timeline below. A vehicle selector filters the timeline by car. All event data is fetched from the backend; neither client stores or computes timeline data locally. The view on both platforms is shown in Figure B.12 (Appendix B).

4.3.7 Public History Lookup

Both clients provide an unauthenticated public history search. The user enters a registration number and the client queries the public backend endpoint, which returns the vehicle's maintenance history and incident reports if the owner has enabled public history sharing. The result displays vehicle details, incident report count, and a list of maintenance events with dates and types. This feature requires no authentication and is designed to support pre-purchase history checks. The screens are shown in Figure B.13 (Appendix B).

4.3.8 Transfer Ownership

Both clients implement a vehicle ownership transfer flow that allows a current owner to generate a transfer code for a vehicle and share it with a prospective new owner. The recipient enters the code via the Claim Car dialog, after which the vehicle and its complete maintenance history are transferred to the new owner's account. This feature directly supports the resale trust use case identified in Section 1.2: a seller can provide a buyer with cryptographically generated evidence of the vehicle's verified maintenance history without requiring any manual data export. Figure 4.6 illustrates the complete transfer flow; the corresponding interface on both clients is shown in Figure B.15 (Appendix B).

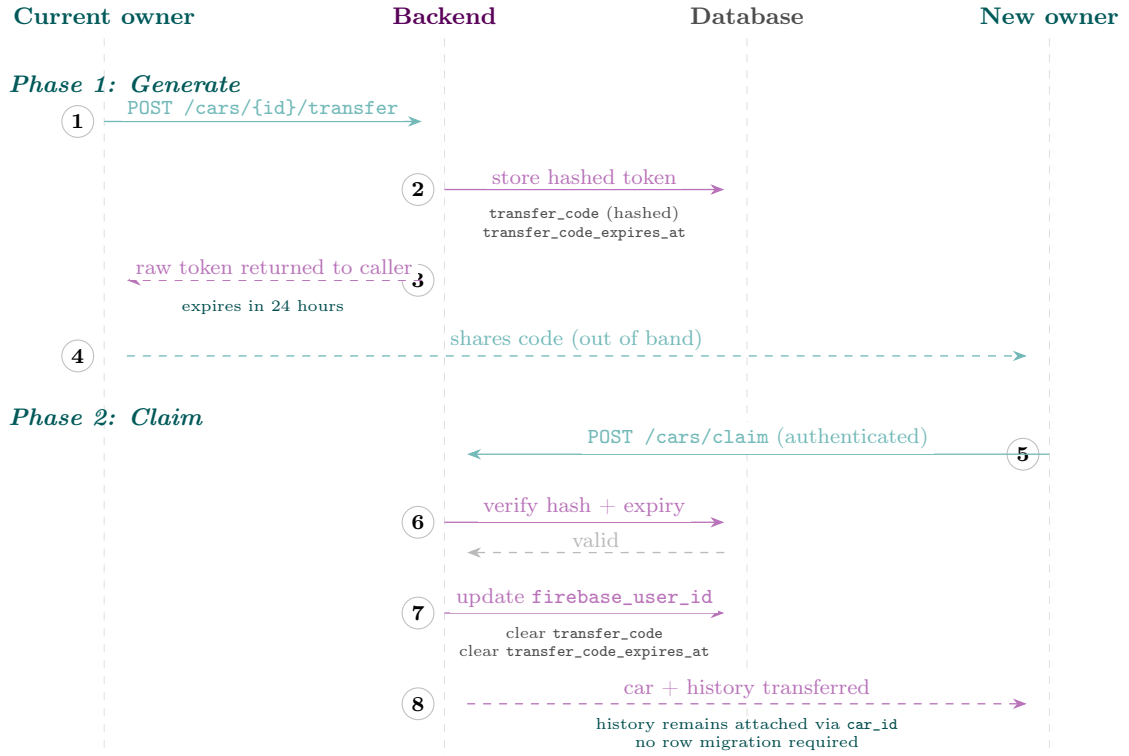


Figure 4.6: Vehicle ownership transfer flow (code generation and claim process).

4.3.9 Vehicle Retirement

The retirement flow allows an owner to mark a vehicle as permanently out of service. The endpoint `POST /cars/{car_id}/retire` verifies ownership through the standard Firebase token dependency, rejects already retired vehicles with a conflict response, and updates the existing car row by setting `retired_at` to the current UTC timestamp while clearing any active `transfer_code` and its expiry. The retired row is preserved rather than deleted so that its chassis number continues to act as a registration block. When a new `POST /cars/` request submits a chassis number that already exists with a non-null `retired_at`, the create service in `car_service.py` rejects the request with a message stating that the vehicle has been retired and cannot be registered again. A chassis number that exists without `retired_at` returns the ordinary duplicate registration error, which keeps the two failure modes distinguishable to the client.

Retirement also closes the vehicle to ownership transfer. The `initiate_transfer` and `claim_car` services both reject retired cars before any code generation or claim validation occurs, which prevents a retired vehicle from re-entering circulation through the transfer flow described in Section 4.3.8. Together with the chassis-number block, this enforces a one-way state transition from active to retired at the service layer, consistent with the centralized domain logic requirement (R1) in Section 3.2.

Because the operation is irreversible, both clients gate the call behind an explicit destructive confirmation that names the vehicle and states the consequences: the chassis number will be blocked from future registration and the action cannot be undone. The mobile client triggers the call through `api.retireCar` after a custom confirmation modal on the profile screen, and the web client posts to the same endpoint through `carsApi.retireCar` after a native browser confirmation. Once `retired_at` is populated, both clients render a Retired badge on the vehicle card, hide the public-history toggle, disable mileage editing, and remove the transfer controls. No retirement-specific logic is duplicated in either client; the disabled controls and the badge are a direct projection of the backend state. The destructive confirmation step on both clients is shown in Appendix B.1.

4.3.10 Platform-Specific Features

Web: Fleet Dashboard

The web home dashboard provides a fleet-level summary across all registered vehicles. Four summary cards display the total number of overdue tasks, services due soon, fleet size, and average care score. Below the summary, each vehicle appears as a card showing its care score, confidence label, and urgency status for each tracked service type. The mobile application surfaces equivalent information per vehicle rather than in aggregate. The dashboard is shown in Figure B.14 (Appendix B).

Mobile: OBD-II Reading

The mobile application implements the OBD-II reading pipeline described in Section 4.2.2. The BLE transport layer handles device discovery, GATT characteristic resolution, ELM327 adapter initialisation, and serial PID command sequencing. Signal decoding is performed on-device before the resulting snapshot is submitted to the backend via `POST /cars/{car_id}/obd-readings`. In addition to the basic diagnostic PIDs, the on-device decoder extracts fuel rate (PID 0x5E) and mass air flow (PID 0x10), and computes fuel consumption per 100 km from fuel rate and speed. Where PID 0x5E is unsupported by the vehicle, fuel rate is derived from mass air flow using stoichiometric assumptions, and the `fuel_rate_source` field records which path produced the value. This division of responsibility is discussed in Section 5.3.

The pipeline was validated on two physical vehicles using a real ELM327 Bluetooth adapter. The Mazda 3 was used for repeated live metric capture and produced 80 persisted readings across the test period, from which the trend charts in Figures B.17 and B.18 are derived. The Toyota Yaris was used for diagnostic trouble code retrieval and returned two codes, P0221 and B1622, both rendered as “Unknown diagnostic trouble code” due to gaps in the client-side DTC description map. This limitation is discussed in Sections 5.3 and 7.4. Real-device scans, live metrics including fuel data, and the corresponding web view are reproduced in Figures B.16, B.17, and B.18 (Appendix B).

Mobile: Push Notifications

The backend delivers push notifications via Expo’s push notification infrastructure. Device tokens are registered on login and removed on sign-out. A cron-triggered backend endpoint evaluates service deadlines and dispatches notifications to devices with approaching or overdue services, with a notification log preventing duplicate dispatches.

Mobile: Apple Sign-In

Apple Sign-In is available on the mobile client on iOS only. The web client offers email/password and Google Sign-In. The Apple sign in Screen is shown in B.5

Mobile: OCR Receipt Scanner

The mobile client provides a receipt-scanning feature supporting semi-automated ingestion of maintenance data. When adding a maintenance event, a Scan receipt (OCR) action captures a photo via the device camera and uploads the image to the backend through an authenticated `POST /ocr/receipt` request. This keeps the client lightweight while the extraction logic is performed centrally.

On the server, the OCR pipeline is a dedicated service using Tesseract through `pytesseract`. After validating MIME type and file size, the service converts the image to grayscale and runs text recognition with a configurable language model (defaulting to `eng+nor`). The backend then applies deterministic pattern-based rules to extract candidate fields: event date, cost, vendor, mileage, and a short notes summary. Each field is accompanied by a confidence score, allowing the client to distinguish recognised values from inconclusive extraction. If no meaningful fields can be extracted, the form is left unchanged, ensuring that OCR acts as an assistive input channel rather than a replacement for human confirmation. The extracted values prefill the standard Add maintenance event form; the user reviews and edits before submitting through the normal backend-controlled creation flow, preserving validation and persistence consistency. The interface is shown in Figure B.19 (Appendix B).

4.4 Validation Results

This section presents the results of the five validation methods described in Section 3.4, organised by the three architectural requirements established in Section 3.2. Each subsection states the requirement, identifies the validation method applied, and reports the outcome.

4.4.1 R1: Centralized Domain Logic

Requirement 1 states that all normalization, validation, and reasoning must reside in the backend, and that client applications must contain no domain logic of their own.

This requirement was validated through two unit tests targeting the scoring engine directly. The determinism test asserts that identical inputs produce identical outputs across three independent calls, confirming that the scoring result is fully determined by the backend and does not vary across clients or invocations. The runtime isolation test performs AST-based inspection of the engine module at runtime, verifying that no infrastructure packages are imported during execution. Both tests (Listings A.5 and A.13 in Appendix A) form part of the 20-test architecture and scoring subset shown in Figure 4.12, which completed in 0.16 seconds. A suite touching the database or web framework cannot complete in that time; the scoring engine is therefore demonstrably isolated from infrastructure at both the import level and the runtime level, satisfying the centralization requirement.

4.4.2 R2: Stable API Contract

Requirement 2 states that clients must interact with the backend exclusively through a documented REST API, and that the contract must be stable enough for independent clients to be developed against it without coordination at the code level.

This requirement was validated through the independent development of the web and mobile clients. Both clients implement equivalent functionality against the same REST API contract, with no shared domain code between them. The web client is built with React and Vite; the mobile client is built with Expo React Native. Neither depends on the other's codebase or any shared logic library.

Two concrete runtime patterns provide the strongest evidence for this claim, illustrated in

Figure 4.9. First, maintenance event types are not defined in either client. Both clients request the list of valid event types from the backend at runtime on each form load, render the returned values as options, and submit the user’s selection back to the API without any local enumeration. The set of valid categories is therefore defined in exactly one place. Second, the complete scoring result is consumed directly from the backend response in both clients. The response includes the overall score, letter grade, confidence value, confidence label, category breakdown, and recommendations. Neither client implements any part of the score calculation, confidence dampening, or recommendation logic. Both act solely as presentation layers for the computed result.

Automated conformance tests verify that neither client duplicates grade thresholds, urgency classifications, or backend-owned vocabularies (Listings A.1, A.2, and A.3 in Appendix A). Figure 4.7 shows all four conformance tests passing.

```
===== test session starts =====
platform darwin -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0 -- /Users/evanbelal/CarMinder/backend-python/.venv/bin/python
cachedir: .pytest_cache
rootdir: /Users/evanbelal/CarMinder/backend-python
plugins: anyio-4.12.1
collected 4 items

tests/test_client_conformance.py::test_clients_do_not_duplicate_score_grade_thresholds PASSED
tests/test_client_conformance.py::test_clients_do_not_duplicate_service_urgency_thresholds PASSED
tests/test_client_conformance.py::test_clients_fetch_backend_owned_vocabularies_from_api_modules PASSED
tests/test_client_conformance.py::test_clients_consume_backend_score_and_service_status_dtos PASSED

===== 4 passed in 0.02s =====
```

Figure 4.7: Thin-client conformance test results showing all four tests passed, confirming that no domain logic is duplicated in either client.

API-level behaviour was verified using FastAPI’s `TestClient`, exercising the full request path including authentication middleware, router validation, and service dispatch without requiring a running server (Listings A.8 and A.9). Figure 4.8 shows all 13 API integration tests passing.

```
===== test session starts =====
platform darwin -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0 -- /Users/evanbelal/CarMinder/.venv-sora/bin/python3.14
cachedir: .pytest_cache
rootdir: /Users/evanbelal/CarMinder/backend-python
plugins: anyio-4.12.1
collected 13 items

tests/test_api_integration.py::test_get_car_score_success PASSED [ 7%]
tests/test_api_integration.py::test_get_car_score_requires_authentication PASSED [ 15%]
tests/test_api_integration.py::test_get_car_score_returns_404_when_car_missing PASSED [ 23%]
tests/test_api_integration.py::test_create_maintenance_event_success PASSED [ 30%]
tests/test_api_integration.py::test_create_maintenance_event_rejects_invalid_payload PASSED [ 38%]
tests/test_api_integration.py::test_create_incident_success PASSED [ 46%]
tests/test_api_integration.py::test_create_maintenance_event_returns_404 PASSED [ 53%]
tests/test_api_integration.py::test_create_maintenance_event_returns_400 PASSED [ 61%]
tests/test_api_integration.py::test_get_service_status_success PASSED [ 69%]
tests/test_api_integration.py::test_get_service_status_requires_authentication PASSED [ 76%]
tests/test_api_integration.py::test_get_service_status_returns_404 PASSED [ 84%]
tests/test_api_integration.py::test_lookup_vehicle_success PASSED [ 92%]
tests/test_api_integration.py::test_lookup_vehicle_not_found PASSED [100%]

===== warnings summary =====
```

Figure 4.8: API integration test results showing all 13 tests passed using FastAPI’s `TestClient`, exercising the full request path without a running server.

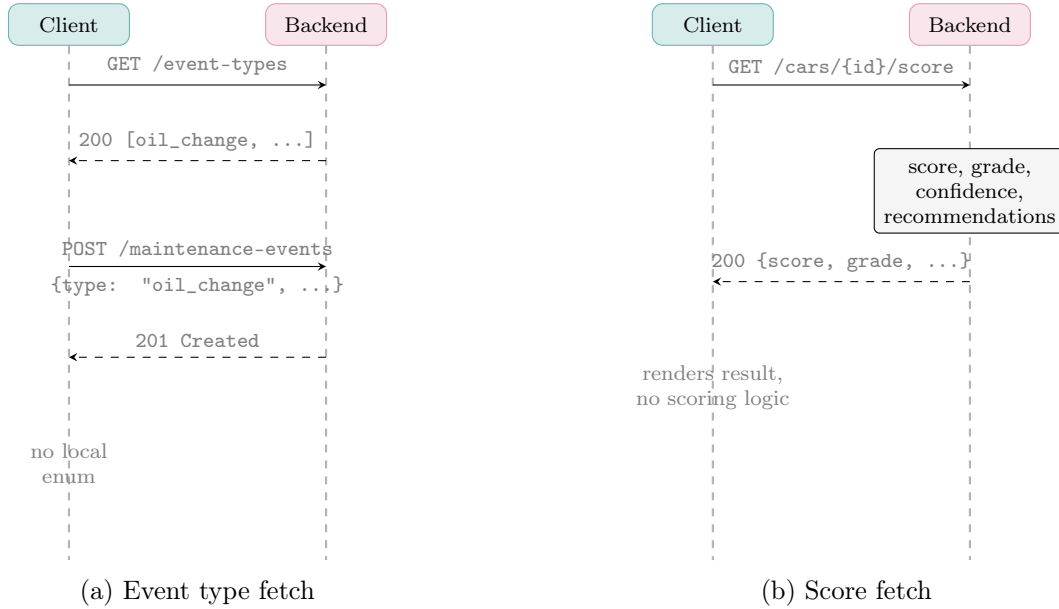


Figure 4.9: Sequence diagrams demonstrating thin-client behaviour across event type retrieval and score consumption. Left: both clients fetch valid event types from the backend at runtime rather than maintaining a local enumeration. Right: both clients consume the complete scoring result from the backend without implementing any part of the scoring, confidence, or recommendation logic. In both flows the client is responsible only for presentation; all domain rules remain in the backend.

The fact that two independently developed clients could deliver equivalent functionality against the same contract, without code-level coordination, provides practical evidence that the API contract was sufficiently stable and self-contained throughout development. The evidence for this requirement is grounded in development practice, observable runtime behaviour, and pattern-based static analysis rather than exhaustive enumeration of every business rule across the client codebases, which remains a direction for future validation work.

4.4.3 R3: Modular Backend Structure

Requirement 3 states that the backend must be organized into cohesive layers with clearly defined responsibilities and minimal cross-layer coupling, so that individual modules can be tested and modified independently (Listing A.4).

This requirement was validated through two independent structural analysis methods. The `import-linter` tool was configured with seven contracts (Listing A.10) and executed against the full backend codebase. As shown in Figure 4.10, the tool analysed 79 files and 212 inter-module dependencies. All seven contracts were kept and none were broken. The passing contracts confirm that routers do not access repositories or domain logic directly, that repositories do not depend on higher layers, that services do not depend on routers, and that schemas remain isolated from the rest of the application structure.

To strengthen this result, a parallel set of AST-based architecture tests (Listings A.11 and A.12) performed direct static analysis of the source files and verified the same boundaries without relying on `import-linter`'s contract engine. As shown in Figure 4.11, eight tests were collected and all eight passed in 0.04 seconds.

The agreement between two independent tools operating through different mechanisms constitutes a stronger conformance argument than either result alone. Together they provide automated and repeatable evidence that the layered structure is enforced in the delivered codebase and not merely reflected in the design.



```

-----
Contracts
-----

Analyzed 79 files, 212 dependencies.
-----

Enforce layered architecture KEPT
Domain layer must not import from any other internal layer KEPT
Routers must not access domain layer directly KEPT
Routers must not access repositories directly KEPT
Repositories must not depend on higher layers KEPT
Services must not depend on routers KEPT
Schemas must remain isolated from app layers KEPT

Contracts: 7 kept, 0 broken.

```

Figure 4.10: Import-linter results showing that all seven architectural contracts were kept.

```

===== test session starts =====
platform darwin -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0 -- /Users/evanbelal/CarMinder/.venv-sora/bin/python3.14
cachedir: .pytest_cache
rootdir: /Users/evanbelal/CarMinder/backend-python
plugins: anyio-4.12.1
collected 8 items

tests/test_architecture_modularity.py::test_routers_do_not_import_repositories_or_domain PASSED [ 12%]
tests/test_architecture_modularity.py::test_services_do_not_import_routers PASSED [ 25%]
tests/test_architecture_modularity.py::test_repositories_do_not_import_upward_layers PASSED [ 37%]
tests/test_architecture_modularity.py::test_schemas_are_isolated_from_app_layers PASSED [ 50%]
tests/test_architecture_modularity.py::test_scoring_engine_is_framework_free PASSED [ 62%]
tests/test_architecture_contracts.py::test_importlinter_contains_required_contracts PASSED [ 75%]
tests/test_architecture_contracts.py::test_main_imports_all_router_modules PASSED [ 87%]
tests/test_architecture_contracts.py::test_main_includes_each_router_module PASSED [100%]

===== 8 passed in 0.04s =====

```

Figure 4.11: AST-based architecture test results showing that all eight tests passed.

```

tests/test_architecture_contracts.py::test_importlinter_contains_required_contracts PASSED [ 5%]
tests/test_architecture_contracts.py::test_main_imports_all_router_modules PASSED [ 10%]
tests/test_architecture_contracts.py::test_main_includes_each_router_module PASSED [ 15%]
tests/test_architecture_modularity.py::test_routers_do_not_import_repositories_or_domain PASSED [ 20%]
tests/test_architecture_modularity.py::test_services_do_not_import_routers PASSED [ 25%]
tests/test_architecture_modularity.py::test_repositories_do_not_import_upward_layers PASSED [ 30%]
tests/test_architecture_modularity.py::test_schemas_are_isolated_from_app_layers PASSED [ 35%]
tests/test_architecture_modularity.py::test_scoring_engine_is_framework_free PASSED [ 40%]
tests/test_importlinter_integration.py::test_importlinter_contracts_pass PASSED [ 45%]
tests/test_scoring_engine.py::test_scoring_is_deterministic PASSED [ 50%]
tests/test_scoring_engine.py::test_engine_has_no_infrastructure_imports PASSED [ 55%]
tests/test_scoring_engine.py::test_adding_good_maintenance_event_does_not_lower_score PASSED [ 60%]
tests/test_scoring_engine.py::test_severe_unrepaired_incident_lowers_score PASSED [ 65%]
tests/test_scoring_engine.py::test_empty_car_produces_dampened_score PASSED [ 70%]
tests/test_scoring_engine.py::test_grade_boundaries[90-A] PASSED [ 75%]
tests/test_scoring_engine.py::test_grade_boundaries[75-B] PASSED [ 80%]
tests/test_scoring_engine.py::test_grade_boundaries[60-C] PASSED [ 85%]
tests/test_scoring_engine.py::test_grade_boundaries[40-D] PASSED [ 90%]
tests/test_scoring_engine.py::test_grade_boundaries[39-F] PASSED [ 95%]
tests/test_scoring_engine.py::test_repaired_incident_penalises_less_than_unrepaired PASSED [100%]

===== 20 passed in 0.16s =====

```

Figure 4.12: Pytest results showing that all 20 tests passed successfully.

In addition to being consumed by both clients, the backend contract is exposed through FastAPI's generated OpenAPI specification and Swagger UI, providing a machine-readable description of endpoints, schemas, and authentication requirements (see Appendix D.1).

4.4.4 External API Contract Testing

The validation evidence presented so far derives from project-authored unit, integration, and conformance tests. To complement these with an externally generated test suite, the deployed Railway API was tested against its own OpenAPI document using Schemathesis [25]. Schemathesis treats the OpenAPI specification as the contract under test, generates request payloads that conform to the schema, sends them to the deployed endpoint, and checks responses against the documented status codes, response schemas, and HTTP semantics. This approach exercises the API from the perspective of an external client that did not write the implementation and supplies direct evidence for requirement R2 (stable API contract).

The tests were run against the deployed instance at `https://carminder-production.up.railway.app` using Schemathesis 4.18.0, with the OpenAPI 3.1.0 document at `/openapi.json` as the contract source. Both the baseline and the final validation runs used the same random seed (20260508) for reproducibility. Each run generated up to 100 examples per operation across three test phases (coverage, fuzzing, stateful). The exact commands used for both runs are listed in Appendix F.1.

The baseline run exposed a number of mismatches between the documented contract and runtime behavior. Table 4.2 summarizes the baseline outcome.

Table 4.2: Schemathesis baseline run against the deployed API.

Metric	Result
Operations selected	36 / 36
Generated test cases	981
Unique failures	38
Coverage phase	5 passed, 31 failed
Fuzzing phase	4 passed, 32 failed
Stateful phase	185 passed, 5 failed

The findings fell into four categories. First, `POST /auth/login` returned `500 Internal Server Error` for input that the schema permitted, indicating that the API accepted a class of input the implementation could not safely process. Second, several protected endpoints returned `401 Unauthorized` for unauthenticated requests, but the OpenAPI document did not declare a `401` response. Third, the `cron` endpoint returned `403 Forbidden` for an invalid secret, again undocumented in the contract. Fourth, unsupported HTTP methods on static car routes such as `/cars/lookup` were silently absorbed by the dynamic `/cars/{car_id}` route, producing `401` responses instead of the expected `405 Method Not Allowed`.

In response, the API contract and the implementation were tightened in parallel. Request schemas were narrowed (`EmailStr` for email fields, non-empty string constraints on required identifiers, minimum length on signup passwords). Shared response definitions were added for the previously undocumented `400`, `401`, `403`, `404`, `409`, and `500` responses. Login error handling was rewritten to convert downstream validation failures into controlled HTTP responses rather than letting them surface as `500`. Duplicate signup attempts were reclassified from `400 Bad Request` to `409 Conflict` to match HTTP semantics. Hidden FastAPI handlers were added for unsupported methods on static car routes, returning `405 Method Not Allowed` with the required `Allow` header.

After redeployment, the same Schemathesis configuration was rerun against the live deployment using the same seed. Figure 4.13 shows the terminal output of the final run.

The final run completed with zero contract violations across 4,759 generated test cases. The three warnings reported by Schemathesis concern test inputs rather than contract correct-

```

p... U Detected: 36/36
sis... U Tested: 36
rv... U

Test Phases:
  Examples
  Coverage
  Fuzzing
  Stateful

Warnings:
  Missing authentication: 24 operations returned only 401/403 responses
  Missing valid test data: 1 operation repeatedly returned 404 responses
  Schema validation mismatch: 23 operations mostly rejected generated data

Test cases:
  4759 generated, 4759 passed

Reports:
  - JUnit: docs/schemathesis-report-final/junit-20260508T181827Z.xml
  - HAR: docs/schemathesis-report-final/har-20260508T181827Z.json
  - NDJSON: docs/schemathesis-report-final/ndjson-20260508T181827Z.ndjson

Seed: 20260508

```

Figure 4.13: Schemathesis final validation run against the deployed Railway API after corrective changes. All 36 operations passed across the Coverage, Fuzzing, and Stateful phases, with 4,759 generated test cases and zero contract violations. The three remaining warnings concern test inputs rather than contract correctness.

ness: 24 operations returned only 401 or 403 because the run did not supply a bearer token, one operation (the public history endpoint) repeatedly returned 404 because randomly generated registration numbers are unlikely to match seeded production data, and 23 operations predominantly rejected generated payloads due to the absence of valid external identity tokens, valid resource identifiers, and a valid cron secret. These limitations are characteristic of unauthenticated black-box fuzzing against a production-like deployment and are discussed further in Section 5.2.

The complete Schemathesis run artifacts in JUnit, HAR, and NDJSON formats for both the baseline and final runs are archived in the project repository under `docs/schemathesis-report/` and `docs/schemathesis-report-final/`. Both runs are reproducible by re-executing the commands in Appendix F.1 against the deployed Railway endpoint with seed 20260508. A condensed deliverable archive containing the run summary and JUnit reports for both runs is also archived at `docs/schemathesis-deliverable.zip`.

The Schemathesis test cases are reported separately from the project-authored pytest count in Section 4.8 because they are generated from the schema rather than authored by the project team, and because they exercise the deployed API from outside the implementation boundary rather than the in-process `TestClient`.

4.4.5 Mutation Testing

The mutation testing using `mutmut` was applied to the scoring domain to evaluate the sensitivity of the existing test suite. The standard metric is the mutation score, defined as the ratio of killed mutants to the total number of non-equivalent, covered mutants [27].

The mutation run used the 11 collected scoring tests in `backend-python/tests/test_scoring_engine.py` as the oracle. Mutation testing did not target the mobile frontend, FastAPI routers, database repositories, or external integration logic.

Results

The final mutation test run produced the results shown in Table 4.3.

The conservative mutation score, treating mutants with no associated tests as undetected, is:

$$\frac{365}{1185} \approx 30.8\%$$

Excluding the 204 mutants for which `mutmut` could not associate any test case, the score

Outcome	Count
Total mutants	1185
Killed mutants	365
Survived mutants	616
Mutants with no associated tests	204
Timeouts	0
Segfaults	0

Table 4.3: Mutation testing results for the backend scoring domain.

risers to approximately 37.2%. A portion of the surviving mutants may also be equivalent mutants, semantically identical to the original code and therefore unkillable by any test [27]. No equivalent-mutant analysis was performed in this work, so the reported score is a lower bound on the true mutation score. A per-file breakdown of the results is provided in Appendix A.1.5.

Interpretation

The results show that the current scoring tests catch some important behavioral changes, but they do not fully specify the detailed scoring model. Mutation testing literature reports that mature test suites typically achieve mutation scores in the 60 to 80 percent range [27], so the result here indicates a test suite that detects broad behavioral relationships and grade boundary checks but does not fully specify detailed business rules. Appendix A.1.2 illustrates a detected mutation at the score-90 grade boundary, while Appendix A.1.3 illustrates an undetected mutation at the 0.25 confidence threshold. The contrast indicates that grade-boundary tests exist, while confidence-threshold tests do not.

The mutation results revealed weaker test coverage around:

- confidence threshold boundaries,
- normalization of input data,
- maintenance interval scoring,
- documentation-quality scoring,
- exact category-score calculations,
- exact overall-score calculations.

The 204 mutants for which mutmut could not associate any test case fall predominantly in normalization code. This indicates that the test-discovery heuristic found no test linked to those source files, which in practice means those functions lack direct unit-test coverage.

Mutation testing does not measure correctness of the scoring model itself. It measures only the test suite’s ability to detect changes in that model. The gaps identified here are documented as a limitation in Section 7.3 and as a concrete work item in Section 7.4. The reproduction command and tooling configuration are provided in Appendix A.1.

4.4.6 Requirement Traceability Summary

Table 4.4 summarises the mapping between each architectural requirement, the validation method applied, and the outcome.

Table 4.4: Requirement traceability summary mapping each architectural requirement to its validation method and result.

Requirement	Validation method	Evidence	Result
R1: Centralized domain logic	Unit tests (determinism + runtime isolation)	20/20 tests passed; execution in 0.16 s confirms no infrastructure dependency (Listings A.5, A.13)	Satisfied
R2: Stable API contract	Thin-client conformance, API integration, and external contract testing	Two clients, no shared domain code; 4/4 conformance tests passed (Figure 4.7); 13/13 API integration tests passed (Figure 4.8); 4,759/4,759 external contract tests passed against the deployed API (Figure 4.13)	Satisfied
R3: Modular backend structure	<code>import-linter</code> + AST-based tests	7/7 contracts kept (79 files, 212 dependencies); 8/8 AST tests passed independently (Listings A.10, A.4)	Satisfied

Chapter 5

Discussion

5.1 Answering the Research Question

The research question asked how an automotive maintenance system can be architected with a modular backend that centralizes ingestion, normalization, and domain reasoning logic, while allowing multiple client applications to operate as thin presentation layers. The validation results in Chapter 4 show that all three derived requirements were satisfied in the delivered prototype, and the requirement traceability summary in Table 4.4 maps each requirement to its evidence. Rather than restating that mapping here, this section develops three findings that the implementation revealed but the design phase did not anticipate.

Architectural intent can be made verifiable. The common alternative to enforced contracts, relying on naming conventions, code review, and developer discipline to maintain layer boundaries, offers no repeatable guarantee that the intended structure survives as a codebase grows. The conformance results in Section 4.4.3 show that import-linter and AST-based analysis, operating through independent mechanisms, agreed on all seven contracts across 79 files and 212 dependencies. When two independent tools produce the same verdict, the architecture is no longer a matter of trust but of evidence. This shifts the cost of architectural drift from late-stage discovery during code review to immediate failure at test time, and reduces reliance on developer discipline as the system evolves.

Thin-client conformance is an emergent property of API completeness. It is straightforward to state that clients should contain no domain logic; it is harder to verify, and harder still to achieve. The experience of developing two independent clients against the same backend revealed that thinness cannot be enforced directly through client-side rules alone. When the backend is genuinely self-contained, clients remain thin because there is nothing left for them to implement. When the API contract is incomplete, clients reimplement missing logic locally, regardless of any prohibition against doing so. This reframes R1 and R2 from independent requirements into a single architectural property: the absence of client-side domain logic is a diagnostic for the completeness of the backend contract, not a constraint on client developers.

Behavioural correctness and structural conformance are independent failure modes. The validation strategy combined behavioural unit testing of the scoring engine with structural conformance testing of the codebase. These methods address two distinct failure modes that neither alone can catch. A system can pass every behavioural test while violating its intended architecture, because tests of input-output behaviour are blind to which module produced the output. A system can satisfy every structural contract while producing incorrect domain results, because import graphs are blind to computational correctness. The

combined approach used here is therefore not redundant but complementary, and represents a transferable validation pattern beyond the specific domain of vehicle maintenance. Taken together, these three findings establish that a layered FastAPI backend with enforced module boundaries is a viable and verifiable solution to the problem identified in Chapter 1. The related work review in Section 2.9 identified that existing applications restrict users to a single platform or operate within closed proprietary systems not designed for independently developed clients. CarMinder’s thin-client architecture addresses this limitation, and the validation evidence supports the claim structurally rather than anecdotally.

5.2 Validity of the Validation Approach

The conclusions in Section 5.1 depend on the validity of the validation approach itself. Three threats to validity are present, and they differ in severity.

The most serious threat is that the architectural requirements, design decisions, and validation instruments were all produced and evaluated by the same team, which introduces the risk that the validation methods were designed to confirm the architecture rather than to challenge it. Two partial mitigations apply. First, the structural validation tools, import-linter and AST-based analysis, operate through independent mechanisms and were configured separately, so a false positive from confirmation bias would have to appear in both parallel sets of contracts, which is substantially harder than in one. Second, the Schemathesis run reported in Section 4.4.4 derives test cases from the OpenAPI document using an externally authored tool, so for R2 specifically the tests themselves are not products of the team. Neither mitigation eliminates the underlying risk, because the team still selected which tools to apply and which contracts to encode; a stronger study would have an external software architect define the contracts before implementation began.

The second threat concerns the behavioural test suite that supports R1. The suite covers seven distinct properties of the scoring engine but does not constitute exhaustive coverage of all domain logic in the system. The service interval engine and the normalization layer were not subjected to the same level of behavioural testing. This is a coverage gap rather than a methodological flaw: the tests that exist are valid, but additional tests would strengthen the evidence for R1 across all backend components. Mutation testing of the scoring domain (Section 4.4.5) both strengthens and qualifies this part of the validity argument. It strengthens the argument because the behavioural tests are themselves probed rather than trusted: 365 of 1185 mutants were detected, providing direct evidence that the tests respond to behavioural change in the scoring logic. It qualifies the argument because the resulting mutation score of 30.8 percent is below the 60 to 80 percent range reported as typical of mature test suites. Two considerations frame how this result should be interpreted. First, the behavioural test suite was scoped to verify the seven specific properties listed in Section 3.4.2: determinism, runtime isolation, grade-boundary mapping, confidence dampening, the directional effect of good maintenance, the effect of severe unrepaired incidents, and the relative penalty for repaired versus unrepaired incidents. The suite was therefore designed as a property-based check on R1 rather than as an exhaustive functional specification of every scoring rule. The mutation score reflects this scope: the tests detect changes that affect the properties they target, and survive changes that do not. Second, extending the suite to raise the mutation score was not pursued in the available project time. Strengthening the suite is a direct continuation of the existing approach and would not require any architectural change, since the scoring engine is already isolated as a pure domain module. This extension is identified as future work in Section 7.4. The overall threat is acceptable for a bachelor-scope prototype and represents a direct extension for future work.

The third threat is that thin-client validation relies on a combination of independent client development, runtime observation of two specific dependency patterns, and pattern-based

static analysis of forbidden idioms (Listings A.1, A.2, A.3). The conformance suite catches known violations but does not enumerate every possible business rule that could be reimplemented in a client. A more rigorous approach would apply equivalent full-coverage static analysis tooling to the client codebases. This threat is the least severe because thin-client behaviour is also supported by the structural fact that two independently developed clients implemented equivalent functionality without shared domain code, which is observable independent of any test suite.

5.3 Architectural Trade-offs and Limitations

The architecture as delivered involves several trade-offs and deliberate design choices worth acknowledging explicitly. This section addresses both the limitations of the delivered system and the key design alternatives that were considered and rejected.

Implementation Trade-offs

The incremental refinement of the architecture during the project, as acknowledged in Section 3.7, meant that some design decisions were made later than ideal. This reflects a practical reality of design science research: architectural understanding deepens through implementation, and the final structure of the system reflects lessons learned during development rather than a fully formed up-front design. The result is a system that is more grounded in actual requirements, but one where early implementation work was occasionally revised as the architectural intent became clearer.

The system’s dependence on external services introduces points of fragility that the architecture itself cannot fully mitigate. The vehicle onboarding flow depends on the Norwegian Vegvesen registry API, which is outside the team’s control. The integration is isolated in a dedicated service module, reducing the impact of upstream changes, but no fallback mechanism was implemented. Similarly, Firebase Authentication introduces dependency on an external identity provider, which limits fine-grained control over authentication behaviour [29]. In the context of this prototype, delegating identity management allowed the team to focus on domain-specific logic and backend consistency, but a production system might require more flexible identity infrastructure.

Controlled vocabulary columns in the database are enforced at both the application layer through backend validators and the database layer through `CHECK` constraints, providing two independent integrity boundaries. However, the `cars` table mixes registry-sourced fields with application-owned state in a single row. A stricter design would separate these concerns into distinct tables, reducing the risk of accidental mutation of externally sourced data. This consolidation was a deliberate pragmatic choice given prototype scope, but represents a limitation for a production-oriented version of the system.

Design Alternatives Considered

Several design alternatives were evaluated and rejected during the architectural design process. Each decision is discussed below in terms of what was considered and why the chosen approach was more appropriate for this system taking into consideration the time constraint as well.

Backend architecture. A microservices architecture was evaluated as an alternative to the monolithic FastAPI backend. Microservices would offer independent deployability and finer grained scalability per domain boundary, but introduce significant operational overhead in service discovery, inter-service communication, and distributed tracing [12]. This overhead is disproportionate for a prototype system with a small development team and a clearly

bounded domain. A monolithic backend with enforced internal layer boundaries achieves the same separation of concerns at a fraction of the operational cost, making it the more appropriate choice at this scale [11].

Database technology. A document store such as Firebase Firestore was evaluated as an alternative to PostgreSQL. A document store offers schema flexibility, which may appear attractive given the difference nature of maintenance inputs described in Section 2.3. However, the CarMinder data model is fundamentally relational: maintenance events, incident reports, OBD readings, and notification logs all reference a central `cars` entity via foreign keys, and ownership and transfer logic depends on referential integrity enforced at the database level. A document store would require this integrity to be managed in application code, reintroducing the consistency risks that a relational schema eliminates. PostgreSQL hosted on Supabase was therefore the more appropriate choice.

Conformance tooling. Import-linter was selected over alternatives such as custom AST lint rules or relying on code review alone to enforce architectural boundaries. Import-linter supports declarative contract definitions that map directly to the layered structure described in Section 2.6, and produces repeatable automated output that can be integrated into a test pipeline. To further strengthen the conformance argument, a parallel set of AST-based tests was implemented independently, ensuring that the architectural boundaries are verified through two mechanisms rather than one.

API style. REST was selected over alternatives such as GraphQL or gRPC. GraphQL offers flexible client-driven querying, which can reduce over fetching in data rich applications, but introduces additional complexity in schema management and query validation that conflicts with the goal of keeping the API boundary simple and stable. gRPC offers strong typing and efficient binary serialisation, but requires specialised client libraries that would couple each client to a specific transport mechanism, undermining the principle of client replaceability described in Section 2.7. REST’s stateless, resource-oriented design maps naturally to the CarMinder data model, where vehicles, maintenance events, and incident reports are discrete addressable resources [14], and its uniform interface ensures that any client regardless of platform or implementation language can consume the API without specialised tooling [10], directly supporting the thin client architecture established in Section 2.7.

Mobile client framework. Expo React Native was selected over native iOS and Android development. Native development would offer maximum platform specific performance and access to the full range of device APIs, but would require maintaining two separate codebases, one per platform, which doubles the implementation effort without architectural benefit in this context. Since the mobile client is designed as a thin presentation layer with no domain logic of its own, platform-specific performance optimisation is not a primary concern. Expo React Native allows a single codebase to target both iOS and Android, keeping the client lightweight and consistent with the thin client principle that clients should be independently replaceable without duplicating backend functionality.

OBD-II protocol handling and the thin client boundary. The OBD-II feature introduces a bounded deviation from the thin client principle that warrants explicit acknowledgement. BLE is a local hardware transport: the ELM327 adapter communicates via a serial-over-BLE UART characteristic, and the raw byte stream it produces is only accessible to the process holding the active GATT connection on the device. Forwarding raw OBD frames to a remote backend in real time would require a persistent relay connection whose latency and reliability characteristics are incompatible with the synchronous PID polling loop that

ELM327 adapters require. Decoding on-device is therefore a constraint imposed by the transport layer rather than an architectural choice.

The consequence is that signal decoding functions (`decodeRpm`, `decodeCoolantTemp`, `decodeSpeed`, `decodeEngineLoad`, `decodeVoltage`, `decodeStoredDtcs`) and a hardcoded DTC description map reside in the mobile client rather than the backend. Once decoded, the snapshot is submitted to the backend as a structured JSON payload, and all subsequent persistence, retrieval, and presentation logic follows the standard backend-centric path. The backend therefore owns the authoritative record; the client’s decoding step is a necessary transport adapter rather than a business rule.

The DTC description map is the more significant limitation of the two. It is a finite hardcoded vocabulary; codes outside that vocabulary are rendered as “Unknown diagnostic trouble code,” as observed during real-device testing on a Toyota Yaris, which produced P0221 and B1622, neither of which was present in the map. Moving DTC lookup to the backend as a dedicated service would restore full conformance with R1 and allow the description database to be extended without a client release. This is identified as a priority direction for future work in Section 7.4.

5.4 Security Trade-offs

CarMinder’s modular thin-client architecture introduces a clear security trade-off. Its main strength is that validation, authorization, and domain logic are centralized in the backend rather than distributed across the web and mobile clients. This reduces the need to trust client applications with security-critical decisions and make it easier to enforce the same rules consistently across platforms. It also makes patching and auditing easier, since security logic is concentrated in one place. This design aligns with OWASP guidance, which recommends that authentication and authorization be enforced server-side and that client-side controls be treated as bypassable [30].

The cost of this approach is that the backend becomes both the main security boundary and the main point of failure. If authorization checks are incomplete, the impact is not limited to one client, but can affect the entire system. This is especially relevant in CarMinder because the API exposes car-specific operations, public history lookup, PDF export, and ownership transfer flows. OWASP identifies broken object-level authorization as one of the most common API risks and states that every endpoint receiving an object identifier should enforce authorization checks on that object. Authentication is equally important. Firebase states that backend token verification must confirm that tokens are properly signed and unexpired, and revocation must be checked separately. For CarMinder, this means that thin clients reduce some client-side risks, but they make ownership checks, token validation, and endpoint protection much more critical [31].

Compared with a more web-specific Backend-for-Frontend (BFF) design, CarMinder’s shared REST backend is a pragmatic compromise. A BFF design offers a stronger browser-side security model because it can keep tokens on the server and use cookies protected with flags such as `HttpOnly` and `Secure`. In the running CarMinder web client, however, inspection of the browser storage shows an `auth_token` stored in `localStorage` under the application origin as it is stated in 5.1. That makes the token readable by any JavaScript executing in the same origin.

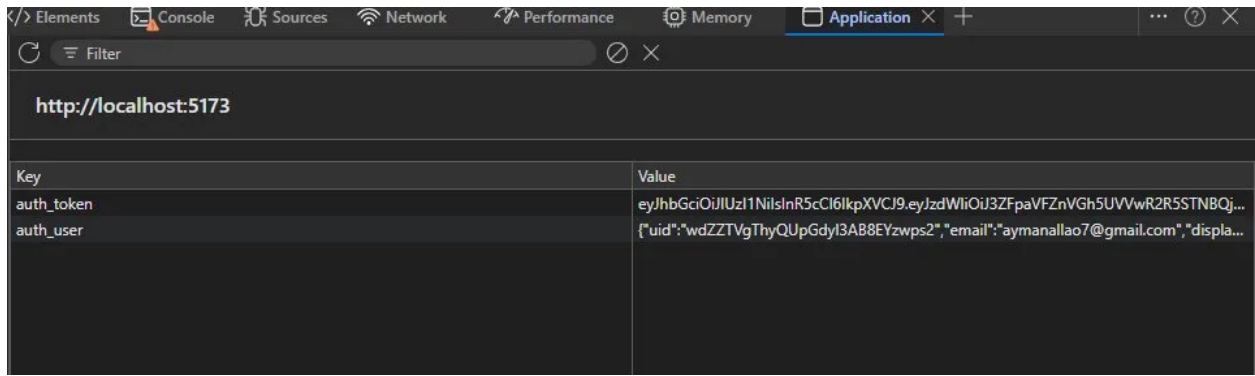


Figure 5.1: Local Storage inspection of the CarMinder web client, showing Firebase ID token stored under the key `auth_token`.

This matters because OWASP explicitly advises against storing session identifiers in `localStorage`, since the data is always accessible to JavaScript. The current IETF guidance for browser-based OAuth also describes BFF-style approaches as offering stronger security properties for browser applications, and specifies that BFF cookies should use protections such as `HttpOnly` and `Secure`. In CarMinder, the decision to use a shared backend for both web and mobile simplifies the architecture and supports the project’s cross-platform goal. The trade-off is that the web client does not benefit from the stronger token isolation that a dedicated BFF would provide [32].

Compared with a direct client-to-database or Backend-as-a-service approach, CarMinder’s backend provides tighter control over workflows such as public sharing, ownership transfer, and centralized validation. That is valuable in a system where domain rules and access control are closely tied together. At the same time, it creates a larger custom API surface that must be secured and monitored. Supabase documents that Row Level Security can provide defense in depth even when data is accessed through third-party tooling [33], which suggests that a more database-centric design could reduce some application-layer responsibility. However, such an approach would make it harder to keep all security and domain rules in one place. For CarMinder specifically, the implication is that future security work should focus on hardening the backend boundary rather than moving logic into the clients. The most important priorities are strict object-level authorization, safer web token handling, rate limiting, and careful protection of the public-history and ownership-transfer features.

5.5 Generalizability of the Approach

The CarMinder system is a single constructed artefact, and the primary contribution of this thesis is the architectural pattern it demonstrates rather than the system itself. The pattern combines enforced layer boundaries, a pure domain module, and automated conformance testing into a coherent approach to centralizing domain reasoning while keeping clients thin and independently replaceable.

This pattern is not specific to automotive maintenance. The same layered structure, the same import-linter contract approach, and the same thin client validation strategy could be applied to any domain where multiple client platforms need to consume a shared backend without duplicating business logic. Medical record management, logistics tracking, and asset maintenance systems represent plausible candidate domains where the same architectural problem recurs.

However, the thesis demonstrates this pattern for a single domain with a single technology stack. Whether it transfers directly to systems with significantly different domain complexity, team size, or technology constraints remains an open question. The conformance testing approach relies on import-linter and Python's `ast` module, both specific to the Python ecosystem. Equivalent tooling exists for other languages, but configuration and contract expressiveness may differ. The thesis therefore does not claim broad generalizability, but offers the pattern as a starting point for future instantiations in other domains and technology stacks.

Chapter 6

Social Perspective

6.1 Societal and Environmental Perspective

CarMinder can also be discussed from a broader societal perspective. McLuhan [1] is relevant because the system does more than store information, it changes how maintenance history is documented, remembered, and communicated between people. Latour [34] is useful because trust in a vehicle transaction is not created by the application alone, but by the interaction between users, interface design, backend rules, maintenance records, and public history settings. Jonas [35] adds an ethical dimension by reminding us that technologies that shape human decisions also expand the responsibility of those who design them.

6.1.1 Environmental implications of maintenance transparency

One possible societal benefit of CarMinder is its potential indirect contribution to more sustainable vehicle use. By making service history, incidents, reminders, and deadlines more visible over time, the system may support more consistent preventive maintenance. In McLuhan's terms, the application can be seen as an extension of the user's memory and documentation practices [1].

This does not mean that the prototype directly reduces emissions. However, it does support the argument that better maintenance transparency may encourage longer and more responsible use of existing vehicles. In that sense, CarMinder may indirectly support reuse and delayed replacement by making it easier for owners to keep track of the vehicle's condition over time. Jonas' perspective is relevant here because he argues that the cumulative effects of technological choices extend far beyond their immediate context, and that this expanded reach also expands the responsibility of those who build such systems [35].

6.1.2 Transparency, accident reporting, and trust in the second-hand market

A more direct societal implication of CarMinder concerns transparency in the second-hand vehicle market. The system brings together maintenance events, incident reports, and public vehicle history into one coherent record, making it easier for a seller to present a structured account of a car's past and for a buyer to inspect that account before purchase.

The accident-reporting feature is particularly important in this case. If a user records that a car has been involved in an accident, together with details about severity and whether the damage was repaired, the system can provide a stronger basis for transparency during resale. Instead of relying only on informal explanation, the transaction can be supported by a documented history of maintenance and damage-related events. In this way, the digital record changes how trust is negotiated between seller and buyer, which is closely aligned with

McLuhan's view that media reshape human relations rather than simply carry information [1].

Latour's actor-network theory helps explain why this matters socially. In CarMinder, trust is not produced by the application alone, but through the relationship between users, technical design, recorded events, and the buyer's interpretation of the presented history [34]. The system therefore does not only manage vehicle data; it also helps structure the social conditions under which second-hand vehicle transactions are understood.

Chapter 7

Conclusions

7.1 Summary of Work

This thesis presented the design, implementation, and validation of CarMinder, a vehicle maintenance management system built around a modular backend architecture. The work was motivated by two limitations identified in existing applications: the restriction of maintenance data access to a single platform, and the fragmented presentation of maintenance records from multiple input sources.

The CarMinder system was designed to address this limitation through a layered FastAPI backend that centralizes all domain reasoning and a stable REST API contract consumed by two independently developed client applications. A React web client and an Expo React Native mobile client were developed independently against the same backend without shared domain code. The architecture was validated through five complementary methods: automated architectural conformance testing using import-linter and AST-based analysis, behavioural unit testing of the scoring engine in isolation, thin client conformance testing through static analysis of client source files, external schema-based contract testing of the deployed API using Schemathesis, and mutation testing of the scoring domain using mutmut. All three architectural requirements derived from the problem context were satisfied in the delivered prototype.

This thesis set out to investigate how domain logic can be centralized in a modular backend architecture while supporting multiple independent client applications. The results demonstrate that this is both feasible and effective when strict architectural boundaries are enforced and verified through automated tooling rather than relied on through convention.

7.2 Conclusion

Three findings from the discussion in Section 5.1 carry beyond the specific case of CarMinder. First, architectural intent can be made verifiable: independent structural analysis tools produced agreeing verdicts on seven contracts, replacing trust with evidence. Second, thin-client conformance is an emergent property of API completeness rather than a constraint on client developers: when the backend is genuinely self-contained, clients have nothing left to reimplement. Third, behavioural correctness and structural conformance are independent failure modes that require independent validation methods; combining them addresses both, while either alone leaves a meaningful gap.

The key takeaway is that backend-centric architectures, when combined with enforceable structural constraints and automated validation, provide a robust foundation for systems that require consistent interpretation across multiple clients. The pattern demonstrated

here is not specific to vehicle maintenance and represents a transferable approach for software systems facing the same recurring problem.

7.3 Limitations

The OBD-II reading pipeline was implemented and validated on two physical vehicles, a Mazda 3 and a Toyota Yaris, using a real ELM327 Bluetooth adapter. Signal decoding is performed on-device prior to backend upload, as the BLE transport does not permit raw frame forwarding to a remote endpoint. The DTC description lookup is currently implemented as a hardcoded client-side map; codes absent from the map are displayed as “Unknown diagnostic trouble code,” as encountered with codes P0221 and B1622 during testing on the Toyota Yaris. Relocating the DTC lookup to the backend remains an identified direction for future work.

Receipt scanning was implemented as an exploratory prototype extension in which receipt images are uploaded from the mobile client and processed by the backend using OCR to suggest candidate maintenance fields for user review. However, this feature was not included in the formal validation of unified multi-source maintenance history, and manual entry remains the primary validated non-diagnostic input source in the current prototype.

No usability testing was conducted with real users, which means the system’s effectiveness from a user experience perspective remains unevaluated. The architectural requirements, design decisions, and validation instruments were all produced and evaluated by the same team, introducing the risk that the validation approach was designed to confirm the architecture rather than to challenge it. Additionally, the system does not enforce verification of vehicle ownership at registration.

Mutation testing of the scoring domain achieved a mutation score of 30.8 percent. The behavioural test suite detects grade-boundary changes and broad directional properties but does not fully specify confidence-threshold behaviour, normalization behaviour, maintenance interval scoring, documentation-quality scoring, or the exact category and overall score calculations. A total of 204 mutants had no associated tests. Of these, 202 were in `normalize.py` and 2 were in `engine.py`. The implications of this for R1 validity are discussed in Section 5.2, and the concrete extension is identified in Section 7.4.

7.4 Future Work

Relocating the DTC description lookup from the mobile client to a dedicated backend service would restore full thin client conformance for the OBD-II feature and allow the diagnostic code database to be extended without requiring a client release. Two approaches are viable. The first is a static backend DTC database mapping standard SAE and manufacturer codes to plain-language descriptions; this approach guarantees deterministic output and is straightforward to extend incrementally. The second is an LLM-powered diagnostic service that receives the decoded DTC codes alongside relevant vehicle metadata such as make, model, year, and engine type, and generates contextual explanations either from training data or via retrieval-augmented search. Such a service could produce richer, vehicle-specific descriptions than a static map, but introduces non-determinism and dependency on an external model. A static database is the more appropriate first step given the accuracy requirements of diagnostic information; an LLM-based service represents a viable extension once a reliable baseline is established. Because the backend is already organised as a modular layered system, either approach could be introduced as an isolated domain service without affecting the existing API contract or client applications.

Strengthening the scoring test suite to address the gaps revealed by mutation testing in

Section 4.4.5 is a second direction for future work. The mutation results identified weaker coverage around confidence-threshold boundaries (in particular the 0.25 and 0.5 thresholds), the normalization layer in `normalize.py` where 202 mutants had no associated tests, maintenance interval scoring, documentation-quality scoring, and the exact category-score and overall-score calculations. Adding direct unit tests targeting these areas would raise the mutation score toward the 60 to 80 percent range reported as typical of mature test suites and would in turn provide stronger evidence for R1. Because the scoring engine is already isolated as a pure domain module with no infrastructure dependencies, extending its test suite does not require changes to any other part of the system.

A further direction for future work is to examine the broader societal effect of the system empirically. The present thesis argues that CarMinder may support more transparent maintenance documentation, stronger trust in second-hand sales, and potentially more responsible long-term vehicle use, but these implications have not been tested with real users. Future studies could therefore investigate whether the system changes how owners document maintenance, whether buyers recognize public accident and maintenance history as useful and trustworthy, and how the system affects the social dynamics of used-car transactions.

Finally, the architectural pattern demonstrated in this thesis is not specific to automotive maintenance. Applying the same layered backend structure, enforced module boundaries, and automated conformance testing approach to other domains such as medical record management or logistics tracking represents a natural direction for future instantiations of the pattern.

Appendix A

Selected Test Units

This appendix includes selected test units that are most relevant to the thesis claims about modular backend architecture, centralized domain reasoning, and thin-client behaviour.

A.0.1 Thin-client conformance tests

Listing A.1: Client test preventing duplicated score grade logic

```
def test_clients_do_not_duplicate_score_grade_thresholds() -> None:
    """
    Clients may render backend-provided grades and color-code score bars,
    but they must not re-implement the backend's grade assignment rules
    (A/B/C/D/F thresholds).
    """
    forbidden_patterns = [
        re.compile(r"grade\s*=\s*score\s*>=\s*90"),
        re.compile(r"grade\s*=\s*overall_score\s*>=\s*90"),
        re.compile(r"if\s*\(\s*(?:overall_)?score\s*>=\s*90\s*\).*['\"]A['\"]", ...
            re.DOTALL),
        re.compile(r"if\s*\(\s*(?:overall_)?score\s*>=\s*75\s*\).*['\"]B['\"]", ...
            re.DOTALL),
        re.compile(r"if\s*\(\s*(?:overall_)?score\s*>=\s*60\s*\).*['\"]C['\"]", ...
            re.DOTALL),
        re.compile(r"if\s*\(\s*(?:overall_)?score\s*>=\s*40\s*\).*['\"]D['\"]", ...
            re.DOTALL),
    ]

    offenders: list[str] = []
    for path in _client_files():
        text = _read(path)
        for pattern in forbidden_patterns:
            if pattern.search(text):
                offenders.append(f"{path.relative_to(REPO_ROOT)} matches ...
                    {pattern.pattern}")

    assert not offenders, (
        "Client files must not duplicate backend grade thresholds.\n"
        + "\n".join(offenders)
    )
```

Listing A.2: Client test preventing duplicated service urgency logic

```
def test_clients_do_not_duplicate_service_urgency_thresholds() -> None:
    """
    Clients may display already-computed urgency values and remaining
```

```

days/km, but they must not encode the backend service-interval
threshold rules locally.
"""
forbidden_patterns = [
    re.compile(r"days_until_due\s*<=\s*30"),
    re.compile(r"0\s*<=\s*days_until_due\s*<=\s*30"),
    re.compile(r"km_until_due\s*<=\s*1000"),
    re.compile(r"0\s*<=\s*km_until_due\s*<=\s*1000"),
    re.compile(r"urgency\s*=\s*['\"]soon['\"]"),
    re.compile(r"urgency\s*=\s*['\"]overdue['\"]"),
    re.compile(r"if\s*\(\s*days_until_due\s*<\s*0\s*\)\s*\{[^}]*urgency", re.DOTALL),
    re.compile(r"if\s*\(\s*km_until_due\s*<\s*0\s*\)\s*\{[^}]*urgency", re.DOTALL),
]

offenders: list[str] = []
for path in _client_files():
    text = _read(path)
    for pattern in forbidden_patterns:
        if pattern.search(text):
            offenders.append(f"{path.relative_to(REPO_ROOT)} matches ...
                             {pattern.pattern}")

assert not offenders, (
    "Client files must not duplicate backend urgency thresholds.\n"
    + "\n".join(offenders)
)

```

Listing A.3: Client test verifying consumption of backend DTOs

```

def test_clients_consume_backend_score_and_service_status_dtos() -> None:
    """
    Thin clients should consume backend DTOs rather than synthesizing
    score or service-interval structures locally.
    """
    web_car_detail = _read(WEB_CAR_DETAIL)
    mobile_car_detail = _read(MOBILE_CAR_DETAIL)
    web_home = _read(WEB_HOME)
    mobile_home = _read(MOBILE_HOME)
    web_api = _read(WEB_API)
    mobile_api = _read(MOBILE_API)

    assert "CarCareScoreResponse" in web_api
    assert "CarServiceStatus" in web_api
    assert "AllCarsServiceStatus" in web_api
    assert "CarCareScoreResponse" in mobile_api
    assert "CarServiceStatus" in mobile_api
    assert "AllCarsServiceStatus" in mobile_api

    for text, label in [
        (web_car_detail, "web car detail"),
        (mobile_car_detail, "mobile car detail"),
        (web_home, "web home"),
        (mobile_home, "mobile home"),
    ]:
        assert "overall_score" in text, f"{label} should render backend score fields"
        assert "grade" in text, f"{label} should render backend grade fields"
        assert "confidence_label" in text, f"{label} should render backend confidence ...
            fields"

```

Architecture boundary tests

Listing A.4: Architecture boundary test for layered backend modularity

```
def test_routers_do_not_import_repositories_or_domain() -> None:
    _assert_no_imports("routers", {"repositories", "domain"})

def test_services_do_not_import_routers() -> None:
    _assert_no_imports("services", {"routers"})

def test_repositories_do_not_import_upward_layers() -> None:
    _assert_no_imports("repositories", {"routers", "services", "domain", "schemas"})
```

A.0.2 Scoring engine tests

Listing A.5: Scoring engine determinism test

```
def test_scoring_is_deterministic():
    result_1 = compute_score(BASE_CAR, [GOOD_OIL_CHANGE], [], REFERENCE_DATE, ...
                             DEFAULT_INTERVALS)
    result_2 = compute_score(BASE_CAR, [GOOD_OIL_CHANGE], [], REFERENCE_DATE, ...
                             DEFAULT_INTERVALS)
    result_3 = compute_score(BASE_CAR, [GOOD_OIL_CHANGE], [], REFERENCE_DATE, ...
                             DEFAULT_INTERVALS)

    assert result_1.overall_score == result_2.overall_score == result_3.overall_score
    assert result_1.grade == result_2.grade == result_3.grade
    assert result_1.confidence == result_2.confidence == result_3.confidence
    assert result_1.confidence_label == result_2.confidence_label == ...
           result_3.confidence_label
```

Listing A.6: Scoring engine metamorphic test for positive maintenance effect

```
def test_adding_good_maintenance_event_does_not_lower_score():
    score_without = compute_score(
        BASE_CAR, [], [], REFERENCE_DATE, DEFAULT_INTERVALS
    ).overall_score

    score_with = compute_score(
        BASE_CAR, [GOOD_OIL_CHANGE], [], REFERENCE_DATE, DEFAULT_INTERVALS
    ).overall_score

    assert score_with >= score_without
```

Listing A.7: Scoring engine metamorphic test for severe unrepaired incident

```
def test_severe_unrepaired_incident_lowers_score():
    score_no_incident = compute_score(
        BASE_CAR, [GOOD_OIL_CHANGE], [], REFERENCE_DATE, DEFAULT_INTERVALS
    ).overall_score

    score_with_incident = compute_score(
        BASE_CAR, [GOOD_OIL_CHANGE], [SEVERE_INCIDENT], REFERENCE_DATE, DEFAULT_INTERVALS
    ).overall_score

    assert score_with_incident < score_no_incident
```

A.0.3 API contract tests

Listing A.8: API test for successful score retrieval

```
def test_get_car_score_success(authenticated_client: TestClient, monkeypatch) -> None:
    expected = build_score_response()

    def fake_compute_car_care_score(uid: str, car_id: str) -> CarCareScoreResponse:
        assert uid == "test-user-123"
        assert car_id == "car-123"
        return expected

    monkeypatch.setattr(
        "routers.car_score.scoring_service.compute_car_care_score",
        fake_compute_car_care_score,
    )

    response = authenticated_client.get("/cars/car-123/score")

    assert response.status_code == 200
    body = response.json()
    assert body["car_id"] == "car-123"
    assert body["overall_score"] == 82
    assert body["grade"] == "B"
```

Listing A.9: API test for invalid maintenance-event payload rejection

```
def test_create_maintenance_event_rejects_invalid_payload(
    authenticated_client: TestClient,
) -> None:
    response = authenticated_client.post(
        "/cars/car-123/events",
        json={
            "event_type": "made_up_type",
            "event_date": "2099-01-01",
        },
    )

    assert response.status_code == 422
    errors = response.json()["detail"]
    messages = [error["msg"] for error in errors]
    assert any("event_type must be one of" in message for message in messages)
    assert any("event_date cannot be in the future" in message for message in messages)
```

A.0.4 Import-linter contract

Listing A.10: Import-linter contracts enforcing backend layering

```
[importlinter]
root_packages =
    routers
    services
    repositories
    domain
    schemas
    config
include_external_packages = True

[importlinter:contract:layers]
name = Enforce layered architecture
```

```

type = layers
layers =
    routers
    services
    repositories
    domain

[importlinter:contract:domain-isolation]
name = Domain layer must not import from any other internal layer
type = forbidden
source_modules =
    domain
forbidden_modules =
    routers
    services
    repositories
    config
    schemas

[importlinter:contract:routers-no-domain]
name = Routers must not access domain layer directly
type = forbidden
source_modules =
    routers
forbidden_modules =
    domain

[importlinter:contract:routers-no-repositories]
name = Routers must not access repositories directly
type = forbidden
source_modules =
    routers
forbidden_modules =
    repositories
allow_indirect_imports = True

[importlinter:contract:repositories-no-upward-deps]
name = Repositories must not depend on higher layers
type = forbidden
source_modules =
    repositories
forbidden_modules =
    routers
    services
    domain
    schemas

[importlinter:contract:services-no-routers]
name = Services must not depend on routers
type = forbidden
source_modules =
    services
forbidden_modules =
    routers

[importlinter:contract:schemas-isolation]
name = Schemas must remain isolated from app layers
type = forbidden
source_modules =
    schemas
forbidden_modules =

```

```
    routers
    services
    repositories
    domain
    config
```

A.0.5 AST-based architecture helpers

Listing A.11: AST-based helper functions for static architecture checks

```
from __future__ import annotations

import ast
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parents[1]
INTERNAL_ROOTS = {"routers", "services", "repositories", "domain", "schemas", "config"}

def _layer_files(layer: str) -> list[Path]:
    base = PROJECT_ROOT / layer
    return sorted(p for p in base.rglob("*.py") if p.name != "__init__.py")

def _top_level_imports(py_file: Path) -> set[str]:
    imports: set[str] = set()
    tree = ast.parse(py_file.read_text(encoding="utf-8"))
    for node in ast.walk(tree):
        if isinstance(node, ast.Import):
            for alias in node.names:
                root = alias.name.split(".", 1)[0]
                if root in INTERNAL_ROOTS:
                    imports.add(root)
        elif isinstance(node, ast.ImportFrom):
            if node.level > 0:
                continue
            if not node.module:
                continue
            root = node.module.split(".", 1)[0]
            if root in INTERNAL_ROOTS:
                imports.add(root)
    return imports

def _assert_no_imports(layer: str, forbidden_roots: set[str]) -> None:
    offenders: list[str] = []
    for py_file in _layer_files(layer):
        imported = _top_level_imports(py_file)
        overlap = sorted(imported & forbidden_roots)
        if overlap:
            rel = py_file.relative_to(PROJECT_ROOT)
            offenders.append(f"{rel} imports {'', '.join(overlap)}")
    assert not offenders, "Layer violations:\n" + "\n".join(offenders)
```

AST and contract verification tests

Listing A.12: Tests verifying import-linter contracts and router wiring through AST

```
from __future__ import annotations
```

```

import ast
import configparser
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parents[1]
IMPORTLINTER_PATH = PROJECT_ROOT / ".importlinter"
MAIN_PATH = PROJECT_ROOT / "main.py"
ROUTERS_DIR = PROJECT_ROOT / "routers"

def test_importlinter_contains_required_contracts() -> None:
    parser = configparser.ConfigParser()
    parser.read(IMPORTLINTER_PATH)

    expected_sections = {
        "importlinter:contract:layers",
        "importlinter:contract:domain-isolation",
        "importlinter:contract:routers-no-domain",
        "importlinter:contract:routers-no-repositories",
        "importlinter:contract:repositories-no-upward-deps",
        "importlinter:contract:services-no-routers",
        "importlinter:contract:schemas-isolation",
    }

    missing = sorted(section for section in expected_sections if section not in parser)
    assert not missing, f"Missing import-linter contracts: {'', '.join(missing)}"

def test_main_imports_all_router_modules() -> None:
    tree = ast.parse(MAIN_PATH.read_text(encoding="utf-8"))

    imported_router_modules: set[str] = set()
    for node in ast.walk(tree):
        if isinstance(node, ast.ImportFrom) and node.module == "routers":
            imported_router_modules.update(alias.name for alias in node.names)

    expected = {
        p.stem
        for p in ROUTERS_DIR.glob("*.py")
        if p.name != "__init__.py"
    }
    missing = sorted(expected - imported_router_modules)
    assert not missing, f"Router modules not imported in main.py: {'', '.join(missing)}"

def test_main_includes_each_router_module() -> None:
    tree = ast.parse(MAIN_PATH.read_text(encoding="utf-8"))

    included_modules: set[str] = set()
    for node in ast.walk(tree):
        if not isinstance(node, ast.Call):
            continue
        if not isinstance(node.func, ast.Attribute):
            continue
        if node.func.attr != "include_router":
            continue
        if not node.args:
            continue
        arg = node.args[0]
        if isinstance(arg, ast.Attribute) and isinstance(arg.value, ast.Name):

```



```

        included_modules.add(arg.value.id)

expected = {
    p.stem
    for p in ROUTERS_DIR.glob("*.py")
    if p.name != "__init__.py"
}
missing = sorted(expected - included_modules)
assert not missing, f"Router modules not included via app.include_router: {'', ...
    '.join(missing)}"

```

Listing A.13: Scoring engine test for empty vehicle history

```

def test_empty_car_produces_dampened_score():
    """
    A car with no events, no incidents, no mileage, and no EU deadline
    must receive a score dampened toward 50 and a low or very_low
    confidence label.

    Validates the confidence dampening mechanism: sparse data must not
    produce misleadingly high or low scores.
    """
    result = compute_score(
        EMPTY_CAR, [], [], REFERENCE_DATE, DEFAULT_INTERVALS
    )

    assert 35 <= result.overall_score <= 65, (
        f"Empty car score {result.overall_score} is outside expected "
        f"dampened range [35, 65]"
    )
    assert result.confidence_label in ("very_low", "low"), (
        f"Empty car confidence label should be very_low or low, "
        f"got {result.confidence_label}"
    )

```

Listing A.14: GitHub Actions workflow executing the full backend validation suite and a production web build on every push to main, api-testing, and frontend branches, and on every pull request.

```

name: CI

on:
  push:
    branches:
      - main
      - api-testing
      - frontend
  pull_request:

jobs:
  backend-tests:
    name: Backend Tests
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.13"

```

```

    cache: pip
    cache-dependency-path: backend-python/requirements.txt

- name: Install backend dependencies
  run: |
    python -m pip install --upgrade pip
    pip install -r backend-python/requirements.txt pytest import-linter

- name: Run backend test suite
  working-directory: backend-python
  run: pytest

web-build:
  name: Web Build
  runs-on: ubuntu-latest

  steps:
    - name: Checkout
      uses: actions/checkout@v4

    - name: Set up Node
      uses: actions/setup-node@v4
      with:
        node-version: "20"
        cache: npm
        cache-dependency-path: web/package-lock.json

    - name: Install web dependencies
      working-directory: web
      run: npm ci

    - name: Build web app
      working-directory: web
      run: npm run build

```

A.1 Mutation Testing Artifacts

A.1.1 Reproduction Command

Listing A.15: Mutation testing reproduction command

```

cd backend-python
sh scripts/run-mutmut.sh

```

A.1.2 Killed Mutant Example

Listing A.16: Example of a killed grade-boundary mutant

```

# Original
def _score_to_grade(score: int) -> str:
    if score >= 90:
        return "A"
    if score >= 75:
        return "B"

# Mutant generated by mutmut
def _score_to_grade(score: int) -> str:
    if score > 90:
        return "A"

```

```

if score >= 75:
    return "B"

```

A.1.3 Survived Mutant Example

Listing A.17: Example of a surviving mutant

```

# Original
if confidence < 0.25:
    return "very_low"

# Mutant
if confidence <= 0.25:
    return "very_low"

```

A.1.4 Mutation Testing Wrapper Script

Listing A.18: Mutation testing wrapper script

```

#!/usr/bin/env sh
set -eu

script_dir=$(CDPATH= cd -- "$(dirname -- "$0")" && pwd)
backend_dir=$(dirname "$script_dir")

cd "$backend_dir"
export ...
    PYTHONPATH="$backend_dir/tools/mutmut_no_setproctitle${PYTHONPATH:+:$PYTHONPATH}"
exec "$backend_dir/.venv/bin/mutmut" run --max-children 1 "$@"

```

A.1.5 Per-File Mutation Breakdown

File	Killed	Survived	No tests
engine.py	365	616	2
normalize.py	0	0	202
Total	365	616	204

Table A.1: Mutation results broken down by source file.

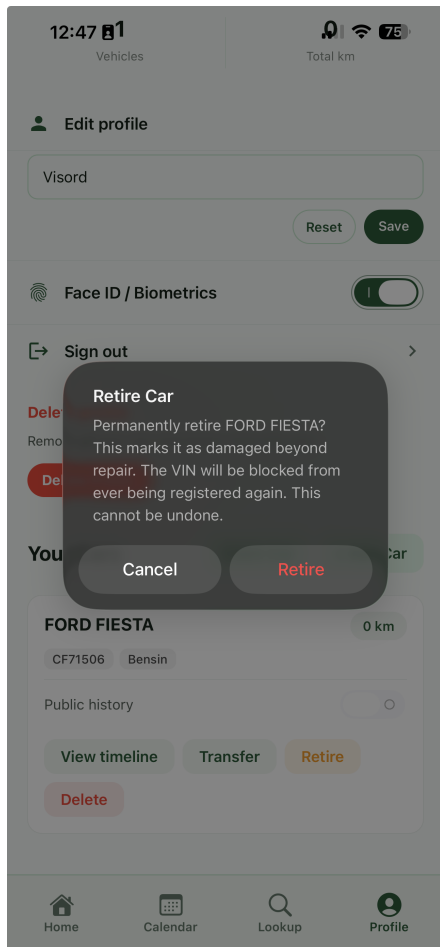
The breakdown shows that nearly all evaluated mutations were generated in `engine.py`, where the scoring logic is implemented. The `engine.py` results include both killed and surviving mutants, meaning the current tests exercise this file but do not fully specify all scoring behavior. In contrast, all 202 mutants in `normalize.py` were classified as having no associated tests. This indicates that the normalization layer was not directly exercised by the mutation-test oracle and should be prioritized for additional unit tests.

Appendix B

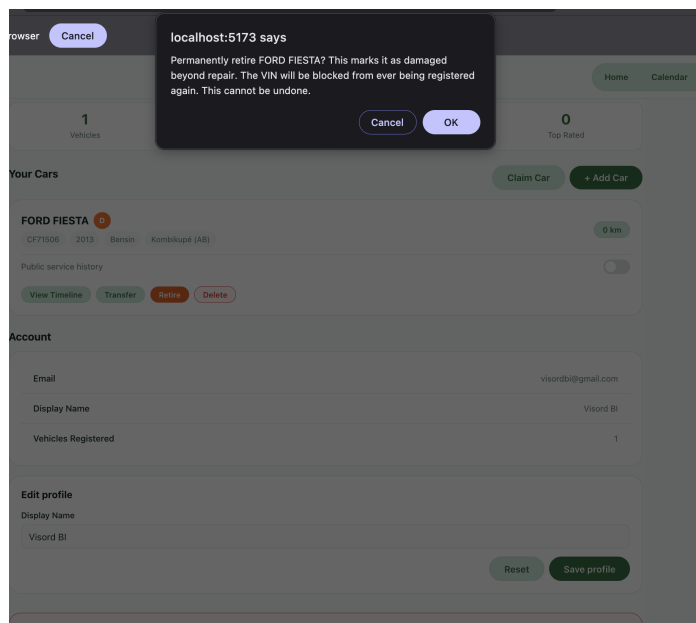
Client Application Screenshots

This appendix collects the user-interface screenshots referenced from Chapter 4. Architectural and validation figures (system architecture diagrams, sequence diagrams, database schema, scoring weights, transfer-flow diagram, and test-result outputs) remain inline in Chapter 4. The screenshots reproduced here illustrate the rendered output of each feature on both clients.

B.1 Vehicle Retirement Screenshots



(a) Mobile profile screen showing the destructive confirmation modal that names the vehicle, warns that the chassis number will be blocked from future registration, and states that the action cannot be undone.



(b) Web profile page showing the equivalent destructive confirmation, presented through the native browser dialog with identical warning text.

Figure B.1: Destructive confirmation step preceding the call to `POST /cars/{car_id}/retire` on the mobile (left) and web (right) clients. Both clients display the same warning text before invoking the backend retirement endpoint.

B.2 Notifications

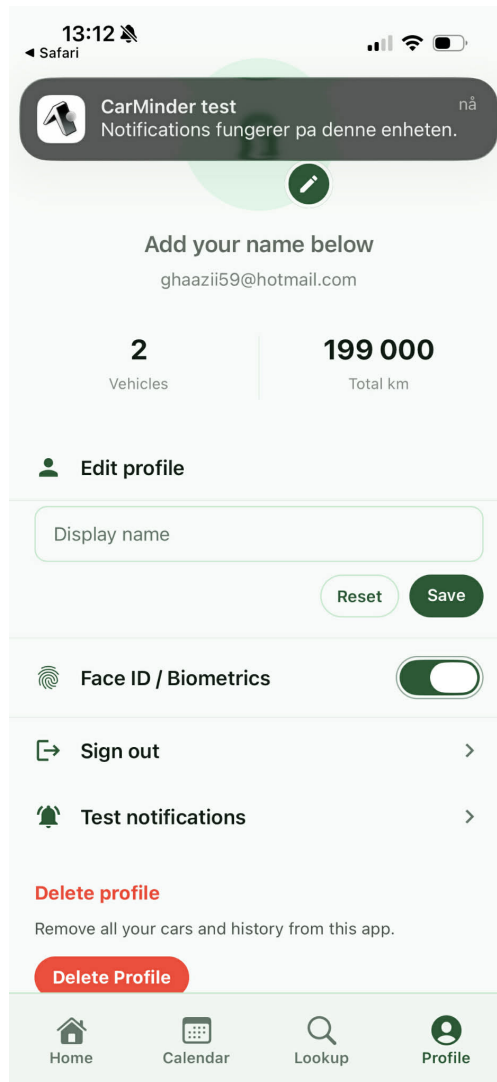


Figure B.2: Test push notification displayed on the mobile client, confirming successful delivery via the backend notification system. Referenced from the Notifications subsection of Chapter 4.

B.3 PDF Export

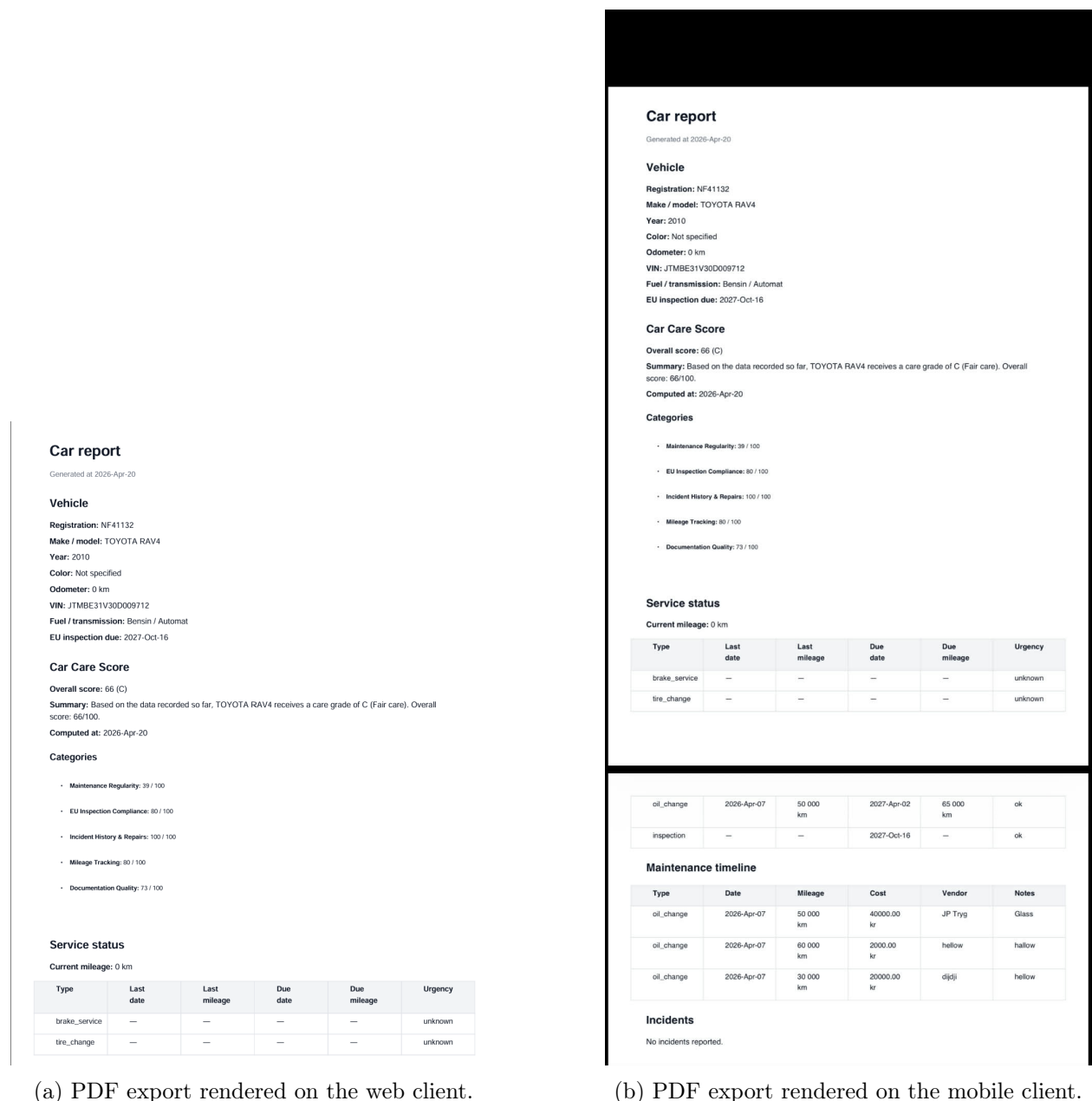
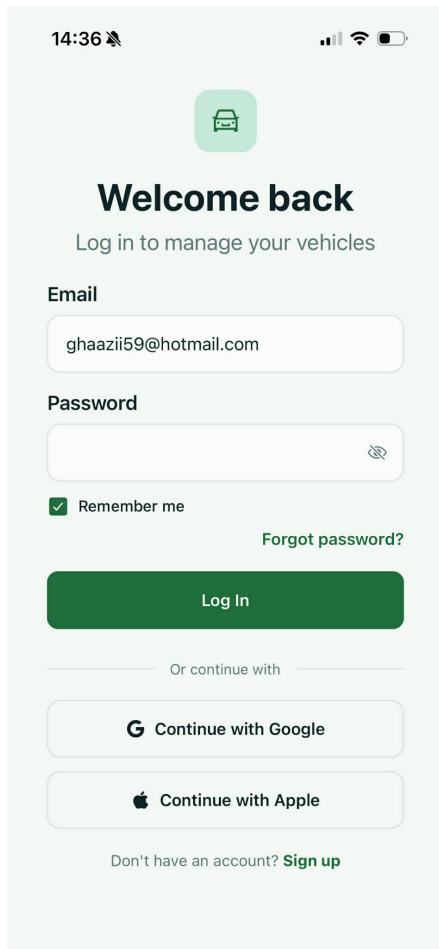


Figure B.3: PDF export output on both platforms. The document is generated entirely by the backend; the client only triggers the request and receives the file.

B.4 Authentication



14:36

📶

🚗

Welcome back

Log in to manage your vehicles

Email

ghaazii59@hotmail.com

Password

☒ Remember me

[Forgot password?](#)

Log In

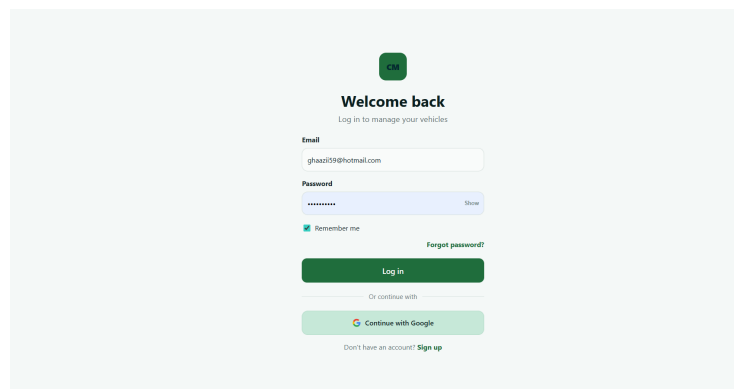
Or continue with

Continue with Google

Continue with Apple

Don't have an account? [Sign up](#)

(a) Mobile login screen offering email/password, Google, and Apple Sign-In options.



Welcome back

Log in to manage your vehicles

Email

ghaazii59@hotmail.com

Password

***** [Show](#)

☒ Remember me

[Forgot password?](#)

Log In

Or continue with

Continue with Google

Don't have an account? [Sign up](#)

(b) Web login screen offering email/password and Google Sign-In.

Figure B.4: Login screens on the mobile (left) and web (right) clients.

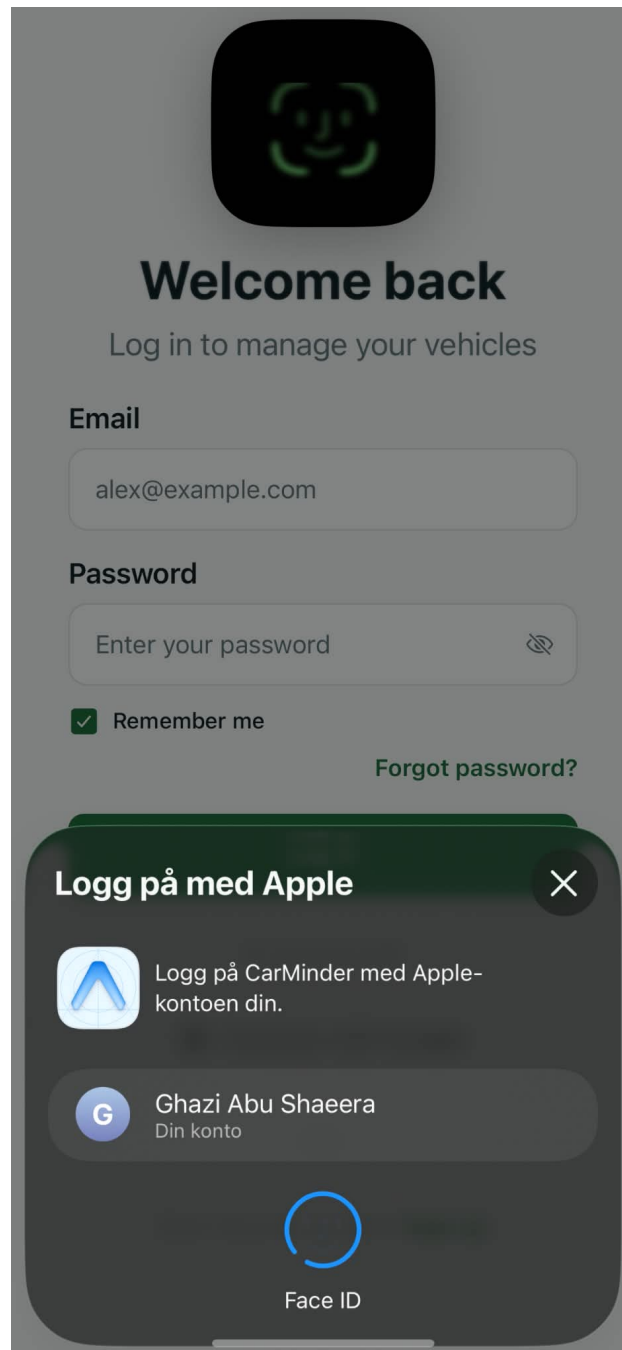


Figure B.5: Apple login

14:37

Create account

Create a new account to get started

Name

Email

Password

Confirm password

Create account

Already have an account? Log in

(a) Mobile registration screen for creating a new account with email and password.

CM

Create account

Create a new account to get started

Name

Email

Password

Confirm password

Create account

Or continue with

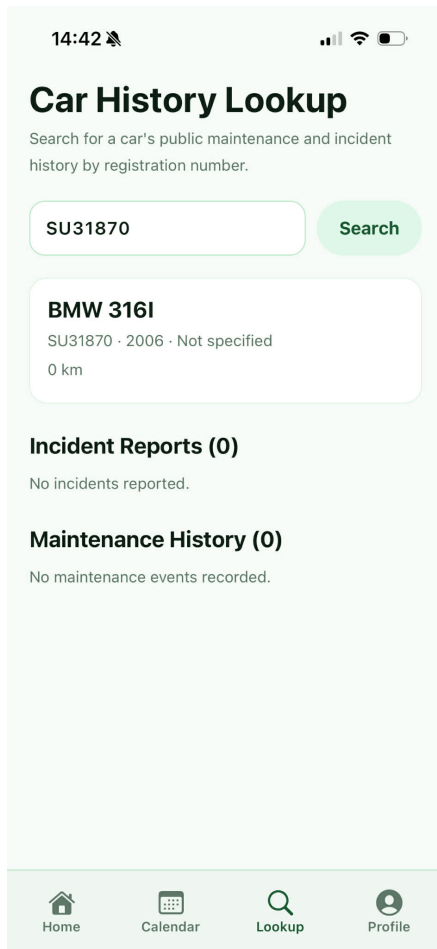
Continue with Google

Already have an account? Log in

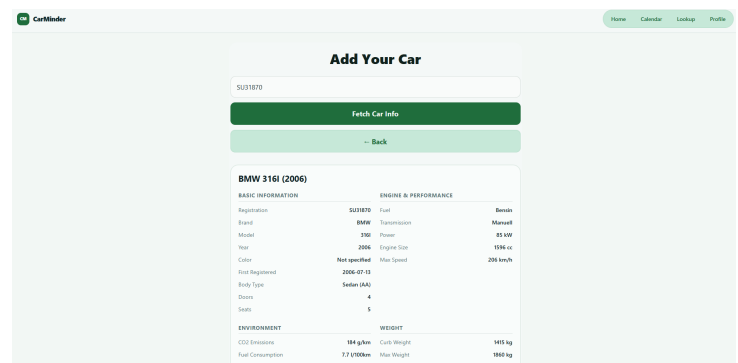
(b) Web registration screen for creating a new account with email and password.

Figure B.6: Registration screens on the mobile (left) and web (right) clients.

B.5 Car Management

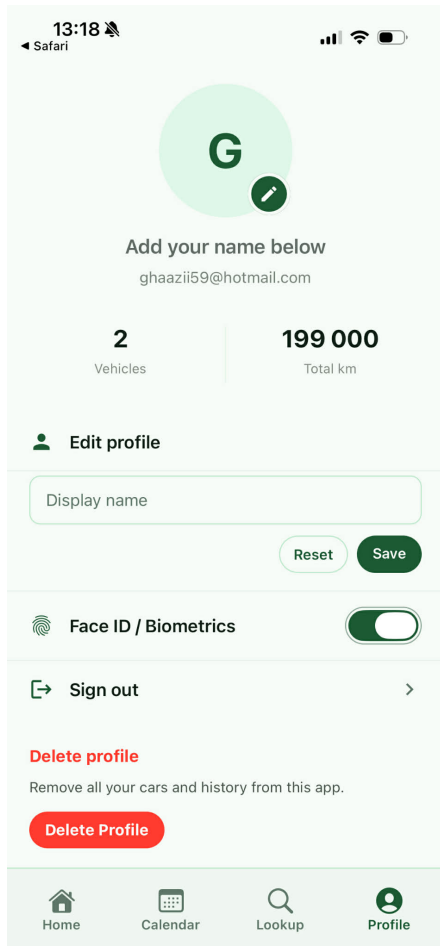


(a) Mobile add car screen showing vehicle data retrieved from the external vehicle registry by registration number.

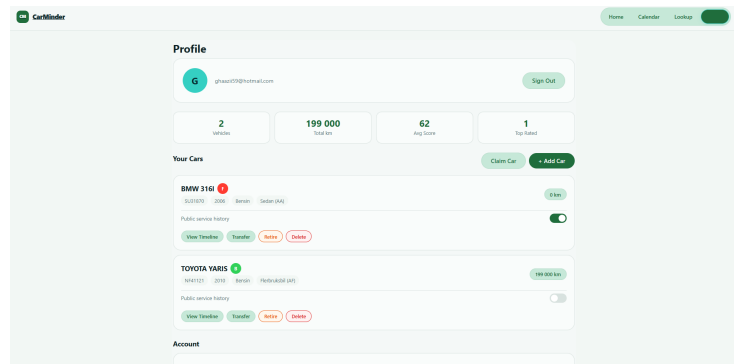


(b) Web add car screen showing the same vehicle data retrieved from the vehicle registry.

Figure B.7: Add car flow on the mobile (left) and web (right) clients, both consuming the same backend vehicle lookup endpoint.



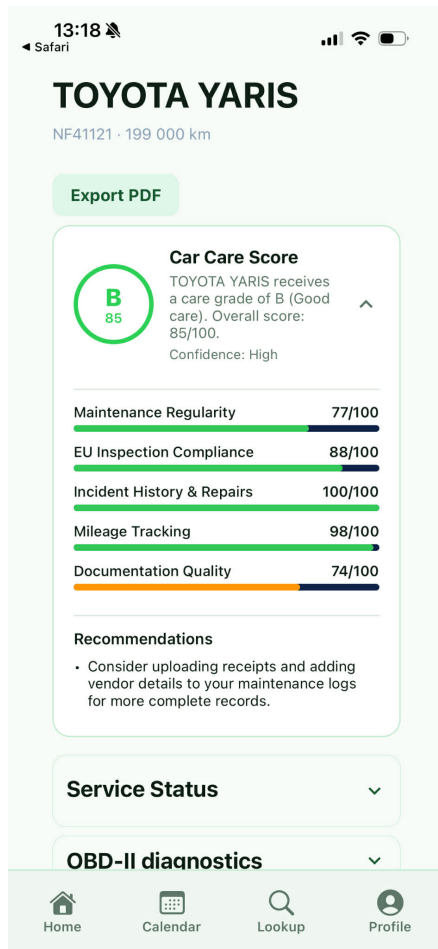
(a) Mobile profile tab showing registered vehicles and the public history toggle.



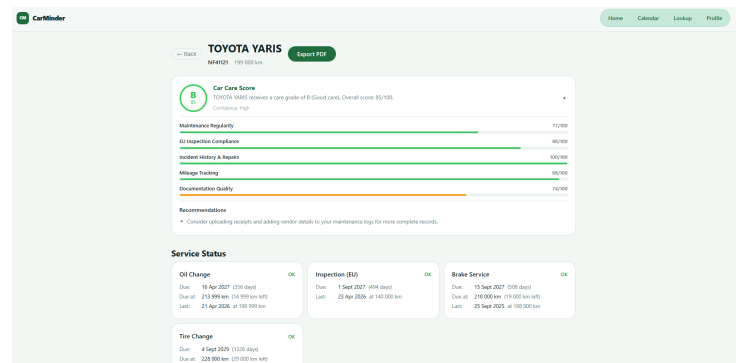
(b) Web profile page showing registered vehicles with transfer, retire, and delete options.

Figure B.8: Vehicle list on the mobile profile tab (left) and web profile page (right).

B.6 Car Care Score



(a) Mobile Car Care Score screen showing overall grade, category breakdown with colour-coded progress bars, and backend-generated recommendations.



(b) Web car detail view showing the Car Care Score, category breakdown, and service status cards with exact due dates and mileage thresholds.

Figure B.9: Car Care Score on the mobile (left) and web (right) clients. Both render the same backend response without client-side computation.

B.7 Maintenance Events

13:19
Safari

Add maintenance event

Event type *

Oil change

Date *

2026-04-25

Scan receipt (OCR)

Mileage (optional)

km

Cost (optional)

e.g. 1299.00

Vendor (optional)

Workshop or shop name

Notes (optional)

Details

Home Calendar Lookup Profile

(a) Mobile add maintenance event form showing event type, date, time, and optional metadata fields.

CarMindr Home Calendar Lookup Profile

Add Maintenance Event

Event Type
Oil Change Brake Service Tire Change Inspection (21.5) Repair
Other

Date
21.04.2026

Mileage (km)
Optional

Cost (€)
Optional

Vendor / Workshop
Optional

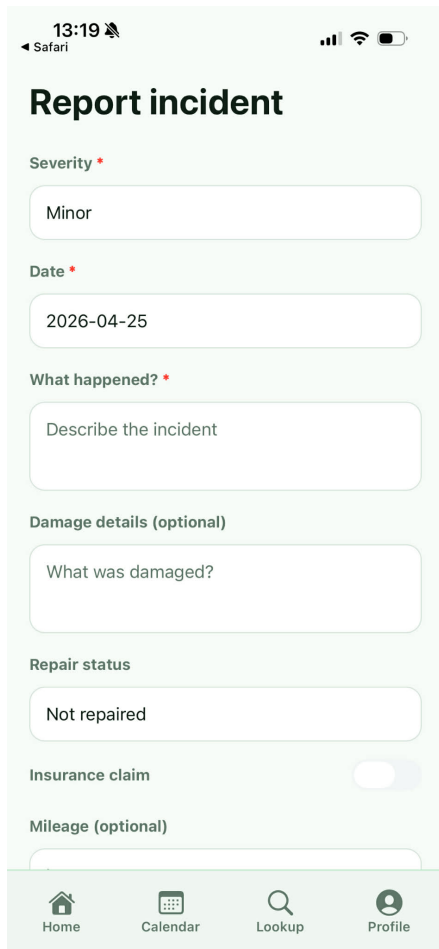
Notes
Optional

Save Event

(b) Web add maintenance event form showing the equivalent fields.

Figure B.10: Add maintenance event form on the mobile (left) and web (right) clients, both submitting to the same backend endpoint.

B.8 Incident Reports

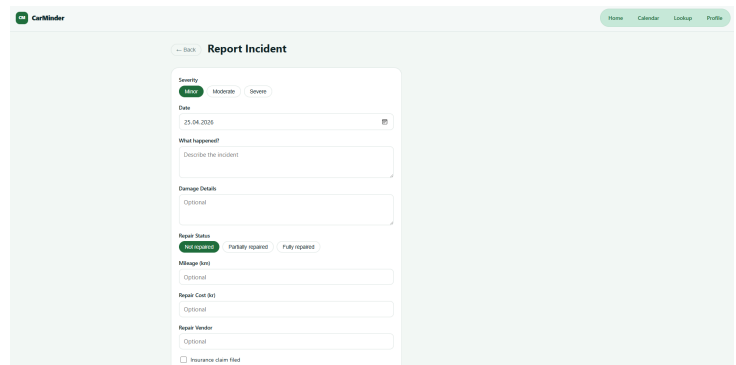


A mobile application interface for reporting an incident. The screen is titled "Report incident" and features a light green background. The form includes the following fields and options:

- Severity ***: A text input field containing "Minor".
- Date ***: A text input field containing "2026-04-25".
- What happened? ***: A text input field with the placeholder "Describe the incident".
- Damage details (optional)**: A text input field with the placeholder "What was damaged?".
- Repair status**: A text input field containing "Not repaired".
- Insurance claim**: A toggle switch currently turned off.
- Mileage (optional)**: A text input field.

At the bottom, there is a navigation bar with four icons: Home, Calendar, Lookup, and Profile.

(a) Mobile report incident form capturing severity, date, damage description, repair status, and insurance claim fields.



A web application interface for reporting an incident. The screen is titled "Report Incident" and features a light green background. The form includes the following fields and options:

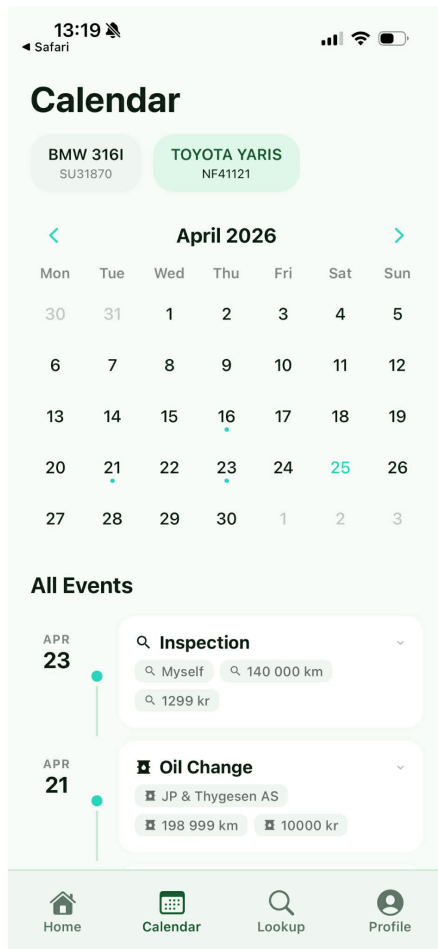
- Severity**: A dropdown menu with options "Minor", "Moderate", and "Severe".
- Date**: A text input field containing "25-04-2026".
- What happened?**: A text input field with the placeholder "Describe the incident".
- Damage Details**: A text input field with the placeholder "Optional".
- Repair Status**: A dropdown menu with options "Not repaired", "Partially repaired", and "Fully repaired".
- Mileage (km)**: A text input field with the placeholder "Optional".
- Repair Cost (\$)**: A text input field with the placeholder "Optional".
- Repair Vendor**: A text input field with the placeholder "Optional".
- Insurance claim filed**: A checkbox.

At the top right, there is a navigation bar with four links: Home, Calendar, Lookup, and Profile.

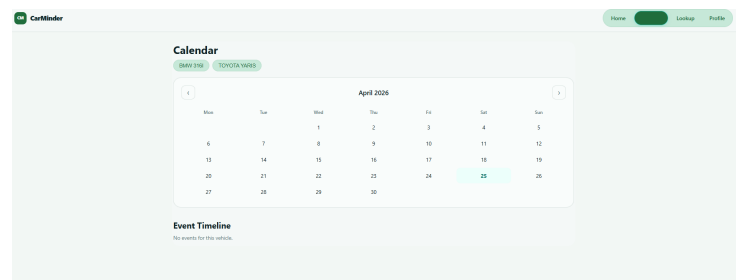
(b) Web report incident form showing the equivalent fields with the same severity and repair status options.

Figure B.11: Report incident form on the mobile (left) and web (right) clients.

B.9 Calendar



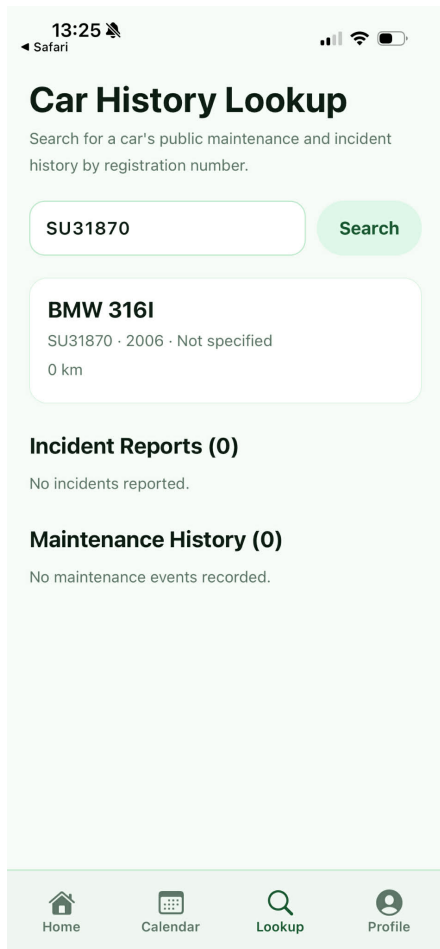
(a) Mobile calendar tab showing the monthly grid view and chronological event timeline for the selected vehicle.



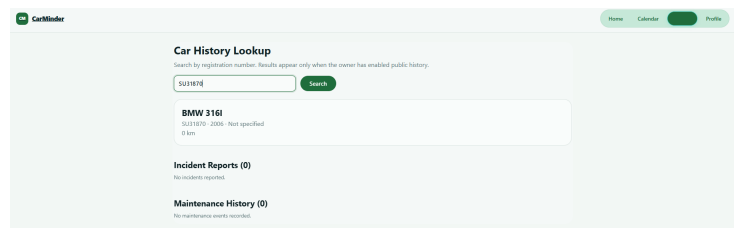
(b) Web calendar view showing the monthly grid and event timeline.

Figure B.12: Calendar view on the mobile (left) and web (right) clients.

B.10 Public History Lookup



(a) Mobile public history lookup showing vehicle details, incident report count, and maintenance history for a searched registration number.



(b) Web public history lookup showing the equivalent result for the same registration number.

Figure B.13: Public history lookup on the mobile (left) and web (right) clients, both consuming the same unauthenticated backend endpoint.

B.11 Web Fleet Dashboard

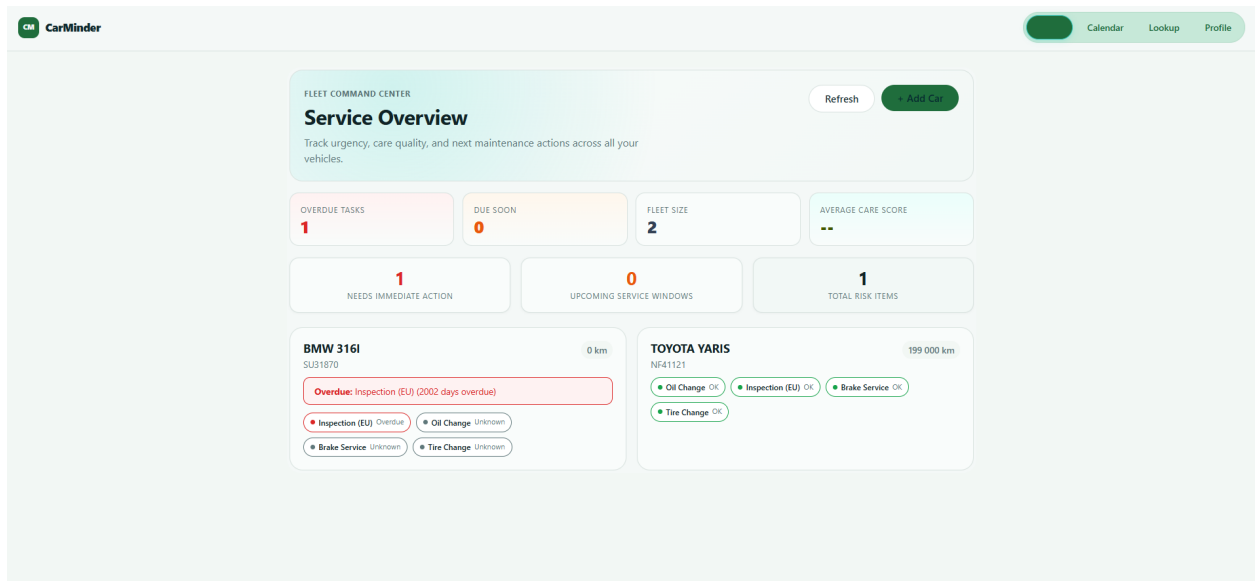
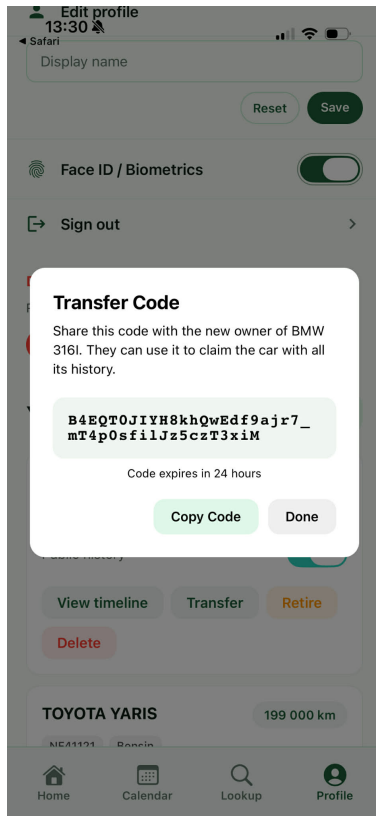
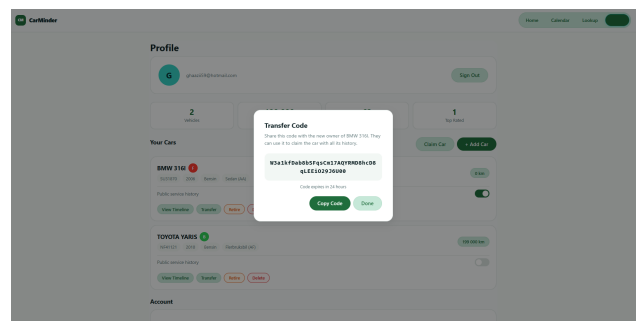


Figure B.14: Web home dashboard showing fleet-level service overview, summary cards, and per-vehicle care score and urgency status.

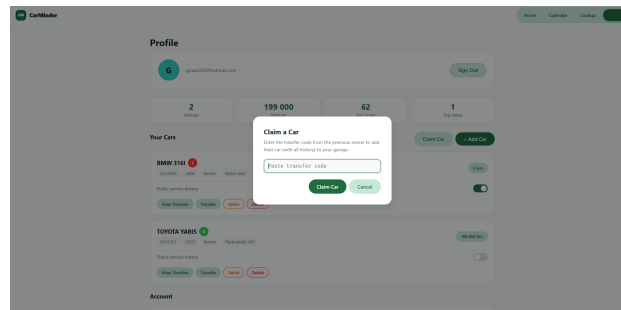
B.12 Vehicle Ownership Transfer Interface



(a) Mobile transfer code generation dialog showing the cryptographic code and 24-hour expiry.



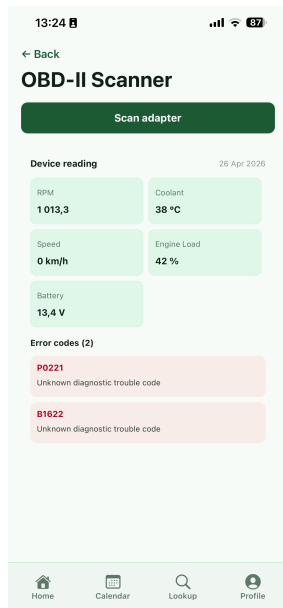
(b) Web transfer code generation dialog allowing the current owner to produce a code for handover.



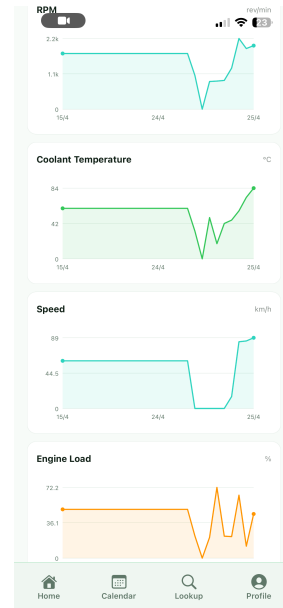
(c) Web claim car dialog allowing a new owner to enter a transfer code and receive the vehicle with its full maintenance history.

Figure B.15: Vehicle ownership transfer interface on mobile and web. Left: mobile transfer code generation. Centre: web transfer code generation. Right: web claim car dialog for the recipient. The underlying flow is illustrated in Figure 4.6 in Chapter 4.

B.13 OBD-II Real-Device Scanning

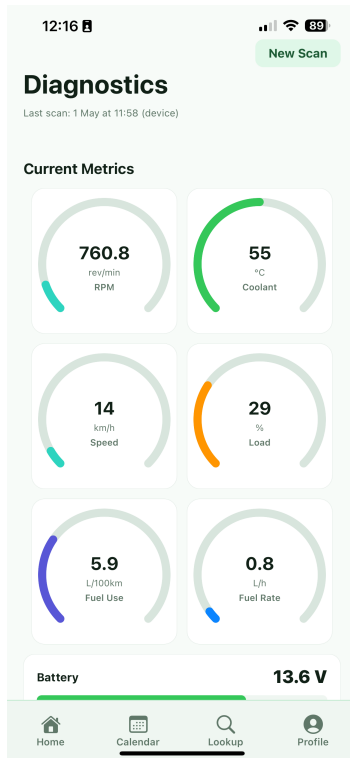


(a) Toyota Yaris scan with two unrecognised DTCs.

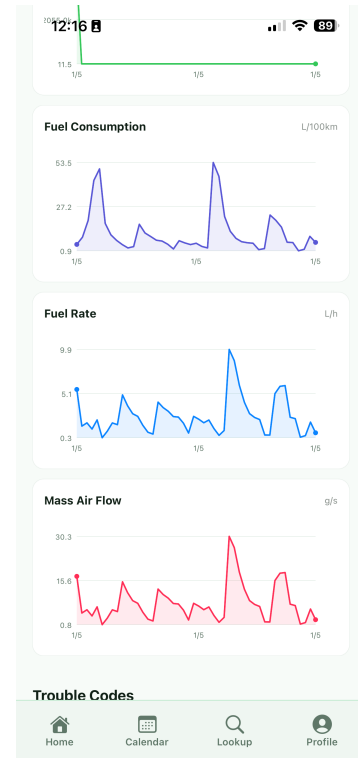


(b) Trend charts derived from persisted backend readings on the Mazda 3.

Figure B.16: Real-device OBD-II scanning on two vehicles. Left: Toyota Yaris scan showing RPM, coolant, speed, engine load, battery voltage, and two unrecognised DTCs. Right: metric trend charts from the Mazda 3 across the test period.



(a) Diagnostics view showing RPM, coolant, speed, engine load, fuel consumption (L/100km), fuel rate (L/h), and battery voltage as live gauges.



(b) Historical trend charts for fuel consumption, fuel rate, and mass air flow derived from persisted backend readings.

Figure B.17: Fuel-related OBD-II metrics on the mobile diagnostics screen. Left: live gauges including fuel consumption and fuel rate. Right: historical charts for fuel consumption, fuel rate, and mass air flow.

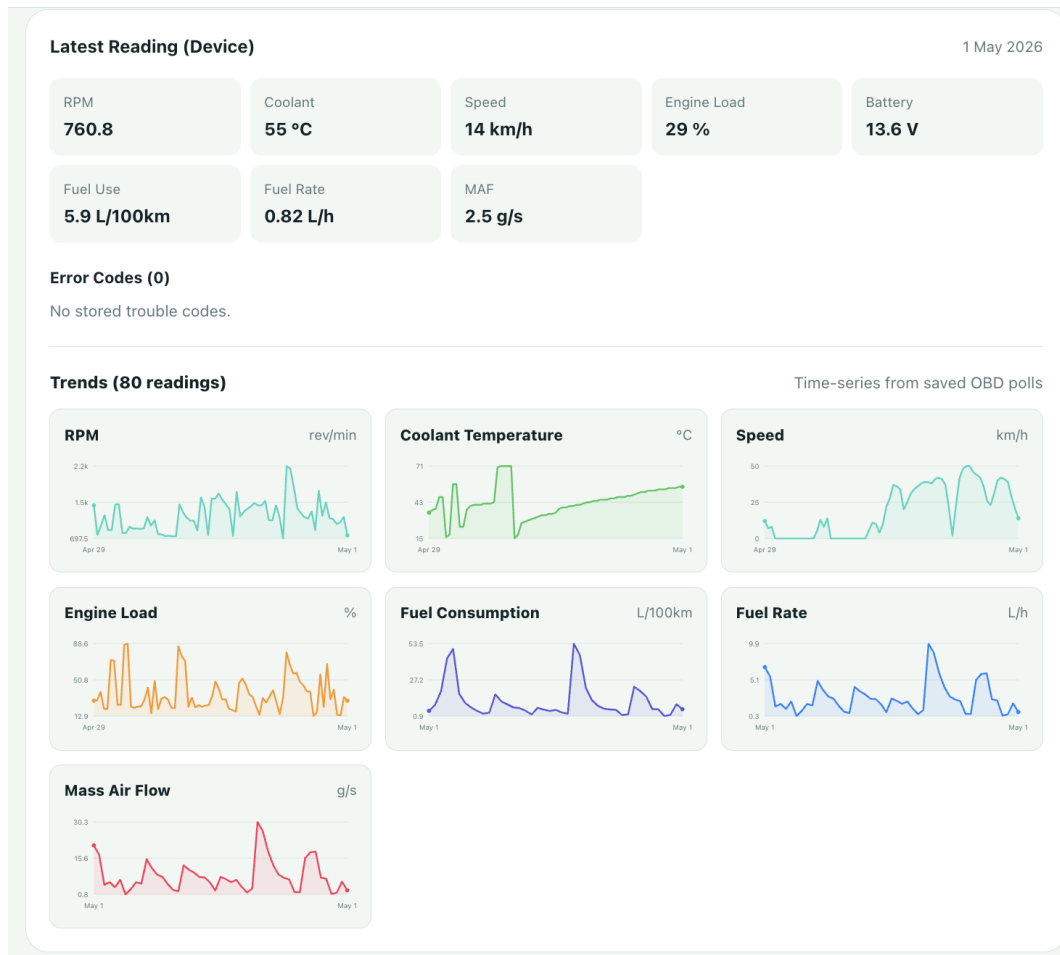


Figure B.18: OBD-II diagnostics view on the web client, showing live metrics including fuel consumption, fuel rate, and mass air flow, with corresponding trend charts for the Mazda 3 across 80 stored readings.

B.14 OCR Receipt Scanner

10:18

◀ TestFlight

📶 🔋

Add maintenance event

Event type *

Oil change

Date *

2026-04-29

Scan receipt (OCR)

Mileage (optional)

189000

Cost (optional)

9662

Vendor (optional)

Sørland Auto AS

Notes (optional)

Details

Home

Calendar

Lookup

Profile

(a) OCR action available in the Add maintenance event form.



(b) Receipt capture via the device camera before backend OCR processing.

Figure B.19: OCR receipt scanning on the mobile client. The captured image is uploaded to the backend, which performs Tesseract-based extraction and returns candidate fields for the user to review before submission.

Appendix C

User Stories

This appendix reproduces the full set of user stories formulated during requirements elicitation. The stories are summarised by category in Section 3.6; the complete list is given here for reference.

Core stories (implemented)

- As a car owner, I want to register a vehicle in the system so that I can collect vehicle information and maintenance history in one place.
- As a car owner, I want to look up a vehicle by its registration number so that technical data is auto-filled from the national vehicle registry instead of being entered manually.
- As a car owner, I want to record maintenance events such as services, repairs, or tyre changes so that I maintain a complete historical record of what has been done to the vehicle.
- As a car owner, I want to register incidents such as warning lights, unexpected behaviour, or minor accidents so that I can track issues and follow them up over time.
- As a car owner, I want to update my vehicle's current mileage so that service-interval predictions and the care score stay accurate over time.
- As a car owner, I want to see an overall vehicle care score so that I can quickly understand how well the vehicle is maintained and what areas may require attention.
- As a logged-in user, I want to access a dashboard on both web and mobile that shows my vehicles, upcoming maintenance tasks, and recent events so that I can quickly obtain an overview of my vehicles.
- As a car owner, I want to view maintenance events and upcoming deadlines on a calendar so that I can plan workshop visits in context with the rest of my schedule.
- As a car owner, I want to define service intervals and receive reminders before maintenance tasks become due so that important work is not forgotten.
- As a car owner, I want to export a PDF report of my vehicle's history so that I can share documentation with workshops, insurers, or prospective buyers.
- As a car owner, I want to control whether my vehicle's history is publicly visible so that I can decide when to share it, for example when preparing to sell the vehicle.
- As a prospective buyer, I want to look up a vehicle by its registration number and view its maintenance and incident history so that I can make a more informed decision before a purchase.

- As a car owner selling a vehicle, I want to transfer it, together with its full maintenance and incident history, to another user via a one-time transfer code so that the history follows the car rather than the previous owner.
- As a car owner, I want to retire a vehicle that is damaged beyond repair so that its chassis number is permanently blocked from being re-registered in the system.
- As a user, I want to edit my profile and, if I wish, permanently delete my account and associated data so that I retain control over my own information.

Secondary stories (implemented)

- As a user, I want to sign in with email and password, Google, or Apple, and reset my password by email if needed, so that I can access the system through the mechanism that best suits me.
- As a returning mobile user, I want to unlock the app using biometrics so that I can access my vehicles quickly without re-entering credentials.

Exploratory extension stories (implemented, but not fully validated)

- As a car owner, I want to connect an OBD-II adapter and upload diagnostic information so that the system can support a richer view of the vehicle's technical condition.
- As a car owner, I want to attach receipts or similar evidence to maintenance records so that documentation can be preserved together with the event itself.

Appendix D

Generated API Documentation

D.1 Generated API Documentation

Figure D.1 shows the generated OpenAPI/Swagger documentation for the CarMinder backend. The documentation is generated by FastAPI and provides an OpenAPI-described REST contract for the implemented backend endpoints, schemas, and authentication requirements. The same documentation was also exposed during development through the backend's Swagger UI endpoint.¹

¹Available at: <https://carminder-production.up.railway.app/docs>. Accessed: 1 May 2026.

CarMinder API

0.1.0 OAS 3.1

[/openapi.json](#)**Authorize** 

auth



POST /auth/login Login



POST /auth/signup Signup



POST /auth/google Google



POST /auth/apple Apple



POST /auth/forgot-password Forgot Password



cars



POST /cars/lookup Lookup Vehicle Info



GET /cars/ Get User Cars



POST /cars/ Create Car



GET /cars/{car_id} Get Car By Id



PATCH /cars/{car_id} Update Car



DELETE /cars/{car_id} Delete Car



Figure D.1: Generated OpenAPI/Swagger documentation for the CarMinder backend API, showing grouped endpoints and authentication-protected routes.

Appendix E

Self Assessments

E.1 Evan Self assessment and Timesheet

This section summarizes my individual contribution to the CarMinder bachelor project. My work covered the initial idea development, technical architecture, backend implementation, database design, external API integration, OBD-II functionality, scoring logic, validation strategy, automated testing, mutation testing, debugging, and major parts of the written report. My Clockify time report documents 562 hours and 32 minutes of registered work across these areas.

Project Idea, Research, and Direction

I came up with the initial project idea and discussed it with a university teacher before the project direction was refined further. I carried out early research into existing vehicle maintenance solutions, market relevance, possible technical approaches, suitable frameworks, and technology choices. This work helped define the product vision, the project scope, and the architectural focus of the thesis.

Architecture and Backend Design

I was responsible for the initial backend architecture and later modular redesign. This included designing the backend-centric structure, defining the API contracts, setting up the FastAPI backend, and organizing the backend into routers, schemas, services, repositories, and domain logic. I also worked on ensuring that the implementation followed the architectural goal of keeping business and domain logic centralized in the backend.

Database and External Integration

I set up the Supabase database structure, including the main database tables and relationships used by the system. I also implemented the integration with the external Vegvesen API so that vehicle information could be retrieved from an authoritative source and connected to the CarMinder backend and user interface.

Core System Functionality

I implemented several of the system's main features, including vehicle management, car persistence and retrieval, duplicate VIN handling, car retirement, ownership transfer, maintenance events, incident reporting, public vehicle history, service interval logic, calendar-related maintenance views, and backend/client error handling. These features formed a large part of the functional prototype.

Car Care Scoring Logic

I implemented the car care scoring functionality. This included the backend scoring route, schema, service, calculation logic, domain separation, scoring documentation, and related tests. I also worked on restructuring the scoring logic so that it followed the intended backend module boundaries and supported the thesis claim that domain reasoning is centralized.

OBD-II Implementation and Real-Device Testing

I implemented and tested the OBD-II functionality. This included Bluetooth OBD support, the dedicated OBD scan screen, ELM327 and iCar Pro adapter handling, OBD service logic, diagnostic trouble code handling, mobile and web OBD views, backend upload of real readings, and expanded diagnostic reporting. I also tested the scanner on my own car, identified failures during real-device testing, debugged the issues, retested fixes, and improved the feature based on the results.

Web and Mobile Client Work

I contributed to both the web and mobile clients. My work included profile screens, home screens, car-management views, add-car functionality, add-event functionality, calendar views, car-detail pages, incident/reporting screens, layout fixes, and general UI polishing. I also worked on connecting the clients to backend responses so that the clients acted mainly as presentation layers.

Validation, Testing, Mutation Testing, and Debugging

I designed and implemented much of the validation strategy used in the thesis. This included researching suitable testing methods, setting up GitHub Actions CI, using Import Linter for architecture checks, adding architecture tests, adding scoring tests, running Schemathesis contract testing, and using Mutmut mutation testing to evaluate the strength of the scoring-domain test suite. I also worked on interpreting mutation-testing results, identifying survived mutants, documenting the limitations they revealed, fixing bugs found by test results, and documenting the validation evidence. This work was important for verifying that the thesis claims were supported by implementation and test results rather than only architectural intention.

Report Writing and Academic Contribution

I wrote and revised major parts of the report, including the introduction, problem statement, related work, method chapter, results chapter, validation sections, discussion of results, conclusion, OBD/testing sections, citations, transitions, figures, screenshots, and appendices. I also worked on connecting the research question, architecture, implementation, validation results, and conclusion into a coherent academic argument.

Summary

Overall, my contribution covered the initial idea development, research, architecture, back-end implementation, database setup, Vegvesen integration, API contracts, scoring logic, OBD-II functionality, real-device testing, debugging, web/mobile UI implementation, validation strategy, automated testing, mutation testing with Mutmut, claim verification, and major thesis chapters.

Detailed report

01/01/2026 - 31/12/2026



Total: 562:32:00 Billable: 00:00:00 Amount: 0,00 USD

Date	Description	Duration	User
11/05/2026	going through the report fixing report issue citation errors and grammer mistake Carminder	04:30:00 16:00:00 - 20:30:00	Ivanbital156
10/05/2026	running mutmut testing targeting the scoring engine code and test and interperating the test results and document it in the report Carminder	04:30:00 11:30:00 - 16:00:00	Ivanbital156
09/05/2026	writing about the Schemathesis in the report and fixing Carminder	06:00:00 10:00:00 - 16:00:00	Ivanbital156
08/05/2026	pushed more fixes after the result of Schemathesis testing Carminder	01:00:00 20:00:00 - 21:00:00	Ivanbital156
08/05/2026	found Schemathesis and mutmut for testing running the Schemathesis test and got bugs and failure test Carminder	02:08:00 18:00:00 - 20:08:00	Ivanbital156
08/05/2026	after meeting reseaching external testing for the backend to add credibility Carminder	05:12:00 13:00:00 - 18:12:00	Ivanbital156
07/05/2026	working on report fixing some issuse with the transation from intro to method to result Carminder	04:06:00 10:00:00 - 14:06:00	Ivanbital156
06/05/2026	shortening and fixing some phrase of all section to make it more clear Carminder	04:06:00 12:00:00 - 16:06:00	Ivanbital156
05/05/2026	worked on report removing screenshot and rewriting each section and putting them in appendix Carminder	07:56:00 12:04:00 - 20:00:00	Ivanbital156
04/05/2026	report working on adding test pictures results obd section and fixing some citation Carminder	05:00:00 12:00:00 - 17:00:00	Ivanbital156
03/05/2026	writing the report rephrasing some section and validation Carminder	04:01:00 12:00:00 - 16:01:00	Ivanbital156
02/05/2026	working on report reuslts validation sections Carminder	05:00:00 12:00:00 - 17:00:00	Ivanbital156
01/05/2026	writing report focus on the results chapter client application Carminder	04:00:00 12:00:00 - 16:00:00	Ivanbital156

30/04/2026	added more complete OBD reporting across mobile, backend, and web. The work expanded the car detail and OBD dashboard views, updated OBD reading schemas and services, added fuel consumption metrics, and improved diagnostic trouble code handling and explanations for Toyota Yaris data. It also updated shared frontend API and OBD service types so the web and mobile clients could consume the new data consistently. Carminder	09:00:00 10:00:00 - 19:00:00	Ivanbiral156
29/04/2026	Added more complete OBD reporting across mobile, backend, and web. The work expanded the car detail and OBD dashboard views, updated OBD reading schemas and services, added fuel consumption metrics, and improved diagnostic trouble code handling and explanations for Toyota Yaris data. It also updated shared frontend API and OBD service types so the web and mobile clients could consume the new data consistently. Carminder	08:00:00 10:00:00 - 18:00:00	Ivanbiral156
28/04/2026	Fixed duplicated or overlapping scoring logic and updated tests around the scoring modules. Carminder	05:13:00 09:15:00 - 14:28:00	Ivanbiral156
27/04/2026	Upgraded the OBD-related user interface on mobile and web. This included changes to the mobile OBD scan screen, BLE/OBD service behavior, global web styling, and the web car detail page so OBD information looked and behaved more consistentl Carminder	05:13:00 14:28:00 - 19:41:00	Ivanbiral156
26/04/2026	working on report focus on discussion and method Carminder	04:00:00 16:00:00 - 20:00:00	Ivanbiral156
25/04/2026	writing repory focus on method Carminder	04:00:00 12:00:00 - 16:00:00	Ivanbiral156
24/04/2026	Description: Improved the Bluetooth OBD scanning flow by fixing adapter disconnection behavior and how nearby Bluetooth devices are listed during scan attempts. Carminder	06:00:00 08:00:00 - 14:00:00	Ivanbiral156
20/04/2026	Finalized a larger OBD fix. The mobile OBD scan screen was changed so real device readings upload to the backend while demo scans remain local. The backend OBD service was updated to normalize diagnostic trouble codes consistently. The BLE transport was hardened with custom Base64 ASCII encoding/decoding, better adapter detection, safer connection handling, stricter characteristic selection, and cleaner ELM327 command responses. The OBD service stopped silently falling back to simulated readings after native transport failures, making real scan errors more visible. Carminder	10:00:00 10:00:00 - 20:00:00	Ivanbiral156
18/04/2026	worked on the report Carminder	04:00:00 10:00:00 - 14:00:00	Ivanbiral156
17/04/2026	working on report reading and writing down things to fix like diagram picture palcement Carminder	07:00:00 10:00:00 - 17:00:00	Ivanbiral156
15/04/2026	wokring on report Carminder	04:14:00 12:00:00 - 16:14:00	Ivanbiral156
08/04/2026	Upgraded the OBD-related user interface on mobile and web. This included changes to the mobile OBD scan screen, BLE/OBD service behavior, global web styling, and the web car detail page so OBD information looked and behaved more consistently. Carminder	06:00:00 10:00:00 - 16:00:00	Ivanbiral156

07/04/2026	writing report intro problem statment related work Carminder	08:00:00 10:00:00 - 18:00:00	Ivanbila156
06/04/2026	testing obd and debugging the code for bugs Carminder	07:00:00 10:00:00 - 17:00:00	Ivanbila156
05/04/2026	Updated the backend Import Linter configuration. This adjusted the architecture rules used to check module boundaries and keep the backend code organized according to the intended layering. Carminder	02:00:00 13:00:00 - 15:00:00	Ivanbila156
04/04/2026	updates to the GitHub Actions CI file plus backend test setup and API/client conformance test files. Carminder	02:00:00 11:00:00 - 13:00:00	Ivanbila156
03/04/2026	Corrected the displayed unit for coolant temperature in the web car detail view. This was a small UI/data presentation fix to make OBD temperature readings clearer and correctly labeled. Carminder	01:00:00 10:00:00 - 11:00:00	Ivanbila156
02/04/2026	Improved the Bluetooth OBD scanning flow by fixing adapter disconnection behavior and how nearby Bluetooth devices are listed during scan attempts. The changes affected the mobile OBD scan screen, app layout, BLE transport layer, and OBD service code. Carminder	06:00:00 16:00:00 - 22:00:00	Ivanbila156
01/04/2026	Upgraded the OBD-related user interface on mobile and web. This included changes to the mobile OBD scan screen, BLE/OBD service behavior, global web styling, and the web car detail page so OBD information looked and behaved more consistently. Carminder	06:00:00 10:00:00 - 16:00:00	Ivanbila156
30/03/2026	Fixed duplicated or overlapping scoring logic and updated tests around the scoring modules. The changes touched the scoring engine, explanation, normalization, and service layers, plus architecture/modularity tests to protect the intended backend structure. Carminder	08:00:00 10:00:00 - 18:00:00	Ivanbila156
29/03/2026	Finalized a larger OBD fix. The mobile OBD scan screen was changed so real device readings upload to the backend while demo scans remain local. The backend OBD service was updated to normalize diagnostic trouble codes consistently. The BLE transport was hardened with custom Base64 ASCII encoding/decoding, better adapter detection, safer connection handling, stricter characteristic selection, and cleaner ELM327 command responses. The OBD service stopped silently falling back to simulated readings after native transport failures, making real scan errors more visible. Carminder	10:00:00 10:00:00 - 20:00:00	Ivanbila156
28/03/2026	Fixed OBD scanner service logic in the shared frontend service layer. This was focused on improving the scan behavior used by the mobile OBD feature. Carminder	10:00:00 10:00:00 - 20:00:00	Ivanbila156
26/03/2026	Updated the web TypeScript configuration so the web project no longer depends on Expo-specific TypeScript settings. This made the web app configuration more independent and better suited for the Vite/React web client. Carminder	06:00:00 10:00:00 - 16:00:00	Ivanbila156
25/03/2026	Added GitHub Actions CI so project tests could run automatically. This was part of setting up a repeatable testing pipeline for the repository. Carminder	04:00:00 10:00:00 - 14:00:00	Ivanbila156

24/03/2026	Fixed ELM327 adapter connection support. The work touched mobile navigation, the car detail and OBD dashboard screens, shared API calls, BLE transport, and package configuration to make mobile OBD adapter connection more reliable. Carminder	09:45:00 10:15:00 - 20:00:00	Ivanbila156
23/03/2026	Added accident reporting support to the web client. This included a new add-incident page, route updates, API changes, car detail integration, lookup/profile-related UI updates, styling, and shared car type updates so incidents could be created and displayed from the web app. Carminder	08:00:00 10:00:00 - 18:00:00	Ivanbila156
22/03/2026	added backend diagnostics for car ownership mismatches. The repository gained a function to fetch a car by ID without the user filter, and the service logs whether a missing car lookup is caused by the car not existing or by the car belonging to another Firebase user. Carminder	09:00:00 11:00:00 - 20:00:00	Ivanbila156
21/03/2026	Improved error handling when users try to add or save cars. Mobile and web now show more specific messages for retired vehicles and already-registered cars, and the web UI added contextual error banners with a transfer-code claim action for already registered vehicles. Carminder	05:00:00 11:00:00 - 16:00:00	Ivanbila156
20/03/2026	Added the ability to retire a car so it can no longer be registered again. This involved backend repository, router, schema, and service changes, plus mobile and web profile/API/type updates so users could mark vehicles as retired and the clients could understand the retired state. Carminder	10:45:00 11:15:00 - 22:00:00	Ivanbila156
19/03/2026	Fixed simulator-related issues by adjusting app configuration, notifications code, and package lock data. Carminder	08:00:00 10:00:00 - 18:00:00	Ivanbila156
18/03/2026	Implemented car ownership transfer. The backend gained transfer-related repository, router, schema, and service support, while mobile and web profile screens and API clients were updated so a car could be transferred between users through the application. Carminder	09:00:00 10:00:00 - 19:00:00	Ivanbila156
17/03/2026	testing the obd again after implementing a fix still failure Carminder	06:00:00 14:00:00 - 20:00:00	Ivanbila156
15/03/2026	Added backend and client-side handling to prevent the same VIN from being duplicated across user profiles. The API and frontend services were updated to detect an already registered vehicle and guide users toward the ownership transfer flow. Carminder	10:00:00 10:00:00 - 20:00:00	Ivanbila156
13/03/2026	testing the obd scanner with my car to debug it and noting the issues Carminder	02:00:00 18:00:00 - 20:00:00	Ivanbila156
12/03/2026	Added more backend architecture and import testing. This expanded Import Linter rules and added tests for architecture contracts, modularity, and import-linter integration. It also touched push-token routing/service code and the mobile home screen. Carminder	10:00:00 10:00:00 - 20:00:00	Ivanbila156
11/03/2026	Added more test coverage for the backend scoring engine. This strengthened validation around the scoring calculations and expected behavior. Carminder	00:00:00 20:00:00 - 20:00:00	Ivanbila156

10/03/2026	Added Import Linter architecture checking and reorganized scoring code to respect backend boundaries. The scoring engine, explanation, normalization, and service code were updated, with tests added or adjusted around the scoring behavior. Carminder	10:05:00 09:55:00 - 20:00:00	Ivanbital156
03/03/2026	Fixed authentication flows for sign in and sign up across mobile and web. The changes touched Expo/app config, auth layouts, forgot-password/login/signup screens, package config, and web login/signup pages, including work that appears intended to help TestFlight behavior. Carminder	11:00:00 09:00:00 - 20:00:00	Ivanbital156
02/03/2026	Added a dedicated OBD scan screen in the mobile app. Navigation and car detail routing were updated so OBD scanning had its own screen instead of being only part of the car detail view. Carminder	07:45:00 12:45:00 - 20:30:00	Ivanbital156
01/03/2026	Adjusted the BLE OBD transport configuration for the iCar Pro adapter. This focused on adapter compatibility at the Bluetooth transport layer. Carminder	10:00:00 10:00:00 - 20:00:00	Ivanbital156
28/02/2026	Split vehicle scoring logic into a clearer backend domain structure and added tests. Scoring engine and normalization code moved into domain modules, while schemas and service/explanation layers were updated to use the separated logic. Carminder	10:00:00 10:00:00 - 20:00:00	Ivanbital156
28/02/2026	Added native Bluetooth connection support for reading data from an OBD scanner. This introduced BLE transport and OBD service support, updated mobile app configuration and layout, connected the feature to the car detail screen, and adjusted package and web TypeScript configuration. Carminder	11:45:00 09:00:00 - 20:45:00	Ivanbital156
27/02/2026	Polished UI across a large part of the mobile and web apps. The changes touched authentication screens, add-car flow, calendar, car detail, incident/event screens, lookup, profile, theme constants, auth context, system overview docs, web auth context, web styling, and several web pages. Carminder	10:45:00 11:15:00 - 22:00:00	Ivanbital156
26/02/2026	Added the first version of a car scoring feature. The backend gained a car score route, schema, and scoring service, while mobile car detail and frontend API calls were updated to request and display score information. Carminder	10:00:00 10:00:00 - 20:00:00	Ivanbital156
25/02/2026	Updated the web UI and related mobile car/home views. Carminder	06:00:00 10:00:00 - 16:00:00	Ivanbital156
24/02/2026	Added backend documentation explaining the vehicle scoring system. This created a scoring reference document for how the car score is calculated and interpreted. Carminder	03:00:00 20:00:00 - 23:00:00	Ivanbital156
23/02/2026	Improved error handling around maintenance events and car detail flows. Mobile calendar/car detail behavior, backend maintenance event routing, and shared frontend API calls were updated to handle failures more cleanly Carminder	03:15:00 20:00:00 - 23:15:00	Ivanbital156
21/02/2026	Created the first draft of the web app shell. This added protected routing, a welcome page, router setup, and initial web styling. Carminder	01:15:00 19:15:00 - 20:30:00	Ivanbital156

18/02/2026	Refactored frontend API calls into separate service modules. The shared API layer was split into auth, car, incident, maintenance, notifications, public, service status, config, and type files to make debugging and future feature work easier. Carminder	09:00:00 10:00:00 - 19:00:00	Ivanbital156
17/02/2026	Fixed dimensions and layout behavior across mobile screens. Carminder	06:00:00 10:00:00 - 16:00:00	Ivanbital156
15/02/2026	Description: Implemented web login support, including Google sign-in UI/API code and web authentication context updates. Login and signup pages were updated to work with the new web auth flow. Carminder	01:15:00 20:30:00 - 21:45:00	Ivanbital156
13/02/2026	Implemented public vehicle history lookup and accident/incident reporting. The mobile app gained lookup and add-incident flows, backend public history and incident reporting repositories/routers/schemas/services were added, and shared API calls were updated to support the new feature Carminder	08:00:00 11:00:00 - 19:00:00	Ivanbital156
12/02/2026	Added a web-specific .gitignore file so generated web files and local web artifacts would not be committed accidentally. Carminder	01:15:00 18:00:00 - 19:15:00	Ivanbital156
12/02/2026	Added major web car-management functionality. The web client gained API calls, layout, add-car, add-event, calendar, car-detail, home, login, profile, signup, routing, car types, and formatting utilities. Carminder	05:15:00 12:45:00 - 18:00:00	Ivanbital156
11/02/2026	Fixed the mobile maintenance event creation flow and persistence to the backend. The add-event screen, car detail screen, and shared API calls were updated so maintenance events could be saved and then reflected in the car view. Carminder	04:00:00 14:00:00 - 18:00:00	Ivanbital156
10/02/2026	Implemented a calendar view for repair and maintenance types in the mobile app. The work updated mobile navigation, calendar and car detail/home screens, icon support, notifications, and package configuration. Carminder	06:00:00 16:00:00 - 22:00:00	Ivanbital156
09/02/2026	Added service interval reminder support based on previous maintenance events. The backend gained service interval routing/schema/service support, and the mobile home/car detail screens plus shared API calls were updated to surface interval reminders. Carminder	04:00:00 18:00:00 - 22:00:00	Ivanbital156
08/02/2026	Merged backend structure from the remote main branch into the current work. This brought in or reconciled backend config, auth, database, exception, repository, router, and service modules for cars, maintenance, vehicle lookup, Firebase auth, and JWT auth. Carminder	04:00:00 10:00:00 - 14:00:00	Ivanbital156
04/02/2026	Added debugging support around backend authentication, database access, car routing, JWT auth, mobile add-event behavior, and frontend API calls. This was intended to make backend and client failures easier to diagnose Carminder	09:00:00 10:00:00 - 19:00:00	Ivanbital156
31/01/2026	Adjusted backend and frontend API configuration so the app could connect to the backend while running in a simulator. Carminder	07:00:00 11:00:00 - 18:00:00	Ivanbital156

29/01/2026	Added manual maintenance event creation. The mobile app gained navigation and a dedicated add-event screen, car detail/profile/add-car updates, and the backend gained maintenance event router/schema support plus API calls for saving those events. Carminder	05:00:00 15:00:00 - 20:00:00	Ivanbiral156
29/01/2026	Added a mobile profile section where users can manage cars linked to their account. This included profile navigation, add-car/profile screens, removal of the explore screen, and icon support updates. Carminder	04:00:00 11:00:00 - 15:00:00	Ivanbiral156
28/01/2026	Added the initial FastAPI backend and moved car lookup/save logic out of the mobile app into backend services. The work introduced backend config, database setup, main app, car router/schema/service structure, requirements, Supabase integration, and frontend API calls to communicate with the backend. Carminder	07:00:00 18:00:00 - 01:00:00 ⁺¹	Ivanbiral156
27/01/2026	recreating the design for the backend after reflecting with our supervisor and creating a modular design Carminder	06:00:00 18:00:00 - 00:00:00 ⁺¹	Ivanbiral156
23/01/2026	creating UI for screen for profile section and homepage on the app and reflecting on the meeting by our examiner Carminder	06:00:00 12:00:00 - 18:00:00	Ivanbiral156
21/01/2026	debugging the endpoint and debuging for rejecting https request Carminder	07:00:00 11:00:00 - 18:00:00	Ivanbiral156
20/01/2026	Added saving of car data to the user database. The add-car screen, car service, and Supabase client were updated so vehicles looked up in the app could be persisted for the current use Carminder	07:00:00 11:00:00 - 18:00:00	Ivanbiral156
18/01/2026	implementing the backend and translating the design into actual code Carminder	07:00:00 12:00:00 - 19:00:00	Ivanbiral156
16/01/2026	creating architectual design for the backend and api contracts Carminder	06:15:00 12:45:00 - 19:00:00	Ivanbiral156
14/01/2026	testing api endpoint through openfastapi and developing more endpoints for car retrieval Carminder	08:03:00 10:00:00 - 18:03:00	Ivanbiral156
13/01/2026	setting up fastapi backend and connecting to web and app Carminder	08:00:00 10:00:00 - 18:00:00	Ivanbiral156
12/01/2026	connecting to the external vegvsen api to get a car info and creating a UI and creating supabase database and connecting to the backend through api contract Carminder	09:00:00 10:00:00 - 19:00:00	Ivanbiral156
11/01/2026	setting about project enviroment for the project in github and expo go Carminder	06:00:00 16:00:00 - 22:00:00	Ivanbiral156
10/01/2026	researching about the best framework and language to use for the project Carminder	06:00:00 10:00:00 - 16:00:00	Ivanbiral156

09/01/2026	researching about the market and different solution in the market Carminder	06:00:00 12:00:00 - 18:00:00	Ivanbital156
08/01/2026	talking with arne about the project and getting feedback on porject and rewriting the product vison Carminder	07:00:00 12:00:00 - 19:00:00	Ivanbital156
06/01/2026	write down product vision for validating Carminder	05:00:00 15:00:00 - 20:00:00	Ivanbital156
05/01/2026	research bachlor project idea and canvas already existing project Carminder	05:00:00 10:00:00 - 15:00:00	Ivanbital156

E.2 Ghazi Self assessment and Timesheet

This section summarizes my individual contribution to the CarMinder bachelor project. My work covered project setup, frontend implementation, mobile and web interface improvements, authentication setup with Firebase, database setup with Supabase, deployment setup with Railway, GitHub project setup, incident reporting functionality, OCR-related work, PDF export, notifications, testing, report writing, and general project refinement. My Clockify time report documents 143 hours and 32 minutes of registered work on the project.

Project Setup, Infrastructure, and Deployment

One of my main contributions was setting up the technical foundation for the project. I set up the project on GitHub and helped establish the shared repository structure used during development. This made it possible for the group to collaborate on the same codebase, track changes, and manage the implementation work throughout the project period.

I also set up Supabase for the project and configured it as the database platform used by CarMinder. This included preparing the database environment needed for storing core system data such as users, cars, maintenance events, incident reports, OBD readings, push tokens, and notification logs. In addition, I set up Railway for backend deployment, which allowed the FastAPI backend to be hosted as an accessible API for both the web and mobile clients.

Authentication and Firebase Setup

Another major technical contribution was setting up the authentication system using Firebase. I configured Firebase for the project and implemented the authentication flow used by the application. This included login and registration with email and password, as well as Google login. I also worked on connecting the frontend authentication flow to the backend so that authenticated users could access protected functionality through the application's API.

In addition to the basic authentication setup, I worked on forgot password/reset password functionality, Apple login, and FaceID login. These features improved the accessibility and usability of the application, especially on mobile. I also helped resolve issues related to TestFlight, which was necessary to test and demonstrate the mobile application in a more realistic environment.

Frontend and User Interface Work

A significant part of my contribution was related to the frontend of the system. I worked on improving the visual design and usability of both the web and mobile clients, including layout adjustments, light mode alignment, and general UI polishing. I also updated screenshots used in the report so that the documented system matched the final implemented version more closely. This work helped make the prototype more consistent across platforms and supported the thesis goal of having multiple thin clients consuming the same backend.

Incident Reporting and Calendar Functionality

I contributed to the incident reporting functionality by implementing support for adding pictures when reporting an incident. This made the incident feature more useful because users could attach visual evidence to vehicle-related damage or problems. I also worked on calendar date functionality, which supported the presentation of maintenance-related events and helped make the client applications more practical for users.

OCR, PDF Export, and Notifications

I contributed to the OCR scanning feature and documented parts of this functionality in the report. I also worked on the PDF export feature and helped connect it to the user stories

and system documentation. In addition, I worked on implementing notifications and setting up GitHub Actions, which supported both user-facing functionality and the development workflow.

Code Refactoring and Technical Improvements

I worked on cleaning up duplicate frontend code, especially by moving repeated API-related logic into a more centralized apiCall structure. This reduced duplication and made the client code easier to maintain. I also helped investigate and fix a duplicate scoring engine issue, which was important because the project depends on the claim that domain reasoning should remain centralized and consistent.

Testing and Real-Device Work

I contributed to testing activities, including extracting git log information and checking that implemented functionality behaved as expected. I also participated in OBD-related testing on a real vehicle together with another group member. This helped the group identify practical issues that only appear during real-device testing and supported the report's discussion of OBD-II as an exploratory prototype extension.

Report Writing and Documentation

I contributed to several parts of the written thesis. My work included writing and revising sections of the theoretical background, method, discussion, results, conclusion, acknowledgements, abstract, and limitations. I also re-read the report to look for contradictions, unclear wording, and mistakes. This work helped improve the consistency between the implemented system, the screenshots, the validation results, and the academic argument of the thesis.

Summary

Overall, my contribution focused on setting up the technical foundation of the project, including GitHub, Firebase, Supabase, and Railway. I also implemented email/password login, registration, Google login, and other authentication-related features, improved the frontend and user experience, implemented and refined user-facing functionality, worked on OCR, PDF export and notifications, contributed to testing and real-device work, and wrote and improved important parts of the report. My work helped connect the implemented prototype to the thesis documentation and supported the project goal of presenting CarMinder as a backend-centric system with thin web and mobile clients.

Detailed report

01/01/2026 - 31/12/2026



Total: 143:32:59 Billable: 00:00:00 Amount: 0,00 USD

Date	Description	Duration	User
13/05/2026	Making a video of the system Carminder	04:00:00 10:00:00 - 14:00:00	ghazifida59
12/05/2026	Implementing the feature of adding pictures of a incident when reporting it Carminder	08:00:00 09:00:00 - 17:00:00	ghazifida59
11/05/2026	Working on writing self assessment, extracting git log. And testing Carminder	08:00:00 09:00:00 - 17:00:00	ghazifida59
08/05/2026	Working on the report and self assessment Carminder	05:00:00 12:30:00 - 17:30:00	ghazifida59
08/05/2026	Meeting with Halvard Carminder	00:30:00 11:30:00 - 12:00:00	ghazifida59
07/05/2026	Working on the report Carminder	06:00:00 09:00:00 - 15:00:00	ghazifida59
04/05/2026	Working on the report Re-reading the whole report too see if everything is ok and there is no contradictions or mistakes Carminder	04:00:00 11:00:00 - 15:00:00	ghazifida59
30/04/2026	Working on the report Carminder	05:00:00 10:00:00 - 15:00:00	ghazifida59
29/04/2026	(Without Description) Carminder	03:19:12 18:59:43 - 22:18:55	ghazifida59
29/04/2026	Working on documenting the OCR-Scanning feature in the report. Also fixing small issues with UI in the web. Carminder	03:43:47 11:04:27 - 14:48:14	ghazifida59
26/04/2026	Working on fixing a duplicate scoring engine problem in the code. Also meeting with Ivan to test the OBD on my car. Carminder	03:00:00 11:00:00 - 14:00:00	ghazifida59
25/04/2026	Working on implementing pictures on Add Incident form. Also Calender date function. And updating the pictures on the report Carminder	03:00:00 13:00:00 - 16:00:00	ghazifida59
24/04/2026	Working on the frontend * Changing the frontend on the whole app * Matching the app to the web by implementing a light mode * Also changing all the screenshots on the report And working on impementing AI Carminder	08:00:00 11:00:00 - 19:00:00	ghazifida59

23/04/2026	Working on implementing the OCR Scanning feature. Carminder	05:00:00 11:00:00 - 16:00:00	ghazifida59
21/04/2026	Working on deleting duplicate code from /frontendService. And also trying to collecting everything and put it in apiCall.ts instead of having alot of files. And seperating it through comments Carminder	03:00:00 17:00:00 - 20:00:00	ghazifida59
21/04/2026	Working on the report, writing about: Conclusion chapter. Also the limitations about the authentication and car registering Carminder	06:00:00 09:00:00 - 15:00:00	ghazifida59
20/04/2026	Working on the report and also working on improving the frontend for the app Carminder	09:00:00 09:00:00 - 18:00:00	ghazifida59
11/03/2026	Working on the report and fixing issues with testflight. Carminder	05:00:00 11:00:00 - 16:00:00	ghazifida59
09/03/2026	Working on implementing the export PDF function. And making user stories Carminder	05:00:00 11:00:00 - 16:00:00	ghazifida59
18/02/2026	Working on the apple login and faceID login Carminder	05:00:00 09:00:00 - 14:00:00	ghazifida59
17/02/2026	Working on implementing notifications. Set up Github Actions Carminder	05:00:00 09:00:00 - 14:00:00	ghazifida59
16/02/2026	Documentation and implementing forgot password/reset password Carminder	05:00:00 09:00:00 - 14:00:00	ghazifida59
11/02/2026	Documentation for the Web, Method, Discussion and Results Carminder	08:00:00 07:00:00 - 15:00:00	ghazifida59
10/02/2026	Working on the report, Theoretical Background and Authentication/Autorization Method Carminder	08:00:00 07:00:00 - 15:00:00	ghazifida59
09/02/2026	Working on the report, Theoretical Background Carminder	05:00:00 07:00:00 - 12:00:00	ghazifida59
04/02/2026	Documentation, Writing the Acknowledgments, The Abstract and the Theoretical background Carminder	07:00:00 08:00:00 - 15:00:00	ghazifida59
02/02/2026	Implementing the web Carminder	06:00:00 15:00:00 - 21:00:00	ghazifida59

E.3 Ayman Self assessment and Timesheet

Over the course of the Carminder project I logged a total of approximately **147.75 hours** between January and May 2026. This self assessment reflects on the contributions I made, what I learned, and the challenges I encountered along the way.

E.3.1 My Contributions

My work spanned the full project period and covered several different areas.

Project initiation (January 2026). I took part in all early team meetings, where we defined the problem statement, decided on tools and architecture, and divided responsibilities. I also read documentation independently to get a better understanding of the technologies we planned to use, and contributed to early-stage planning and feature definition.

Development and debugging (February 2026). A significant part of this period was spent on setting up the project and resolving environment problems. I investigated errors and bugs, tried different approaches when a feature did not work as expected, read about security considerations relevant to our system, and explored how Railway is used to host and deploy our application.

Frontend work (April 2026). I made adjustments to the web frontend, contributing to the user-facing side of the application. This gave me practical experience with how the client layer interacts with the backend and reinforced my understanding of the thin-client architecture we had chosen.

Security research and report writing (March–April 2026). I researched our thin-client architecture and analysed its security trade-offs in depth. This work formed the basis for the security trade-offs section of the report, where I examined how centralising validation and authorization in the backend reduces client-side risk but also concentrates responsibility at the backend boundary. I analysed the implications of storing the Firebase ID token in `localStorage` on the web client, compared our shared REST backend against a Backend-for-Frontend design, and discussed OWASP guidance on object-level authorization, token verification, and endpoint protection. I also wrote the social perspective chapter, drawing on McLuhan’s argument that a technology’s significance lies in how it reshapes human activity rather than in its content alone. I used this framing to argue that CarMinder changes how maintenance history is documented and how trust is established in second-hand vehicle transactions. I further drew on Latour’s actor-network theory to explain that trust in a vehicle transaction is produced not by the application alone, but through the interaction between users, technical design, recorded events, and the buyer’s interpretation of that history. Jonas’ ethics of responsibility was used to highlight that a system influencing human decisions also expands the responsibility of those who design it.

Finalisation and presentation (May 2026). In the final weeks I refined the social perspective sections, added sources to the AI-assistance part, corrected grammar and academic language throughout the report, verified citations, and fixed remaining errors. I also prepared thoroughly for the oral examination by reviewing the presentation with the team and practicing answers to likely follow-up questions.

E.3.2 Strengths

- **Depth in written contributions.** The security trade-offs section and the social perspective chapter both required reading primary sources, synthesizing arguments, and connecting theoretical frameworks to concrete design decisions in the CarMinder system.

- **Breadth across the project.** I contributed in multiple areas, environment setup, frontend adjustment, security research, academic writing, which gave me a broad understanding of the system as a whole.
- **Consistent engagement.** Time was logged every month, with sustained effort across the full semester rather than concentrated only at the end.
- **Quality focus in written work.** Multiple sessions were dedicated to reviewing grammar, correcting academic language, and verifying sources, reflecting a commitment.

E.3.3 Challenges

- **Uneven distribution of hours.** The workload peaked in April (≈ 54 h) after a lighter period in March (≈ 13.25 h). More consistent weekly effort would have reduced pressure during the final reporting phase.
- **Getting started with the report.** Understanding what to include and how to structure the academic writing required consulting other students and spending several sessions just orienting myself. An agreed outline earlier in the project would have helped.

E.3.4 Learning Outcomes

This project gave me practical experience with a full-stack cross-platform application developed in a team setting. I deepened my understanding of security in thin-client architectures, became familiar with cloud deployment via Railway, and improved my ability to write academically rigorous technical documentation. The security trade-offs analysis in particular sharpened my ability to compare architectural patterns against each other using concrete criteria from OWASP and IETF guidance.

The social perspective work broadened my view of what a technical system actually does in society. Applying frameworks from McLuhan, Latour, and Jonas to CarMinder helped me see the system not only as a technical artefact but as something that reshapes documentation practices, trust relationships, and potentially long-term vehicle use and sustainability. This is a perspective I will carry into future projects.

On a personal level, I became more confident working across different parts of a project, from implementation to research to academic writing, and more deliberate in preparing for high-stakes situations such as the oral examination.

Detailed report

01/01/2026 - 31/12/2026



Total: 147:45:00 Billable: 00:00:00 Amount: 0,00 USD

Date	Description	Duration	User
11/05/2026	Refining the self assessment, gathering files to be delivered, and fixing the report. Carminder	05:00:00 12:00:00 - 17:00:00	mohammedal
08/05/2026	Meeting with supervisor, fixing errors from overleaf, and making self assessment. Carminder	05:00:00 10:00:00 - 15:00:00	mohammedal
07/05/2026	Practiced the questions that might come after the presentation. Carminder	05:00:00 11:00:00 - 16:00:00	mohammedal
06/05/2026	Went through the presentation and gave a feedback to the team. Carminder	04:00:00 11:00:00 - 15:00:00	mohammedal
05/05/2026	Started preparing answers for possible questions. Carminder	04:00:00 10:00:00 - 14:00:00	mohammedal
02/05/2026	Worked on redoing the social perspective sections, added some sources to the AI assistance part, and refined some sections of the report. Carminder	03:00:00 15:00:00 - 18:00:00	mohammedal
29/04/2026	Meeting with the supervisor and worked a little bit on the report fixing some misspellings. Carminder	02:30:00 12:30:00 - 15:00:00	mohammedal
22/04/2026	Wrote down social perspective parts. Carminder	04:00:00 11:00:00 - 15:00:00	mohammedal
21/04/2026	wrote down the security trade offs part. Carminder	04:00:00 11:00:00 - 15:00:00	mohammedal
20/04/2026	Did some research on the security trade offs. Carminder	02:00:00 11:00:00 - 13:00:00	mohammedal
17/04/2026	Meeting with the supervisor, fixed some errors based on the feedback from the supervisor. Carminder	03:15:00 11:45:00 - 15:00:00	mohammedal
15/04/2026	Reviewed thesis chapters for grammar and corrected academic language issues. Carminder	02:00:00 10:00:00 - 12:00:00	mohammedal
14/04/2026	Fixed errors on the report, and started researching about social perspective part. Carminder	03:00:00 11:00:00 - 14:00:00	mohammedal

13/04/2026	Meeting with the team, divided tasks and fixed some errors on the report. Carminder	02:00:00 17:00:00 - 19:00:00	mohammedal
12/04/2026	Fixed some sources, went through some documents for verifying the citation of the LLM's used in the project. fixed some errors in the report. Carminder	03:30:00 13:00:00 - 16:30:00	mohammedal
10/04/2026	Meeting with the supervisor then with the team. Carminder	01:30:00 11:30:00 - 13:00:00	mohammedal
09/04/2026	Started researching our thin client system and comparing tradeoffs in terms of security. Carminder	07:00:00 09:00:00 - 16:00:00	mohammedal
08/04/2026	Went over the tests, and tried to run and debug some of them. Carminder	06:00:00 09:00:00 - 15:00:00	mohammedal
07/04/2026	Made some adjustments in the website frontend. Carminder	07:30:00 09:00:00 - 16:30:00	mohammedal
06/04/2026	Started with getting a better overview and fixed the report on overleaf. Carminder	05:45:00 15:15:00 - 21:00:00	mohammedal
25/03/2026	Talked with other students to get ideas and advice for writing the report. Carminder	03:30:00 09:30:00 - 13:00:00	mohammedal
24/03/2026	Worked a bit on writing parts of the report. Carminder	02:30:00 09:30:00 - 12:00:00	mohammedal
23/03/2026	Tried to understand what we should include in the report. Carminder	03:30:00 09:30:00 - 13:00:00	mohammedal
13/03/2026	Started gathering the assigned tasks in order to be on track, and plan accordingly for working. Carminder	02:00:00 15:30:00 - 17:30:00	mohammedal
13/03/2026	Online meeting with supervisor. Carminder	00:45:00 11:45:00 - 12:30:00	mohammedal
06/03/2026	Online meeting with supervisor. Carminder	01:00:00 10:30:00 - 11:30:00	mohammedal
17/02/2026	Looked at how Railway works and how our app is hosted there. Carminder	04:30:00 11:30:00 - 16:00:00	mohammedal
16/02/2026	Read a bit about security related to our system. Carminder	06:30:00 10:30:00 - 17:00:00	mohammedal
13/02/2026	Spent time trying to understand errors and bugs. Carminder	03:00:00 09:00:00 - 12:00:00	mohammedal
12/02/2026	Online meeting, assigned different tasks for each other while discussing the algorithm scoring system. Carminder	01:00:00 12:30:00 - 13:30:00	mohammedal

11/02/2026	Tried different ways to solve a feature we are working on. Carminder	05:30:00 08:30:00 - 14:00:00	mohammedal
10/02/2026	Worked on setting up the project and fixing setup problems. Carminder	08:00:00 09:15:00 - 17:15:00	mohammedal
06/02/2026	Meeting with halvard first then the team, we discussed the problemstatement, and how the structure of the backend should be. Carminder	01:00:00 11:00:00 - 12:00:00	mohammedal
28/01/2026	Online meeting, discussed the problem statement. Carminder	02:00:00 15:00:00 - 17:00:00	mohammedal
23/01/2026	Meeting with supervisor and getting the first feedback. Carminder	00:30:00 12:30:00 - 13:00:00	mohammedal
22/01/2026	Tried to figure out how frontend and backend connect together. Carminder	03:00:00 16:00:00 - 19:00:00	mohammedal
22/01/2026	Third meeting discussed how to divide tasks. Carminder	02:30:00 12:30:00 - 15:00:00	mohammedal
16/01/2026	Read some documentation to understand the tools we are using better. Carminder	07:00:00 10:00:00 - 17:00:00	mohammedal
15/01/2026	Second meeting, discussed which tools to use and the scope of the project. Carminder	03:00:00 12:30:00 - 15:30:00	mohammedal
14/01/2026	Online meeting with team members and preparing questions for the second meeting Carminder	02:00:00 12:00:00 - 14:00:00	mohammedal
08/01/2026	First meeting and discussing the project and its features. Carminder	03:00:00 14:00:00 - 17:00:00	mohammedal
07/01/2026	Contributed to early stage planning and feature definition of the system. Carminder	02:00:00 11:00:00 - 13:00:00	mohammedal

Appendix F

Git-log

F.1 Git-log for the project

This appendix contains the exported Git-log from the CarMinder repository. The log is included as supplementary evidence of the development process and shows commits related to authentication, frontend development, backend integration, testing, deployment, OBD-II, OCR, PDF export, notifications, and report-related work.

The Git-log was exported using the following command:

```
git log --pretty=format:"%h - %an, %ad : %s" --date=short
```

F.2 Exported Git-log

```
cf32416 - Ghazi, 2026-05-11 : Merge pull request #45 from GH4ZII/IncidentReportPictures
904a4fa - GH4ZII, 2026-05-11 : Refactor incident photo display in CarTimelineScreen
cf32416 - Ghazi, 2026-05-11 : Merge pull request #45 from GH4ZII/IncidentReportPictures
904a4fa - GH4ZII, 2026-05-11 : Refactor incident photo display in CarTimelineScreen
13b2b9a - Ghazi, 2026-05-11 : Merge pull request #44 from GH4ZII/IncidentReportPictures
a0b98c9 - GH4ZII, 2026-05-11 : Enhance incident reporting with photo viewing and upload error handling
ff4f91d - GH4ZII, 2026-05-11 : Add image upload functionality to incident reports
17e97a7 - Evan Belal, 2026-05-08 : fixing validation
131e45a - Evan Belal, 2026-05-08 : password validation
b699a47 - Evan Belal, 2026-05-08 : Fix API contract validation gaps found by Schemathesis
f55e4ea - Ghazi, 2026-05-07 : Merge pull request #43 from GH4ZII/biometrics
1e9dcd6 - GH4ZII, 2026-05-07 : Refactor AuthContext to retain biometric preference on sign-out
cb2a274 - GH4ZII, 2026-05-07 : Implement biometric authentication support and update Face ID usage description
9512767 - Evan Belal, 2026-04-30 : added fuel consumption metric and error code explanation for toyota yaris
a893401 - Evan Belal, 2026-04-29 : updated import linter
65838c3 - Evan Belal, 2026-04-29 : added the test to the the worflow
7716bc7 - Evan Belal, 2026-04-29 : updated the unti for the cooleant temprature
916f03b - GH4ZII, 2026-04-29 : Refactor UI styles for improved theming and consistency
efe24be - Evan Belal, 2026-04-27 : Merge pull request #42 from GH4ZII/upgrade_obd
edb7cb6 - Evan Belal, 2026-04-27 : fix disconnecting and bluetooth listing of devices
fd62b64 - Evan Belal, 2026-04-26 : Merge pull request #41 from GH4ZII/upgrade_obd
cd5e054 - Evan Belal, 2026-04-26 : upgrade UI
7a8ac39 - Evan Belal, 2026-04-26 : fix duplicated issues and relevant test
4795cc1 - GH4ZII, 2026-04-26 : Add Tesseract OCR dependencies for pytesseract support
```

3f3cf79 - Ghazi, 2026-04-25 : Merge pull request #40 from GH4ZII/webFrontend

016898d - GH4ZII, 2026-04-25 : Refactor authentication screens to unify theming and improve styling

f85289b - GH4ZII, 2026-04-25 : Enhance theming and layout consistency across multiple screens

b0b78c9 - GH4ZII, 2026-04-25 : Implement "Remember Me" functionality in LoginScreen

349fae8 - GH4ZII, 2026-04-24 : Add Google Sign-In support and update AuthContext error handling

cc6e601 - GH4ZII, 2026-04-24 : Enhance styling and theming in maintenance and OBD screens

5874123 - GH4ZII, 2026-04-24 : Refactor layout and styling for improved consistency and usability

a17f381 - GH4ZII, 2026-04-24 : Enhance theming and styling across the application

6a9528c - Evan Belal, 2026-04-24 : Merge pull request #39 from GH4ZII/testing

63e7eaa - Evan Belal, 2026-04-24 : final fix

3fac64d - Evan Belal, 2026-04-24 : Merge pull request #38 from GH4ZII/testing

3b41f55 - Evan Belal, 2026-04-24 : Update types.ts

4b26985 - Evan Belal, 2026-04-24 : fix for the obd scanner

47658d2 - GH4ZII, 2026-04-23 : Refactor AddMaintenanceEventScreen to remove time handling logic

ddba6ad - GH4ZII, 2026-04-23 : OCR Scanning feature beta

0d46cb3 - Evan Belal, 2026-04-22 : testing

02d17b3 - GH4ZII, 2026-04-21 : Remove unused API modules to streamline the frontend services

1928941 - GH4ZII, 2026-04-21 : Remove authApi module to streamline authentication handling

0bef09d - Evan Belal, 2026-04-21 : Make web tsconfig independent from Expo

39e2c30 - Evan Belal, 2026-04-21 : add a ci for testing

f2697c2 - Ghazi, 2026-04-20 : Merge pull request #37 from GH4ZII/frontend

8ec1b59 - GH4ZII, 2026-04-20 : Refactor UI components and styles for improved consistency and user experience

82758c1 - GH4ZII, 2026-04-20 : Refactor ForgotPassword and Login screens for improved error handling and ...

UI updates

bbee0cc - GH4ZII, 2026-04-20 : Add token expiration handling in AuthContext

6f4cdf3 - Evan Belal, 2026-04-16 : Delete backend-python/SCORING.md

113602d - GH4ZII, 2026-04-13 : Update package-lock.json to add react-native-svg and new dependencies, and ...

mark several packages as peer dependencies

00d95f5 - Evan Belal, 2026-04-12 : Merge pull request #35 from GH4ZII/elm_connection

428ed48 - Evan Belal, 2026-04-12 : fix for connecting to a elm

2613a04 - Evan Belal, 2026-04-10 : Merge pull request #34 from GH4ZII/web-accident_reporting

743f937 - Evan Belal, 2026-04-10 : Merge branch 'main' into web-accident_reporting

9aa85e0 - Evan Belal, 2026-04-10 : added accident reporting to the web client

288f6ef - Ayman Allaw, 2026-04-07 : Maintenance timeline changed

e63a104 - Evan Belal, 2026-03-19 : Merge pull request #33 from GH4ZII/ownership_transfer

46bf177 - Evan Belal, 2026-03-19 : changes

29c7bba - Evan Belal, 2026-03-19 : changes

62cac9e - Evan Belal, 2026-03-19 : added retiring a car and to no longer to be registered

ddedd24 - Evan Belal, 2026-03-19 : only one car be registd to a user with vin number no more duplicated ...

profile

6bc8cb9 - Evan Belal, 2026-03-18 : Merge pull request #32 from GH4ZII/ownership_transfer

5ca1c61 - Evan Belal, 2026-03-18 : implemented transfer of ownership

24f04a2 - Evan Belal, 2026-03-18 : added more testing for import and functions

499fd72 - GH4ZII, 2026-03-17 : Add expo-file-system dependency and refactor PDF export logic

040034b - GH4ZII, 2026-03-17 : Refactor PDF export functionality and improve error handling

f0945c3 - GH4ZII, 2026-03-16 : Add xhtml2pdf to requirements for PDF generation support

f2a0f20 - Ghazi, 2026-03-16 : Merge pull request #31 from GH4ZII/ReimplementExportPDF

3405381 - GH4ZII, 2026-03-16 : Add PDF report generation and export functionality for car details

f4ea1c8 - Evan Belal, 2026-03-13 : multiple test functons for the backend

547ccaa - Evan Belal, 2026-03-13 : have added import-linter to check for violation of contracts

594dbff - Evan Belal, 2026-03-11 : fix sign in and sign up and added potential fix for testflight

3b48b22 - GH4ZII, 2026-03-11 : Update dependencies for `expo-print` and `expo-sharing` to specific ...
versions in package.json and package-lock.json

36410c7 - Evan Belal, 2026-03-10 : Merge pull request #30 from GH4ZII/obdscanner

a018bc5 - Evan Belal, 2026-03-10 : configuring for the i car pro adapter

dc54880 - Ghazi, 2026-03-09 : Merge pull request #29 from GH4ZII/exportPDF

3eb6349 - GH4ZII, 2026-03-09 : Add PDF export functionality for car reports

ba65a23 - Evan Belal, 2026-03-09 : Merge pull request #28 from GH4ZII/obdscanner

a066620 - Evan Belal, 2026-03-09 : create obd own screen

b29a4f0 - Evan Belal, 2026-03-09 : saving obd scanner to database, reflecting on web and phone UI

589328a - Evan Belal, 2026-03-09 : added native connection to bluetooth to connect to the obd scanner and ...
read data

4d40098 - GH4ZII, 2026-03-05 : Update README.md to reflect project name change and enhance setup ...
instructions for CarMinder app

c782cf1 - Evan Belal, 2026-03-01 : Merge pull request #27 from GH4ZII/UI_Polish

ca79a31 - Evan Belal, 2026-03-01 : Delete docs directory

6402f2f - Evan Belal, 2026-03-01 : polished the UI

f5e3985 - Evan Belal, 2026-02-26 : updated web ui

ef56cbe - Evan Belal, 2026-02-26 : Merge pull request #26 from GH4ZII/scoring-algorithm

199f23c - Evan Belal, 2026-02-26 : page to explain scoring

4c43382 - Evan Belal, 2026-02-26 : added testing and seperated logic

74e7bcf - Evan Belal, 2026-02-26 : added better error handling

4ea655d - Evan Belal, 2026-02-26 : added a scoring alghthorim for the cars

c2a7533 - Evan Belal, 2026-02-23 : Merge pull request #25 from GH4ZII/mobile-dimesion

8db61d2 - Evan Belal, 2026-02-23 : fixed dimenstion for the mobile UI

6301ba2 - Ghazi, 2026-02-18 : Merge pull request #24 from GH4ZII/faceID&AppleLogin

7ab4380 - GH4ZII, 2026-02-18 : Merge branch 'faceID&AppleLogin' of https://github.com/GH4ZII/CarMinder ...
into faceID&AppleLogin

2d7d8b1 - GH4ZII, 2026-02-18 : Refactor authentication storage keys for SecureStore integration

c98ba35 - Ghazi, 2026-02-18 : Merge pull request #23 from GH4ZII/faceID&AppleLogin

2660618 - Ghazi, 2026-02-18 : Merge branch 'main' into faceID&AppleLogin

038d283 - GH4ZII, 2026-02-18 : Add Apple authentication availability check in login screen

530bdea - GH4ZII, 2026-02-18 : Add Apple authentication and biometric login support

09cc312 - Evan Belal, 2026-02-18 : refactored the api call to seperate file for easier debugging in the ...
future follow the same template when wanting to implement the same

ee649bb - Ghazi, 2026-02-17 : Create notifications-cron.yml

ce32fa8 - GH4ZII, 2026-02-16 : Add password reset functionality to API

ca68062 - Ghazi, 2026-02-16 : Merge pull request #22 from GH4ZII/reset/forgot-password

08b4cdb - GH4ZII, 2026-02-16 : Add pydantic dependency for email validation

96da4b6 - Ghazi, 2026-02-16 : Merge pull request #21 from GH4ZII/reset/forgot-password

b15cecf - GH4ZII, 2026-02-16 : Add forgot password functionality

c6f0506 - Evan Belal, 2026-02-13 : Merge pull request #20 from GH4ZII/incident-reporting

dfb4d41 - Evan Belal, 2026-02-12 : have implemented searching functionality for other car histroy and ...
also implemented reporting accident on car

dd229c6 - Evan Belal, 2026-02-11 : implemented log in

b1bc6d2 - Evan Belal, 2026-02-11 : create first draft of the website

f7d68f7 - Evan Belal, 2026-02-11 : gitignore

9abcf10 - Evan Belal, 2026-02-11 : Add car management pages and calendar functionality

691230e - Evan Belal, 2026-02-11 : bugs fixed for simulator

a9c028c - Evan Belal, 2026-02-09 : Merge pull request #15 from GH4ZII/claude/wizardly-villani

Offbb94 - Evan Belal, 2026-02-09 : implemented the calendar for repair types

8a58352 - Evan Belal, 2026-02-09 : Merge pull request #14 from GH4ZII/claude/wizardly-villani

a847965 - Evan Belal, 2026-02-09 : changes

```

dce4ad2 - Evan Belal, 2026-02-09 : Merge pull request #13 from GH4ZII/interval-reminder
e4a2cc9 - Evan Belal, 2026-02-09 : added a inveral reminder based on previous ones
594e6e6 - Evan Belal, 2026-02-09 : fixed maitence add screen and saving to database
5fb47b3 - Evan Belal, 2026-02-09 : Merge remote main branch
09ddf74 - Evan Belal, 2026-02-04 : Merge pull request #12 from GH4ZII/maitenceevent2
fa9bcd1 - Evan Belal, 2026-02-04 : Add comprehensive debugging to backend
101dbb4 - Ghazi, 2026-02-04 : Merge pull request #11 from GH4ZII/website
756a712 - GH4ZII, 2026-02-03 : Remove unused custom hook `useApi` to streamline codebase
96ba60f - GH4ZII, 2026-02-02 : Add web application structure with Vite and React
e239277 - Evan Belal, 2026-01-31 : to connect to the backed on simulator
79eee7d - Evan Belal, 2026-01-31 : Merge pull request #10 from GH4ZII/maitance-events
3d1dd95 - Evan Belal, 2026-01-31 : Merge branch 'main' into maitance-events
5a8bf8f - Evan Belal, 2026-01-31 : Merge pull request #9 from GH4ZII/migrate-firebase-auth
6b727be - GH4ZII, 2026-01-30 : Remove Firebase service account JSON file for enhanced security and compliance
ed3cccd - GH4ZII, 2026-01-30 : Enhance authentication module with detailed comments
31b6160 - GH4ZII, 2026-01-30 : Enhance profile functionality and API integration
25dedad - GH4ZII, 2026-01-29 : Refactor authentication and API integration
1cf1bf9 - Evan Belal, 2026-01-29 : have added manually adding of maitance events with its own screen
faea1eb - Evan Belal, 2026-01-29 : Merge pull request #8 from GH4ZII/userProfile
6c2884f - Evan Belal, 2026-01-29 : profile section for the user where he can add delete car to his profile
7d47056 - Evan Belal, 2026-01-29 : Merge pull request #7 from GH4ZII/migrating-to-fastapi
a1b80e3 - Evan Belal, 2026-01-28 : updates read me
ded3e44 - Evan Belal, 2026-01-28 : have set the fastapi backend and have migrated the logic for getting a ...
    car info and saving database from the mobile logic to the backed
e50cc2f - GH4ZII, 2026-01-21 : Update dependencies and refactor Google Sign-In integration
b6641fb - Evan Belal, 2026-01-20 : Merge pull request #6 from GH4ZII/homescreen
d696963 - Evan Belal, 2026-01-20 : saving car to user database
7d2a6c9 - Evan Belal, 2026-01-20 : Merge pull request #5 from GH4ZII/homescreen
3d335ef - Evan Belal, 2026-01-20 : extracting car data through vegvesen api
380d8a0 - Ghazi, 2026-01-18 : Merge pull request #4 from GH4ZII/firebase-auth
9da238b - GH4ZII, 2026-01-18 : Refactor authentication flow and improve layout structure
888db27 - Ghazi, 2026-01-18 : Merge pull request #3 from GH4ZII/firebase-auth
3b99ef9 - GH4ZII, 2026-01-17 : Add Google Sign-In functionality and update app configuration
d911a8b - GH4ZII, 2026-01-17 : Implement login and signup functionality with "Remember Me" feature. ...
    Updated AuthContext to include user name during signup and added AsyncStorage for email persistence. ...
    Refactored login screen to enhance user experience.
1f677a6 - Ghazi, 2026-01-15 : Merge pull request #2 from GH4ZII/firebase-auth
d6367b2 - GH4ZII, 2026-01-15 : Clean Expo Template Code, Deleted files that were not neccsissary
115893f - Ghazi, 2026-01-15 : Merge pull request #1 from GH4ZII/firebase-auth
b79972d - GH4ZII, 2026-01-15 : Authentication with Firebase
8d7ede1 - GH4ZII, 2026-01-15 : Initial commit

```

F.3 Schemathesis Commands

This appendix lists the exact Schemathesis commands used to produce the baseline and final validation runs reported in Section 4.4.4. Both commands target the deployed Railway endpoint, use the same random seed, and write reports in JUnit, HAR, and NDJSON formats to disk for archival. A condensed deliverable archive containing the run summary (schemathesis-run-2026-05-08.md), the baseline JUnit report (initial-baseline-junit.xml), and the final JUnit report (final-validation-junit.xml) is provided at docs/schemathesis-deliverable.zip.

Listing F.1: Schemathesis commands for the baseline and final validation runs against the deployed Railway API.

```
# Baseline run
schemathesis run \
  https://carminder-production.up.railway.app/openapi.json \
  --checks all \
  -n 100 \
  --seed 20260508 \
  --report ndjson,junit,har \
  --report-dir docs/schemathesis-report \
  --no-color

# Final validation run (same seed, post-fix)
schemathesis run \
  https://carminder-production.up.railway.app/openapi.json \
  --checks all \
  -n 100 \
  --seed 20260508 \
  --report ndjson,junit,har \
  --report-dir docs/schemathesis-report-final \
  --no-color
```

Bibliography

- [1] Marshall McLuhan. *Understanding Media - The Extensions of Man*. eng. 1964. URL: <http://archive.org/details/ETC0624> (visited on 04/22/2026).
- [2] OpenAI. (2026). *ChatGPT*. version: April 12 [Large Language Model]. URL: <https://chatgpt.com/> (visited on 04/12/2026).
- [3] Anthropic. (2026). *Claude*. version: April 12 [Large Language Model]. URL: <https://claude.ai/> (visited on 04/12/2026).
- [4] International Organization for Standardization. *Asset management — Overview, principles and terminology*. Tech. rep. ISO 55000:2014. ISO, 2014. URL: <https://www.iso.org/standard/55088.html>.
- [5] Pacific Northwest National Laboratory. *An Advanced Maintenance Approach: Reliability Centered Maintenance*. Tech. rep. 2022. URL: <https://www.pnnl.gov/projects/om-best-practices/advanced-maintenance-approach-reliability-centered-maintenance?>
- [6] Gregor Hohpe and Bobby Woolf. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Boston, MA: Addison-Wesley, 2003. ISBN: 978-0321200686.
- [7] Ian Sommerville. *Software Engineering*. 10th. Pearson, 2016. ISBN: 978-0133943030.
- [8] Chris Richardson. *Pattern: Event Sourcing*. Describes event sourcing as persisting the state of a business entity as a sequence of state-changing events; new events are appended whenever state changes, and event sourcing provides a reliable audit log of all changes, enabling temporal queries. 2026. URL: <https://microservices.io/patterns/data/event-sourcing.html> (visited on 03/01/2026).
- [9] T.Y. Chen, S.C. Cheung, and S.M. Yiu. *Metamorphic Testing: A New Approach for Generating Next Test Cases*. Tech. rep. HKUST-CS98-01. Department of Computer Science, Hong Kong University of Science and Technology, 1998.
- [10] Roy T. Fielding, Mark Nottingham, and Julian Reschke. *HTTP Semantics*. RFC 9110. Internet Engineering Task Force, 2022. URL: <https://www.rfc-editor.org/rfc/rfc9110>.
- [11] W. P. Stevens, G. J. Myers, and L. L. Constantine. “Structured Design.” In: *IBM Systems Journal* 13.2 (1974), pp. 115–139.
- [12] Martin Fowler. *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley, 2002. ISBN: 978-0321127426.
- [13] Ganesh Pagade. *Difference Between Cohesion and Coupling*. Discusses cohesion as the degree to which elements inside a module belong together and high cohesion improves readability, maintainability and testability defines coupling as the degree of interdependence between modules and explains that low coupling allows modules to be developed, tested, and modified independently. 2025. URL: <https://www.baeldung.com/cs/cohesion-vs-coupling> (visited on 03/01/2026).
- [14] Roy Thomas Fielding. “Architectural Styles and the Design of Network-based Software Architectures.” PhD thesis. University of California, Irvine, 2000. URL: https://roy.gbiv.com/pubs/dissertation/fielding_dissertation.pdf.

- [15] Robert Orfali, Dan Harkey, and Jeri Edwards. “The Essential Client/Server Survival Guide.” In: (1996).
- [16] Tapronix LLC. *MyAutoLog – Car Maintenance Tracker & Service Log App*. <https://carmaintenance.app/>. Official product website. Accessed: 2026-05-01. 2026.
- [17] Tapronix LLC. *Car Maintenance App: MyAutoLog*. Apple App Store. Available: <https://apps.apple.com/us/app/car-maintenance-app-myautolog/id6748665282>. Accessed: 2026-05-01. 2026.
- [18] Mobifolio. *Simply Auto: Car Maintenance and Mileage Tracker App*. <https://simplyauto.app/>. Official product website. Accessed: 2026-05-01. 2026.
- [19] Mobifolio. *Simply Auto: Car Maintenance*. Google Play Store. Available: <https://play.google.com/store/apps/details?id=mrigapps.andriod.fuelcons>. Accessed: 2026-05-01. 2026.
- [20] CARFAX, Inc. *CARFAX Car Care App*. Google Play Store. Available: <https://play.google.com/store/apps/details?id=com.carfax.mycarfax>. Accessed: 2026-01-15. 2024.
- [21] CARFAX, Inc. *CARFAX Car Care – Apple App Store*. Apple App Store. Available: <https://apps.apple.com/ca/app/carfax-car-care/id552472249>. Accessed: 2026-01-15. 2024.
- [22] FIXD Automotive Inc. *FIXD OBD2 Scanner – Vehicle Health Monitor*. Google Play Store. Available: <https://play.google.com/store/apps/details?id=com.fixdapp.two>. Accessed: 2026-01-15. 2024.
- [23] The Track Ahead. *FIXD Bluetooth OBD2 Scanner Review*. <https://thetrackahead.com/reviews/tool-equipment-product-reviews/scan-tools/fixd-scan-tool-review/>. Accessed: 2026-01-15. 2024.
- [24] T.Y. Chen et al. “Metamorphic Testing: A Review of Challenges and Opportunities.” In: *ACM Computing Surveys* 51 (2018). DOI: [10.1145/3143561](https://doi.org/10.1145/3143561).
- [25] Zac Hatfield-Dodds and Dmitry Dygalo. “Deriving Semantics-Aware Fuzzers from Web API Schemas.” In: *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*. ICSE ’22. New York, NY, USA: Association for Computing Machinery, 2022. DOI: [10.1145/3510454.3528637](https://doi.org/10.1145/3510454.3528637).
- [26] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. “Hints on Test Data Selection: Help for the Practicing Programmer.” In: *Computer* 11.4 (1978). DOI: [10.1109/C-M.1978.218136](https://doi.org/10.1109/C-M.1978.218136).
- [27] Yue Jia and Mark Harman. “An Analysis and Survey of the Development of Mutation Testing.” In: *IEEE Transactions on Software Engineering* 37.5 (2011). DOI: [10.1109/TSE.2010.62](https://doi.org/10.1109/TSE.2010.62).
- [28] David J. Anderson. *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press, 2010. ISBN: 978-0984521401.
- [29] Google LLC. *Firebase Authentication*. 2024. URL: <https://firebase.google.com/docs/auth> (visited on 04/12/2026).
- [30] OWASP Security. *HTML5 Security - OWASP Cheat Sheet Series*. URL: https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html? (visited on 04/21/2026).
- [31] OWASP API Security. *API1:2023 Broken Object Level Authorization - OWASP API Security Top 10*. URL: <https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/?> (visited on 04/21/2026).
- [32] OWASP Security. *HTML5 Security - OWASP Cheat Sheet Series*. URL: https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html? (visited on 04/21/2026).
- [33] Supabase. *Row Level Security | Supabase Docs*. en. Apr. 2026. URL: <https://supabase.com/docs/guides/database/postgres/row-level-security> (visited on 04/21/2026).

- [34] Bruno Latour. “Introduction: How to Resume the Task of Tracing Associations*.” en. In: *Reassembling the Social*. Oxford University PressOxford, July 2005, pp. 1–17. DOI: [10.1093/oso/9780199256044.003.0001](https://doi.org/10.1093/oso/9780199256044.003.0001). URL: <https://academic.oup.com/book/52349/chapter/421264933> (visited on 04/22/2026).
- [35] Hans Jonas. “Technology and Responsibility: Reflections on the New Tasks of Ethics.” en. In: *Ethics and Emerging Technologies*. Ed. by Ronald L. Sandler. London: Palgrave Macmillan UK, 2014, pp. 37–47. DOI: [10.1057/9781137349088_3](https://doi.org/10.1057/9781137349088_3). URL: http://link.springer.com/10.1057/9781137349088_3 (visited on 04/22/2026).